

Litmus User Guide

Product Version 26.10

May 2026

© 2026 Cadence Design Systems, Inc.
Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Open SystemC, Open SystemC Initiative, OSCI, SystemC, and SystemC Initiative are trademarks or registered trademarks of Open SystemC Initiative, Inc. in the United States and other countries and are used with permission.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522.

All other trademarks are the property of their respective holders.

Restricted Permission: This publication is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this publication, or any portion of it, may result in civil and criminal penalties. Except as expressly permitted below, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, including automated processes, training or services involving a large language model, foundation model, deep machine learning, generative artificial intelligence, or any other similar process or technology, without prior written permission from Cadence. Unless otherwise agreed to by Cadence in writing, customers are granted permission to print one (1) hard copy of this publication subject to the following conditions:

1. The publication may be used only in accordance with a written agreement between Cadence and its customer.
2. The publication may not be modified in any way.
3. Any authorized copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
4. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information. Cadence is committed to using respectful language in our code and communications. We are also active in the removal and replacement of inappropriate language from existing content. This product documentation may however contain material that is no longer considered appropriate but still reflects long-standing industry terminology. Such content will be addressed at a time when the related software can be updated without end-user impact.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

Contents

About This Manual	4
Audience	4
Conventions Used in This Manual	4
1	8
Product and Licensing Information	8
Related Topics	8
About Licensing	8
Check Out Litmus Licenses for Product Options	9
Litmus Licenses	10
Product and Licensing Overview	11
2	14
Introduction to Litmus	14
Related Topics	16
Tempus Common Timing Engine Integration	16
Litmus With Other Cadence Tools	18
3	19
Using Litmus	19
Inside the Tool	20
4	23
Installation and Startup	23
Related Topics	23
Setting up the Litmus Runtime Environment	24
Supported and Compatible Platforms	24
Temporary File Locations	25
Launching the Litmus Console	26
Completing Command Names	26
Command-Line Editing	26
Starting the Litmus Software	28
Using Litmus GUI	28
Launching the Litmus GUI	29
Exiting or Hiding the GUI	30

Accessing Litmus Documentation and Help	31
Using the Litmus man and help Commands on the Command-Line	32
Other Sources of Cadence Product Information	34
5	36
Constraints	36
Related Topics	37
Setting Constraints	37
Common Object Access Commands	39
Constraints Signoff	40
Constraint Signoff Requirements	41
Correctness	41
Completeness	42
Consistency	42
Constraint Signoff Benefits	43
Constraints Signoff Solution	44
Constraints Signoff Use Model	45
Constraints - Design Initialization	47
Design Setup with MMMC Flow	47
MMMC Setup	47
Example of MMMC Setup	48
Constraint Command Summary	49
Constraints - Analysis	51
Define Timing Intent	51
Perform Constraints Analysis	52
Constraints Checks	52
Constraints Analysis Examples	53
Constraints - Review Methodology	58
Constraint Signoff - Sample Run Script	60
6	62
Hierarchical Constraints	62
Related Topics	63
Hierarchical Constraints Consistency Checks	64
Hierarchical Constraints Signoff Requirements	65
Hierarchical Constraints Signoff Solution	66
Hierarchical Constraints Signoff Use Model	67
Hierarchical Constraints - Design Initialization	69

Hierarchical Constraints - Consistency Analysis	69
Defining Timing Intent in Hierarchical Constraints Analysis	70
Performing Hierarchical Constraints Analysis	70
Hierarchical Constraints Checks	71
Hierarchical Constraints Analysis Examples	72
Hierarchical Constraints Analysis Summary	77
Hierarchical Constraints - Consistency Review	77
Hierarchical Constraints - Consistency Review Methodology	77
Hierarchical Constraint Signoff - Sample Run Script	80
7	83
Constraints Comparison	83
Reference vs Target Constraint Consistency	83
Violation Rules	84
Constraints Comparison Checks	84
Constraints Comparison Signoff Requirements	85
Correctness	86
Completeness	86
Consistency	86
Supported Comparison Modes	86
Timing-Aware Constraint Comparison (1D2S)	86
Timing-Unaware Constraint Comparison (2SDC)	86
Running Constraint Comparison	87
Constraints Comparison Examples	88
Example 1 - create_clock	88
Example 2 - create_generated_clock	89
Example 3 - set_case_analysis	90
Example 4 - set_input_delay	92
Example 5 - set_multicycle_path	93
Example 6 - set_max_delay	95
Example 7 - set_false_path	96
Example 8 - Physically Exclusive Clock Exceptions	98
Example 9 - Comprehensive Example - 2SDC Compare	99
Constraints Comparison - Sample Run Script	102
8	104
Constraints Promotion	104
Constraint Promotion Flow	105

Setup Phase	106
Analysis Phase	106
Review Phase	107
Design Initialization and Constraint Setup	107
Reading Block-Level Constraints	107
Command Usage	108
Options	108
Examples	108
Promotion Configuration	109
Promotion Priority	109
Command Usage	109
Options	109
Promotion Strategy	110
Command Usage	110
Options	110
Examples	111
Running Constraint Promotion	112
Constraint Promotion Command	112
Command Usage	113
Options	113
Examples	113
Root Attributes Controlling Promotion	113
Available Attributes	114
Typical Use Cases	115
Constraint Promotion Results	115
Interpretation of CP-Tags	115
Detailed CP-Tag Scenarios	116
CP +1: Equivalent Constraint Exists at the Top	116
CP +2: Constraint Promoted (Missing at Top)	117
CP +3: Multiple Top-Level Candidates	117
CP +4: Conflict With Top-Level Constraint	118
CP +5: Impact on External Objects	119
Sample Run Scripts	120
9	122
Working with the GUI	122
GUI Components	123

Menu Bar	123
Rules Summary	124
Navigation Pane	125
Occurrences Table	127
Description Tab	128
GUI Features	128
Cross-Probing	128
Cross-Probing to HDL / RTL	129
Cross-Probing to Schematic	130
Cross-Probing to SDC	133
Creating Rule Filters	134
Creating Rule Filters from the Navigation Pane	134
Creating Rule Filters from the Occurrences Table	135
Add Rule Filter Dialog	136
10	138
SVA Generation for Functional Checks	138
SVA Generation Flow	138
Related Topics	139
Design Initialization for SVA	139
Generate SVA	140
Constraints for SVA	141
Generated Clock (create_generated_clock)	142
Examples	142
SFP and MCP Exceptions	144
Set Multicycle Path (set_multicycle_path)	145
Set False Path (set_false_path)	147
Set Case Analysis (set_case_analysis)	150
Set Disable Timing (set_disable_timing)	150
Examples	151
11	152
CUI Scripting Capability	152
12	154
Templates	154
RTL Signoff Template	154
Netlist Signoff Template	155

Hierarchical Check Template	155
-----------------------------	-----

About This Manual

This manual describes how to install, configure, and use the Litmus[®] software with the Stylus Common user interface.

Audience

This manual is written for experienced designers of digital integrated circuits. Such designers must be familiar with design planning, placement and routing, block implementation, chip assembly, and design verification. Designers must also have a solid understanding of UNIX and Tcl/Tk programming.

Conventions Used in This Manual

This section describes the typographic and syntax conventions used in this manual.

<i>text</i>	Indicates text that you must type exactly as shown. For example: <code>analyze_connectivity -analyze all</code>
<i>text</i>	Indicates information for which you must substitute a name or value. In the following example, you must substitute the name of a specific file for <i>configfile</i> : <code>wroute filename configfile</code>
<i>text</i>	Indicates the following: • Text found in the graphical user interface (GUI), including form names, button labels, and field names • Terms that are new to the manual, are the subject of discussion, or need special emphasis • Titles of manuals
[]	Indicates optional arguments. In the following example, you can specify none, one, or both of the bracketed arguments: <code>command [-arg1] [arg2 value]</code>

[]	Indicates an optional choice from a mutually exclusive list. In the following example, you can specify any of the arguments or none of the arguments, but you cannot specify more than one: <pre>command [arg1 arg2 arg3 arg4]</pre>
{ }	Indicates a required choice from a mutually exclusive list. In the following example, you must specify one, and only one, of the arguments: <pre>command {arg1 arg2 arg3}</pre>
{ [] [] }	Indicates a required choice of one or more items in a list. In the following example, you must choose one argument from the list, but you can choose more than one: <pre>command {[arg1] [arg2] [arg3]}</pre>
{ }	Indicates curly braces that must be entered with the command syntax. In the following example, you must type the curly braces: <pre>command arg1 {x y}</pre>
...	Indicates that you can repeat the previous argument.
.	Indicates an omission in an example of computer output or input.
<i>Command - Subcommand</i>	Indicates a command sequence, which shows the order in which you choose commands and subcommands from the GUI menu. In the following example, you choose <i>Power</i> from the menu, then <i>Power Planning</i> from the submenu, and then <i>Add Rings</i> from the displayed list: <i>Power - Power Planning - Add Rings</i> This sequence opens the <i>Add Rings</i> form..

Product and Licensing Information

Each Cadence Litmus product is sold as part of a product package. Product packages might also include product options. The options provide advanced features and capabilities, such as support for enabling distributed client servers and/or accelerating performance through increased CPU multi-threading.

Each product and product option has a corresponding license. The software uses licenses to determine the features that are available when the software runs.

Related Topics

- [About Licensing](#)
- [Check Out Litmus Licenses for Product Options](#)
- [Litmus Licenses](#)
- [Product and Licensing Overview](#)

About Licensing

The following terminology is useful in understanding licenses.

- **Base license**
The license that is checked out when the software starts. Only a full-fledged product license can be used as a base license. You cannot use a product option license as a base license to start the software.
- **Dynamic license**
A license for a product option that is not checked out until a feature provided by the product option is needed. You can check out more than one dynamic license per base license. For more information on dynamic licenses, see [Checking Out Litmus Licenses for Product Options](#).
- **Multi-CPU license**
A license that enables additional CPUs for multi-threading, Superthreading, or distributed processing. Multi-CPU licenses must be product licenses and can be checked out after the base license is checked out. You can check out more than one multi-CPU license per base license. For more information on multi-CPU licenses, for more information on multi-CPU licenses, see Innovus System Licensing and Packaging on SourceLink®.
- **Licensing Server license**
Tempus running on Linux x86 platforms is integrated with FLEXnet version 11.16.4.0. The Tempus software using this version also requires the upgraded license server version, which is integrated with FLEXnet 11.16.4.0 or higher versions. If the license server is not updated, the Tempus software will not be

able to run.

You can check the version of the licensing server by running the following commands in the terminal where the license server is installed:

```
lmgrd -v
```

```
cdslmd -v
```

Check Out Litmus Licenses for Product Options

The `litmus` command specifies the product options to be checked out when you invoke the software. The product options specified with the `-lic_startup_options` parameter are checked out immediately and the product options specified with the `-lic_options` parameter are checked out dynamically.

The use model is as follows:

```
litmus -lic_startup_options "lic1 lic2 ..."
```



If you do not want any product options to be checked out dynamically, use empty quotes with the `-lic_options` parameter, as follows:

```
litmus -lic_options "lic1 lic2 ..."
```

Litmus Licenses

When you run a command to invoke a product or product option, a license is checked out. Each product and product option has a unique license string (also called a license key). These options provide the extendibility and cost-effective access to additional advanced technologies.

The following table lists the product and product number and provides the corresponding license strings.

Name and License String	Abbreviation	Product Number	Description
Litmus Signoff <code>Litmus_Signoff</code>		LIT100	
Litmus Compare Option <code>Litmus_Compare_Opt</code>		LIT200	
Litmus Constraint Management Option <code>Litmus_Const_Mgmt_Opt</code>		LIT110	
Litmus SVA Generation Option <code>Litmus_SVA_Gen_Opt</code>		LIT120	
Litmus CPU Accelerator Option <code>Litmus_CPU_Opt</code>		LIT140	

To start any of these products, type the following UNIX/Linux command:

```
litmus
```

The version of vendor daemon (`cdslmd`) and license daemon (`lmgrd`) should be 11.11.1.1 or higher. If the daemon versions have not been updated, Tempus may fail to checkout a license:

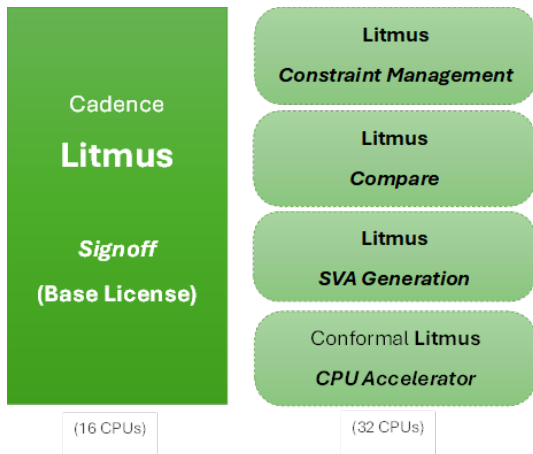
```
$ litmus
Checking out Litmus license ...
Fail to find any Litmus license...
Check your Litmus license status.
```

To fix the above error, restart the license server using the latest `cdslmd` and `lmgrd` versions included in the release tree.

Product and Licensing Overview

Litmus has the following 2 option parts:

- Shared CPU accelerator option (+16 CPUs)
- Constraint management option available on top of signoff base



The following table shows the mapping between functional licenses and Litmus features.

	SDC Lint	SDC Structural Checks	SDC Promotion / Merging	SDC Integration Hierarchical Consistency Checks	16 CPU Parallelism	Tempus Timer Integration	Stylus Common User interface
Litmus Signoff	✓	✓			✓	✓	✓
Litmus Constraint Management			✓	✓			
Litmus CPU Accelerator					✓		

The following table shows Litmus naming and licensing.

Product Number	Product Name	License Feature
LIT100	Litmus Signoff	Litmus_Signoff
LIT110	Litmus Constraint Management Option	Litmus_Const_Mgmt_Opt
LIT120	Litmus SVA Generation Option	Litmus_SVA_Gen_Opt

Litmus User Guide

Product and Licensing Information--Product and Licensing Overview

LIT140	Litmus CPU Accelerator Option	Litmus_CPU_Opt
LIT200	Litmus Compare Option	Litmus_Compare_Opt

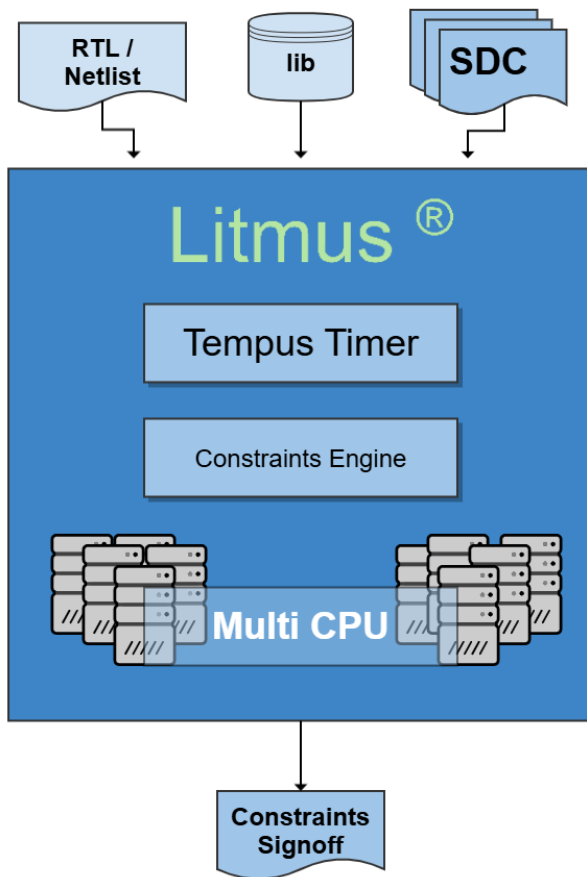
Introduction to Litmus

Litmus is the next generation of Cadence's mature Conformal Constraints Designer (CCD). It provides the fastest path to full chip constraints signoff. The major benefit of Litmus is that it can run at both RTL and gate-level to support the full implementation and signoff of both blocks and full SoCs.

Litmus has three main flows:

- Constraints signoff
- Hierarchical consistency checks
- Clock/Reset domain crossing signoff

The following figure shows an overview of the Litmus flow:



It has the following improvements over the prior generation.

- 100% Accuracy
 - Industry's first static timer (Tempus Timing) integration for Constraints Signoff.
 - Accurately models the design and the constraints using the same interpretation as signoff timer.
 - Enables you to start verification early at RTL.
- Industry-Leading Performance and Capacity
 - Built-in multi-CPU architecture.
 - The SoC-level capacity of more than 1 billion gates.
 - Scalable to larger designs.
- Smart Constraints Checking
 - Enables accurate, low-noise reporting.

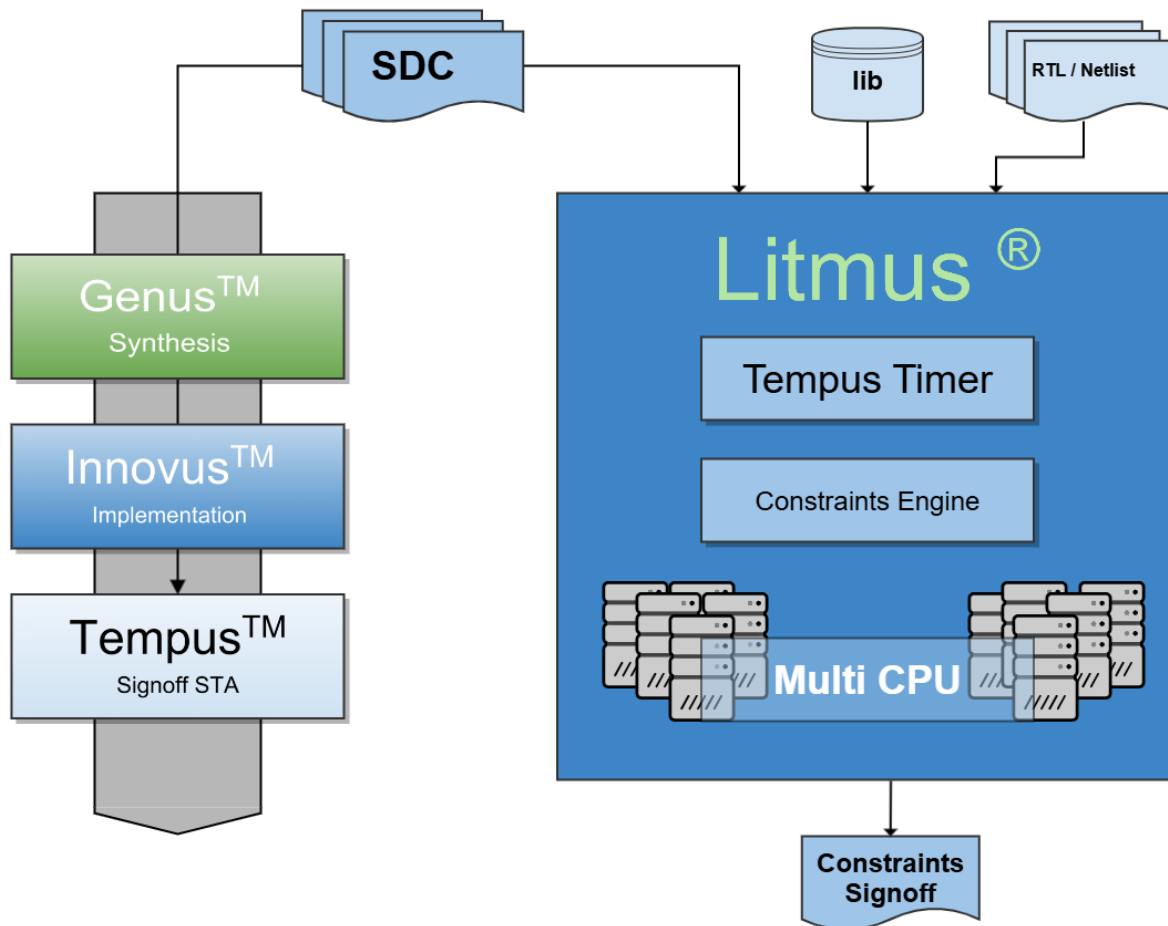
- Provides insights into the design and constraints intent.
- Provides root cause analysis for rapid signoff.
- Hierarchical consistency checking.
- Constraints comparison.

Related Topics

- [Using Litmus](#)

Tempus Common Timing Engine Integration

Litmus is closely integrated with Tempus, Common timing Engine (CTE), Innovus, and other Signoff tools. The following figure demonstrates this integration with other tools:



The benefits of this integration include the ability to share scripts or setup, have identical interpretation and unrolling of constraints, and view accurate interdependencies between constraints and timing or crossings.

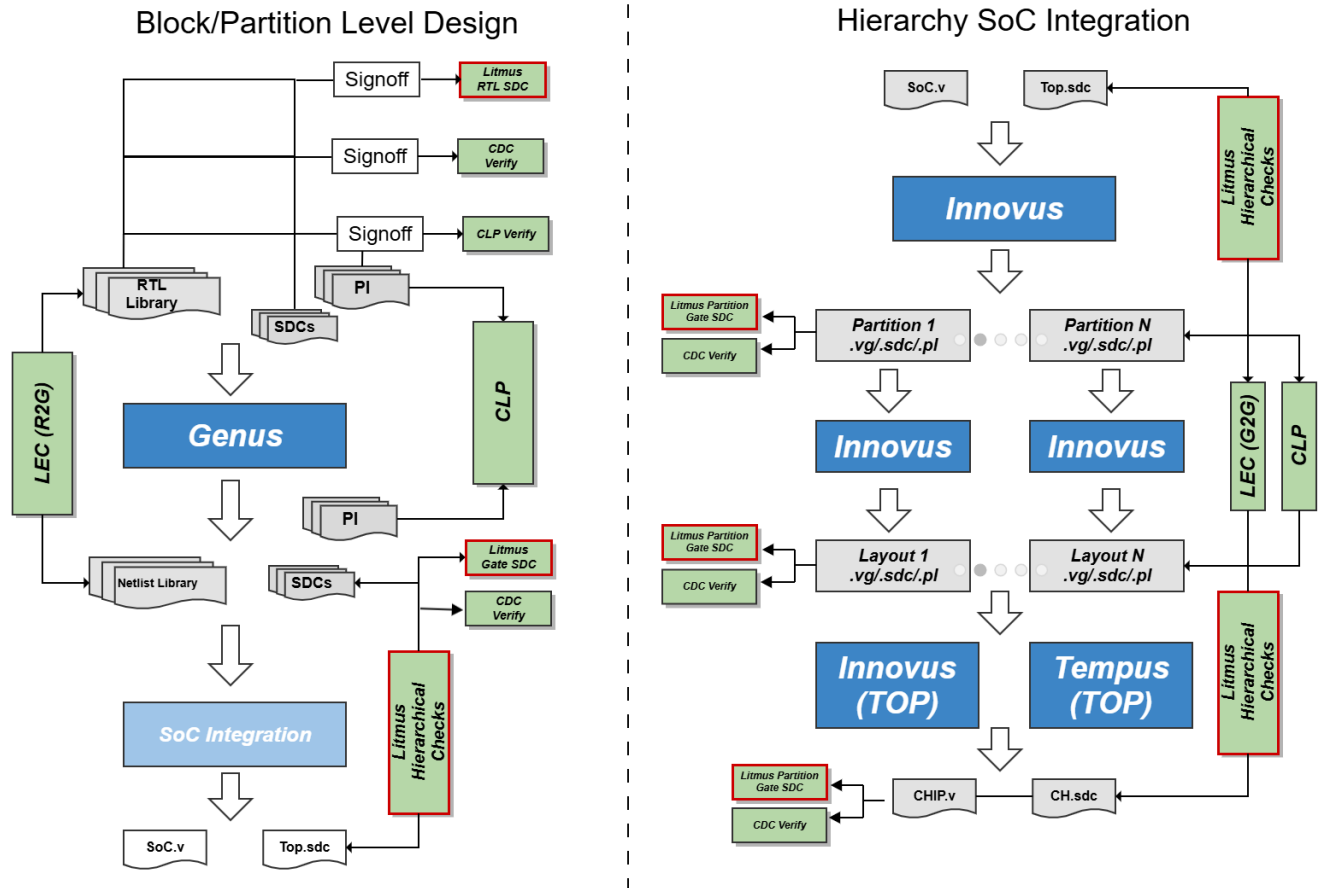
This is important for constraints in the following ways:

- Common SDC parser.
- Read the SDC the same way as a static timer (Tempus).
- Stylus CUI enabled - shared setup with Genus/Innovus/Tempus.

Litmus With Other Cadence Tools

Litmus is extensively used throughout the design and implementation flow at the RTL and gate-level. Thus, it supports both block-level and hierarchical or top-level designs.

The following figure shows the multiple full flow use cases Litmus supports.



Using Litmus

Use the `litmus` command, to start a Litmus session. The command has the following options.

Parameter	Description
Controls	
<code>-abort_on_error</code>	Exit if an error is detected at any point.
<code>-batch</code>	No interactive prompt occurs. Exit after <code>-execute</code> and <code>-files</code> options are processed.
<code>-cds_lib_file</code> <code>in_file</code>	Loads <code>in_file</code> rather than the default <code>cds.lib</code> file in the current directory to obtain the names and locations of OpenAccess libraries.
<code>-cpus integer</code>	Specify the number of CPUs to use for multi-threading on current host.
<code>-disable_user_startup</code>	Reads only the init file in the installation hierarchy. Does not read init files present in the home or current directory.
<code>-execute tcl_command</code>	Specify the TCL commands to be executed as a quoted string at startup.

<code>-files file_list</code>	Executes the script file at startup. One or more TCL files to source at startup, after <code>-execute</code> . Set the environment variable <code>CDS_STYLUS_SOURCE_VERBOSE</code> to 0 if you don't want the source command output to default to verbose in Stylus.
<code>-log file</code>	Setup the log file to save the transcript.
<code>-no_logv file</code>	Specify to avoid creating verbose logv file.
<code>-no_cmd</code>	Specify to avoid creating verbose command file.
<code>-no_gui</code>	Disable GUI and Tk support. GUI will not be available later in the session.
<code>-overwrite</code>	Specify to overwrite existing log files
<code>-version</code>	Return program version information.

Use the `exit` command to exit a session.

Inside the Tool

Litmus supports command line tab completion. It also supports the following commands.

Commands	Description
<code>man <command></code> <code>man <rule></code>	Shows the details of a command or a rule.
<code>history</code>	Shows the list of previous commands. Alternatively, you can use the up or down arrow keys as well.
<code>set_message -id <rule> -severity <warn error info reset></code> <code>set_message -id <rule> -suppress -unsuppress</code>	Changes rule severity level or suppress/unsuppress a rule.

<code>report_rule_check -severity <severity_name> -category <category_name> -status <name></code>	Reports the rule occurrences encountered when running the specified rule instances
<code>set_rule_handling</code>	Sets the severity level of rule instances or limits the number of occurrences that each rule instance can have.

Installation and Startup

For product, release, and installation information, see the README file at any of the following locations:

- [Cadence Downloads Site](#)
- In the software installation, where is also available when you are using or running the software.

For information about the Litmus licenses, see [Product and Licensing Information](#).

Related Topics

- [Setting up the Litmus Runtime Environment](#)
- [Launching the Litmus Console](#)
- [Starting the Litmus Software](#)
- [Using Litmus GUI](#)
- [Accessing Litmus Documentation and Help](#)

Setting up the Litmus Runtime Environment

If `install_dir` is the location of Litmus installation, then you should setup the runtime environment as follows:

- Add the `install_dir/bin` directory to the path. The `bin` directory has links to all the public executable files in the install hierarchy.
- If you want the Litmus man pages to be available with the Unix `man` command, then you can add the `install_dir/share/litmus/man` to the `MANPATH` envvar.
- If you want the Tcl man pages to be available with the Unix `man` command, then you can add the `install_dir/share/tcltools/man` to the `MANPATH` envvar.

For example, you can add the following to the startup shell script:

```
set install_dir = /tools/litmus24.1/lnx86
set path = ($install_dir/bin $path)
setenv MANPATH $install_dir/share/litmus/man:$install_dir/share/tcltools/man:$MANPATH
```

 When Litmus launches, it automatically adds man pages (if the `-stylus` parameter is specified), and the Tcl man pages at the beginning of the current `MANPATH` inside Litmus. Therefore, from within Litmus the `man` command will show both the sets of man pages before any other man pages.

Supported and Compatible Platforms

The README file lists the supported and compatible platforms for this release.

Temporary File Locations

Each Litmus session creates its own temporary directory to store temporary files at the beginning of the run.

By default, the `tmp_dir` is created in the `/tmp` directory. If the Unix envvar `TMPDIR` is set, then the `tmp_dir` is created inside `$TMPDIR`.

The name of the `tmp_dir` will appear as `ssv_tmpdir_[pid]_xxxxxx`

Where `_xxxxxxx` is a string that is added to make the directory unique. For example:

`ssv_tmpdir_10233_nfp9ez`

The temporary directory is automatically removed on exit or if the run is terminated with a catchable signal (for example, `SIGSEGV`).

Launching the Litmus Console

When you start Litmus, the UNIX window (shell tool, xterm, and so on) where you start the Litmus session is called the Litmus console. This is where you enter all Litmus text commands and where the software displays messages. When a session is active, this console displays the following prompt:

```
litmus>
```

If you use this console for other actions, for example, to use the vi editor, the session suspends until you finish the action.

If you suspend the session by typing Control-z, the `litmus>` prompt is no longer displayed. To return to the Litmus session, type `fg`, which brings the session to the foreground.

Related Topics

- [Completing Command Names](#)
- [Command-Line Editing](#)

Completing Command Names

Use the Tab key within the software console to complete text command names.

After you type a partial text command name and press the Tab key, the software displays the exact command name that completes or matches the text you typed (if the string is unique to one text command) or all the commands that match the text you typed.

For example, if you type the text `report_ti` and press the Tab key, the software displays the following command: `report_timing`

If you type the text `report_t` and press the Tab key, the software displays the following commands:

```
report_timing  report_timing_derate  report_timing_lib  report_timing_summary
```

Command-Line Editing

The Litmus software provides a GNU Emacs-like editing interface. You can edit a line before it is sent to the calling program by typing control characters. A control character, shown below as a caret followed by a letter, is typed by holding down the Control key when typing the character.

Most editing commands can be given a repeat count, *n*, where *n* is a number. To enter a repeat

count, press the *Esc* key, the number, and then the command to execute. For example, *Esc 4 ^f* moves forward four characters. If a command can be given a repeat count, the text *[n]* is shown at the end of its description.

You can type an editing command anywhere on the line, not just at the beginning. You can press Return anywhere on the line, not just at the end.

 Editing commands are case sensitive: *Esc F* is not the same as *Esc f*.

Control (^) Characters

^A	Move to the beginning of the line
^B	Move left (backwards) [<i>n</i>]
^C	Exits from editing mode, returning the console to normal Tempus System mode
^D	Delete character [<i>n</i>]
^E	Move to end of line
^F	Move right (forwards) [<i>n</i>]
^G	Ring the bell
^H	Delete character before cursor (backspace key) [<i>n</i>]
^I	Complete filename (Tab key); see below
^J	Done with line (Return key)
^K	Kill to end of line (or column [<i>n</i>])
^L	Redisplay line
^M	Done with line (alternate Return key)
^N	Get next line from history [<i>n</i>]
^P	Get previous line from history [<i>n</i>]
^R	Search backward (forward if [<i>n</i>]) <i>through history for text; must start line if text begins with an up arrow</i>

<code>^T</code>	Transpose characters
<code>^V</code>	Insert next character, even if it is an edit command
<code>^W</code>	Wipe to the mark
<code>^X^X</code>	Exchange current location and mark
<code>^Y</code>	Yank back last killed text
<code>^[</code>	Start an escape sequence (Esc key)
<code>^]c</code>	Move forward to next character <i>c</i>
<code>^?</code>	Delete character before cursor (Delete key) [<i>n</i>]

Starting the Litmus Software

To start a Litmus session, type the following command with the appropriate parameters on the UNIX/Linux command line.

```
litmus
```

For information on using this command, see `litmus` in the *Litmus Text Command Reference*.

Using Litmus GUI

The Litmus Graphical User Interface (GUI) provides an interactive environment to inspect and analyze rule violations generated during Litmus runs. The GUI supports advanced debug workflows by enabling you to review violations, cross probe to design representations, access SDC constraints, and manage large result sets efficiently.

The following are the key capabilities of the Litmus GUI:

- **Violation visualization and analysis:** Review rules across categories, their status, and details of the violated rules interactively in the GUI.
- **Violation-based schematic visualization:** Navigate from a violation to the schematic view and inspect the highlighted objects with standard zoom controls.
- **Advanced debug via cross-probing:** Navigate from violations to **HDL/RTL** using context-menu actions.

- **Direct interface to access SDC files:** Navigate from violations to **SDC** representations. This opens the SDC viewer for the occurrence that contains the SDC file/line reference and jumps to the exact line.
- **Filtering for large datasets:** Refine results using multiple filtering options, such as rule-level filtering (status/severity), name-based filtering, and occurrence-based filtering.
- **Scales for very large (1B+) designs:** Supports interactive review for high-capacity designs and large violation sets.

For detailed information about GUI components and workflows, refer to [Working with the GUI](#).

Launching the Litmus GUI

The Litmus GUI is launched from the Litmus command environment after the design has been initialized and analysis has been performed.

- Before invoking the GUI, ensure that:
 - The design is initialized using the Litmus setup command: `init_design`
 - Rule checks are executed using the `verify_timing_intent` or the `check_target_constraints` command.
- To invoke the GUI, type `gui_show` on the '`litmus>`' command prompt.

This displays the **Rules Summary** workspace:

The screenshot shows the Litmus Signoff Solution(TM) 26.1 GUI. On the left, a 'Rules Summary' pane lists various constraints with their severity, count, and status. The main pane displays a detailed view of the 'CONST_CLK_DEF1' constraint, showing a table of occurrences with columns for occurrence_id, scope, witness, clocks, count, clock_pin_witnesses, clock_pin_witnesses_filters, type, shared_driver, and blocks.

occurrence_id	scope	witness	clocks	count	clock_pin_witnesses	clock_pin_witnesses_filters	type	shared_driver	blocks
1	fifo	pin.fifo/mem2rw/mem_reg(14)(4)(clk)		1	{pin.fifo/mem2rw/mem_reg(14)(4)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
10	fifo	pin.fifo/mem2rw/mem_reg(3)(4)(clk)		1	{pin.fifo/mem2rw/mem_reg(3)(4)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
100	fifo	pin.fifo/mem2rw/mem_reg(9)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(9)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
101	fifo	pin.fifo/clkst/clk_d4_reg/clk		1	pin.fifo/clkst/clk_d4_reg/clk		MISSING_CLOCK	pin.fifo/clkst/clk_d2_reg/q	
102	fifo	pin.fifo/mem2rw/mem_reg(7)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(7)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
103	fifo	pin.fifo/wctrl/wctrl_reg(2)(clk)		1	{pin.fifo/wctrl/wctrl_reg(2)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
104	fifo	pin.fifo/mem2rw/mem_reg(15)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(15)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
105	fifo	pin.fifo/mem2rw/mem_reg(11)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(11)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
106	fifo	pin.fifo/mem2rw/mem_reg(3)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(3)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
107	fifo	pin.fifo/mem2rw/mem_reg(2)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(2)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
108	fifo	pin.fifo/mem2rw/mem_reg(10)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(10)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
109	fifo	pin.fifo/mem2rw/mem_reg(6)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(6)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
11	fifo	pin.fifo/mem2rw/mem_reg(11)(4)(clk)		1	{pin.fifo/mem2rw/mem_reg(11)(4)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
110	fifo	pin.fifo/mem2rw/mem_reg(14)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(14)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
111	fifo	pin.fifo/wctrl/wfull_reg/clk		1	pin.fifo/wctrl/wfull_reg/clk		MISSING_CLOCK_NO_SHARED_DRIVER		
112	fifo	pin.fifo/mem2rw/mem_reg(12)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(12)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
113	fifo	pin.fifo/mem2rw/mem_reg(4)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(4)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
114	fifo	pin.fifo/mem2rw/mem_reg(0)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(0)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		
115	fifo	pin.fifo/mem2rw/mem_reg(8)(5)(clk)		1	{pin.fifo/mem2rw/mem_reg(8)(5)(clk)}		MISSING_CLOCK_NO_SHARED_DRIVER		

Below the table, a 'Description' pane shows the text: 'Clock is missing'.

In typical workflows, the categories and violations shown depend on the analyses executed before launching the GUI. For example, violations may appear under categories such as *constraint signoff* and *constraint lint* after timing intent verification.

Exiting or Hiding the GUI

You can either **exit** the GUI (terminate the GUI application) or **hide** the GUI (remove it from view while keeping the Litmus tool in command mode available).

Exiting the GUI

You can exit the GUI by using any of the following methods:

- From the Litmus command prompt, type `quit` or `exit`.
- From the Menu Bar, select *File* → *Exit*, and click *OK* in the confirmation dialog to close the GUI.
- Use the *Close* button at the top-right corner of the main GUI window.

Hiding the GUI

Hiding the GUI removes it from view but allows the Litmus session to continue running. The GUI process runs in the background.

- From the Litmus command prompt, type `gui_hide`.
- From the Menu Bar, select *File* → *Hide GUI* to hide the GUI.
- Use the *Hide GUI* option (while using *File* → *Exit* or the *Close* button) to hide the GUI.
- Use `gui_show` again to re-display the GUI window.

Accessing Litmus Documentation and Help

You can access the Litmus documentation and help system by using the following methods:

- Launching Doc Assistant
- Accessing Documentation and Help from Litmus GUI
- Using the Litmus man and help Commands on the Command-Line
- Other Sources of Cadence Product Information

Launching Doc Assistant

To launch the Doc Assistant, you can use one of the following ways:

- Linux systems: Navigate to the `<install_directory>/lnx86/bin` directory, and run the `cda` command.
- Within the Litmus shell run `cda` command.
- Tool's *Help* menu.

For more information, see the Doc Assistant User Guide.

Accessing Documentation and Help from Litmus GUI

From the Litmus GUI Menu Bar, you can access product documentation and help resources using the *Help* menu, which provides entry points to the Cadence Doc Assistant (CDA) and related reference material. In addition to menu-based access, the Litmus GUI supports context-sensitive documentation access: when a rule is selected in the Rules Summary, pressing **F1** opens CDA directly at the corresponding Rule Reference page for the selected rule. This allows you to quickly review detailed rule descriptions and reference information without leaving the GUI.

Related Topics

- [Using the Litmus man and help Commands on the Command-Line](#)
- [Other Sources of Cadence Product Information](#)

Using the Litmus man and help Commands on the Command-Line

The man and help commands in Litmus provides information about all the applicable commands, attributes, and messages. All the available scenarios are explained below.

Using the help Command to View the Command Syntax

- To see syntax information for a command, type the following command in the software console:

```
help command_name
```

For example, to see syntax information for the `read_libs` command, type the following command:

```
help read_libs
```

The software displays the following text:

```
Command: read_libs:
```

```
Usage: read_libs [-help] [<in_file <file1[<file2> ...>>] [-aocv_libs <string>] [-le
lef_files <string>] [-library_side_file <string>] [-max_aocv <string>]
                [-max_libs <string>] [-max_noise_libs <string>] [-min_aocv
<string>] [-min_libs <string>] [-min_noise_libs <string>] [-noise_libs <string>]
                [-oa_ref_libs <string>] [-power_ground_files <string>] [-socv_libs
<string>]
```

```
-help                # Prints out the command usage
<in_file <file1[<file2> ...>>] # read the library files  (string, optional)
-aocv_libs <string>    # read the default AOCV libraries  (string,
optional)
...
-power_ground_files <string> # read the power grid library files  (string,
optional)
-socv_libs <string>      # read the default SOCV libraries  (string,
optional)
```

- To see the entire list of Litmus commands and their syntax, type the following command in the software console:

```
help
```

Using the man Command to View the Command Description

To see the complete set of information for a Litmus command, type the following command in the software console:

```
man command_name
```

For example, to see the complete set of information for the `read_lib` command, type the following command:

```
man read_libs
```

Using the help Command to View Message Summary

To see the message summary of a particular message ID, type the following command in the software console:

```
help msg_id
```

For example, to see the message summary for the TECHLIB-1002 message ID, type the following command:

```
help TECHLIB-1002
```

Using the man Command to View Message Detail

To see the message detail of a particular message ID, type the following command in the software console:

```
man msg_id
```

For example, to see the message summary for the SI-2253 message ID, type the following command:

```
man SI-2253
```

The detailed description is not available for all active message IDs.

Other Sources of Cadence Product Information

You can also get help on Cadence products by selecting Customer Support on the *Help menu*. The Customer Support sub menu provides access to the following Cadence resources:

- Cadence Online Support Opens Cadence Online Support in your browser.
- Web Collaboration Opens your default browser and explains the current methodology followed by Cadence to support web-based collaboration.

Constraints

Constraints provide timing intent specifications that a design must meet through optimization. Synopsys Design Constraints (SDC) format is the standard for writing constraints in the industry, which conforms to Tcl syntax.

The following table lists the constraints categories, and the common commands based on them.

Categories	Commands
Clock constraints	<code>create_clock</code> <code>create_generated_clock</code> <code>set_clock_uncertainty</code> <code>set_clock_latency</code> <code>set_clock_transition</code> <code>set_propagated_clock</code> <code>set_sense</code> / <code>set_clock_sense</code> <code>set_clock_groups</code> <code>set_clock_gating_check</code>
IO constraints	<code>set_input_delay</code> <code>set_output_delay</code> <code>set_drive</code> <code>set_driving_cell</code> <code>set_input_transition</code> <code>set_load</code>
Exception	<code>set_false_path</code> <code>set_multicycle_path</code> <code>set_max_delay</code> / <code>set_min_delay</code>

Case analysis constraints	set_case_analysis
Environmental constraints	set_wire_load_mode set_wire_load_model
Design rule constraints	set_max_fanout set_max_transition set_max_capacitance
Others	set_disable_timing set_data_check

The recommended order for reviewing constraints based on their categories is:

1. Case analysis
2. Clock
3. IO
4. Exception
5. Others (including Environmental and Design rule)

Related Topics

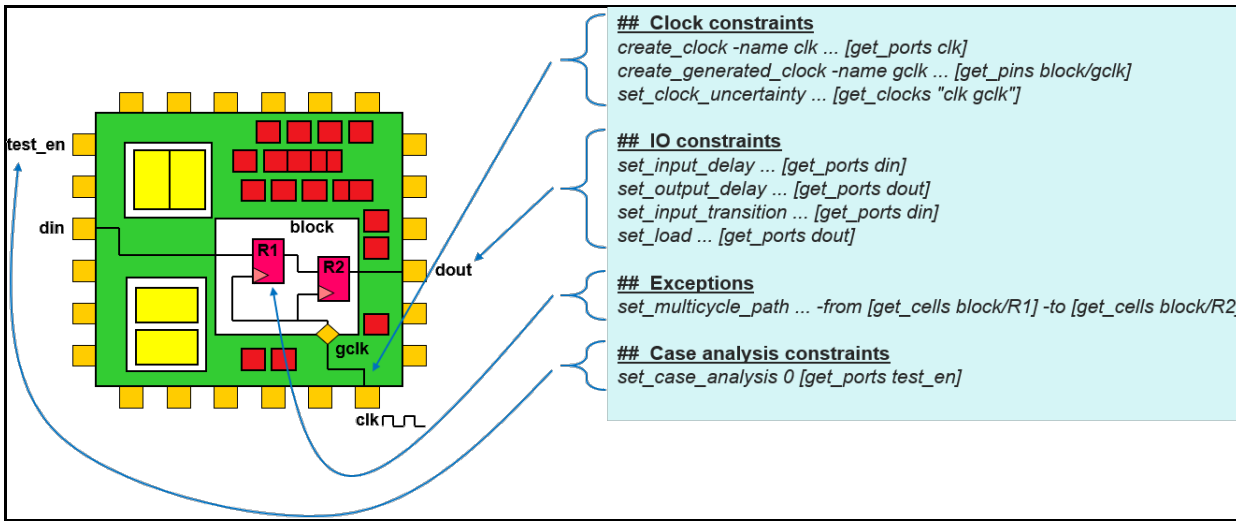
- [Setting Constraints](#)
- [Common Object Access Commands](#)
- [Constraints Signoff](#)
- [Constraint Signoff Requirements](#)
- [Constraint Signoff Benefits](#)
- [Constraints Signoff Solution](#)

Setting Constraints

Setting constraints is to apply constraints to design objects. Ensure that the paths are constrained for timing checks to meet timing intent specifications.

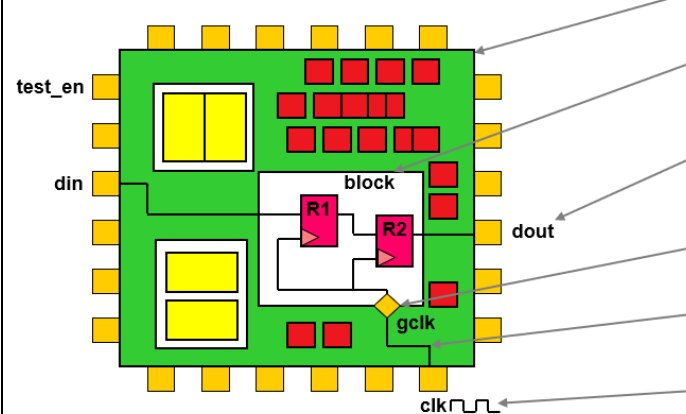
The following figure shows a sample for setting constraints for the basic constraint categories and

The following figure shows a sample for setting constraints for the basic constraint categories and ensuring that the paths are constrained for timing checks:



Common Object Access Commands

The following are the common object access commands based on the design object types:

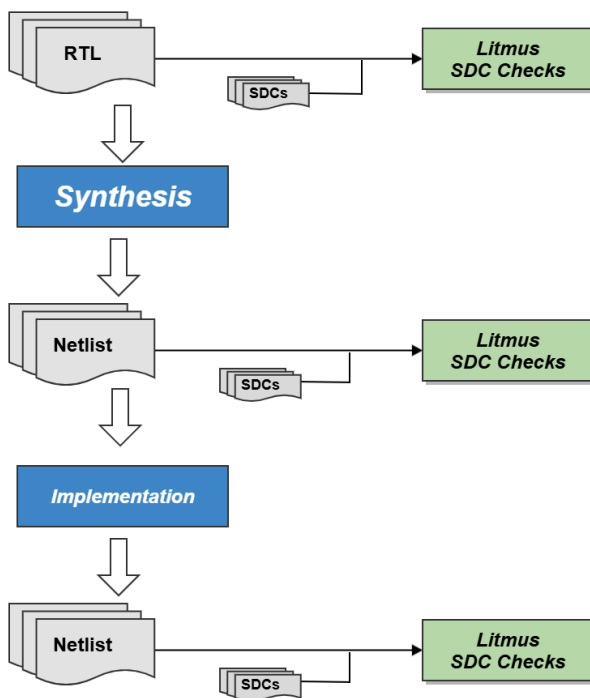
		Object	Command	Description
		Design	current_design	Design is a container for cells or is the entire circuit.
		Cell	get_cells	Cell is an instance of a design or is a library component.
		Port	get_ports all_inputs all_outputs	A port is a signal entry point or exit point to a design.
		Pin	get_pins	A pin is a hierarchical port of a design, port of an instance, or a port of a library cell.
		Net	get_nets	A net is an interconnect between cell pins and design ports.
		Clock	get_clocks all_clocks	A clock is a port or a pin that drives sequential cells.

Constraints Signoff

Litmus uses rules to ensure that the constraints are written correctly with the RTL and applied correctly throughout the design, implementation, and signoff stages. The constraints signoff verifies the correctness and completeness of constraints at the top and/or block-level. It also identifies the missing constraints, exception overlaps, and case analysis conflicts. It runs the following checks:

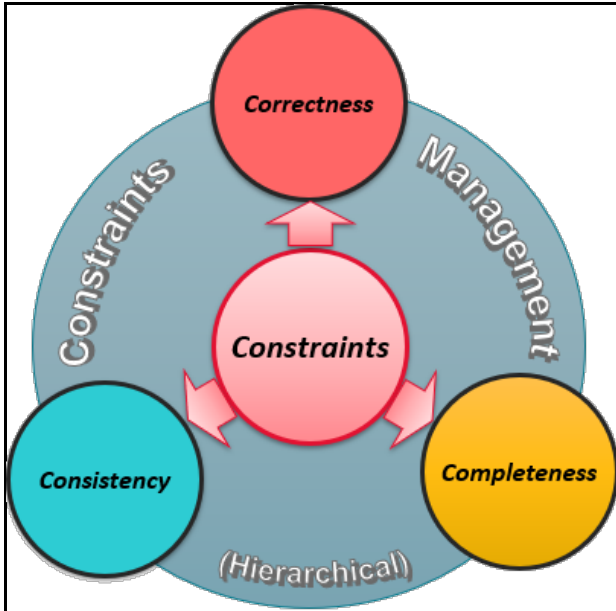
- Clock checks
- IO checks
- Exception checks
- Case Analysis checks

The following figure shows the Litmus flow for proven and comprehensive Constraints Signoff checking.



Constraint Signoff Requirements

The following figure shows the requirements for complete coverage of constraints failure modes:



Correctness

You require language level syntax; semantics checks and structural checks to ensure correctness.

The following table lists all the lint or semantic checks:

Type	Rule Checks
Lint	CTE-* / IMPTCM-* / TA-* / TCLCMD-* / TCLNL-* CONST_LINT_CMD* / CONST_LINT_REF*
Clock constraints	CONST_LINT_CLK* / CONST_LINT_UNC* / CONST_LINT_LAT* / CONST_LINT_CTR* / CONST_LINT_GRP* / CONST_LINT_CG*
IO constraints	CONST_LINT_IDL* / CONST_LINT_ODI* / CONST_LINT_ITR* / CONST_LINT_OLD*
Exceptions	CONST_LINT_EXC* / CONST_LINT_FLP* / CONST_LINT_MCP*
Case analysis constraints	CONST_LINT_SCA*

Others	CONST_LINT_DAT* / CONST_LINT_HFN*
--------	-----------------------------------

Completeness

Completeness covers missing, redundant, and inefficient constraints.

The following table lists all the signoff rules for completeness checks:

Type	Rule Checks
Clock constraints	CONST_CLK_DEF* / CONST_CLK_UNC* / CONST_CLK_LAT* / CONST_CLK_CTR* / CONST_CLK_GRP* / CONST_CG_CHK*
IO constraints	CONST_IO_IDL* / CONST_IO_ODL* / CONST_IO_ITR* / CONST_IO_OLD*
Exceptions	CONST_EXC_FLP* / CONST_EXC_MCP* / CONST_EXC_SMD* / CONST_EXC_OLP*
Case analysis constraints	CONST_SCA_CHK*
Design rule constraints	CONST_IO_DRC*
Others	CONST_DGN_CMB* / CONST_DGN_TIM* / CONST_DAT_CHK* / CONST_HFN_CHK*

Consistency

Consistency covers conflicting constraints, inconsistent clock schemes, and inconsistent constraints.

The following table lists all the signoff rules for consistency checks:

Type	Rule Checks
Clock constraints	CONST_CLK_*
IO constraints	CONST_IO_*
Exceptions	CONST_EXC_OLP*
Case analysis constraints	CONST_SCA_*
Others	CONST_DGN_*

Constraint Signoff Benefits

Constraints Signoff supports the following four key areas:

- Duplicate Constraints
 - Same commands, path specification, and values.
 - Same exceptions and delays, such as `set_drive`, `set_load`, etc.
- Overwritten Constraints
 - Same commands and path specification, but different values.
 - Same exceptions and delays, such as `set_drive`, `set_load`, etc.
- Overlapping Exceptions
 - Two exceptions share one or more matching paths.
- Overridden Exceptions
 - An exception on a path is overridden by exceptions of higher priority applied to the same path.

Constraints Signoff Solution

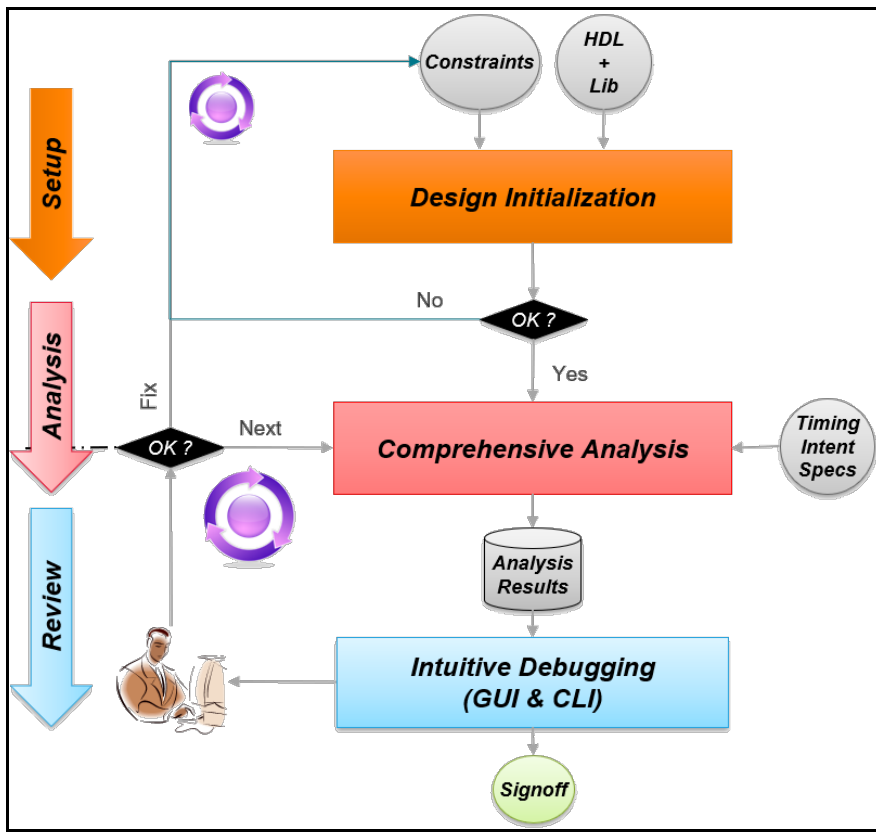
- [Constraints Signoff Use Model](#)
- [Constraints - Design Initialization](#)
 - [MMMC Setup](#)
 - [Example of MMC Setup](#)
 - [Constraint Command Summary](#)
- [Constraints - Analysis](#)
 - [Define Timing Intent](#)
 - [Perform Constraints Analysis](#)
 - [Constraints Checks](#)
 - [Constraints Analysis Examples](#)
- [Constraints - Review Methodology](#)
- [Constraint Signoff - Sample Run Script](#)

Constraints Signoff Use Model

Verification is an iterative process that can be minimized by streamlined methodology. Constraints Signoff flow consists of the following:

- **Setup: Design Initialization**
 - Defines checkpoints in the flow.
 - Specifies constraints that provide full coverage for the design.
 - Validates correctness of the design and constraints setup or coverage.
 - Provides standard CUI setup support, leveraging common design across Cadence implementation/signoff tools.
- **Analysis**
 - Captures timing intent with constraints (hierarchical) signoff checks.
 - Provides additional timing intent specifications that configure rules to meet design methodology.
- **Review: Reporting and Diagnosis**
 - Provides concise and less noisy reports for your review.
 - Annotates failure modes.
 - Provides access to analyze information via CUI DB access commands.

The following figure shows the Constraints Signoff Flow:



Constraints - Design Initialization

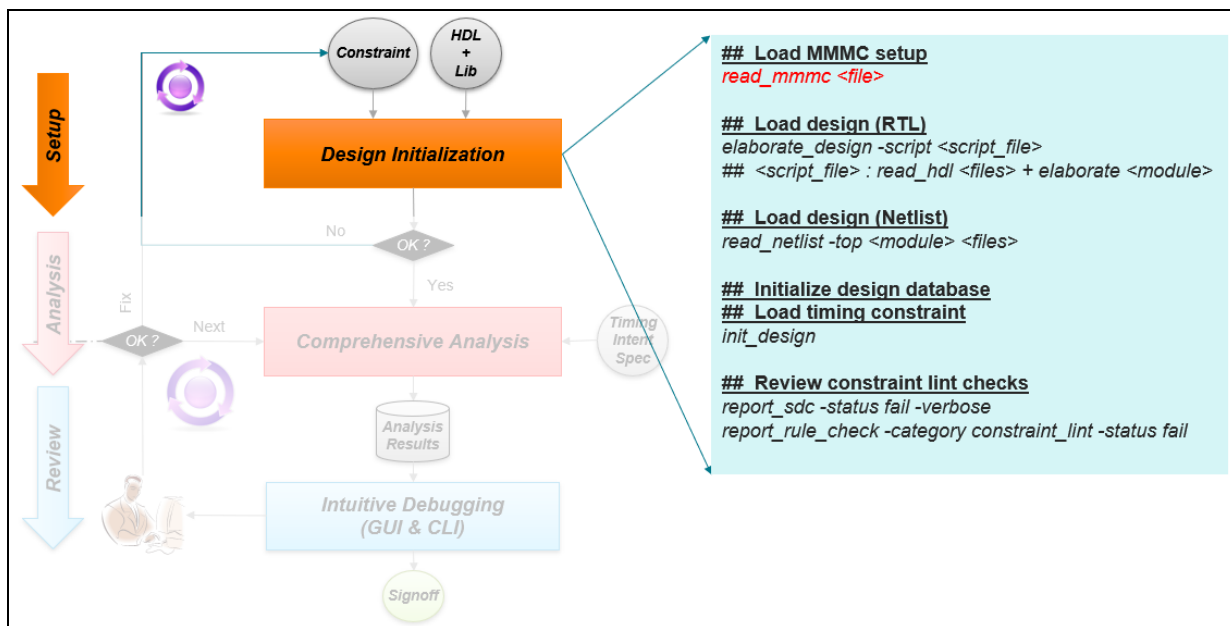
This is supported for Multi-Mode Multi-Corner and uses this method of importing design data. Litmus supports only one active view for analysis in MMMC flow.

Design Setup with MMMC Flow

The following are the commands for design setup with MMMC initialization flow:

- Use the `read_mmmc` command to load timing libraries and setup timing constraints.
- Use the `init_design` command to load timing constraints.

The following figure shows the design setup with MMMC flow:

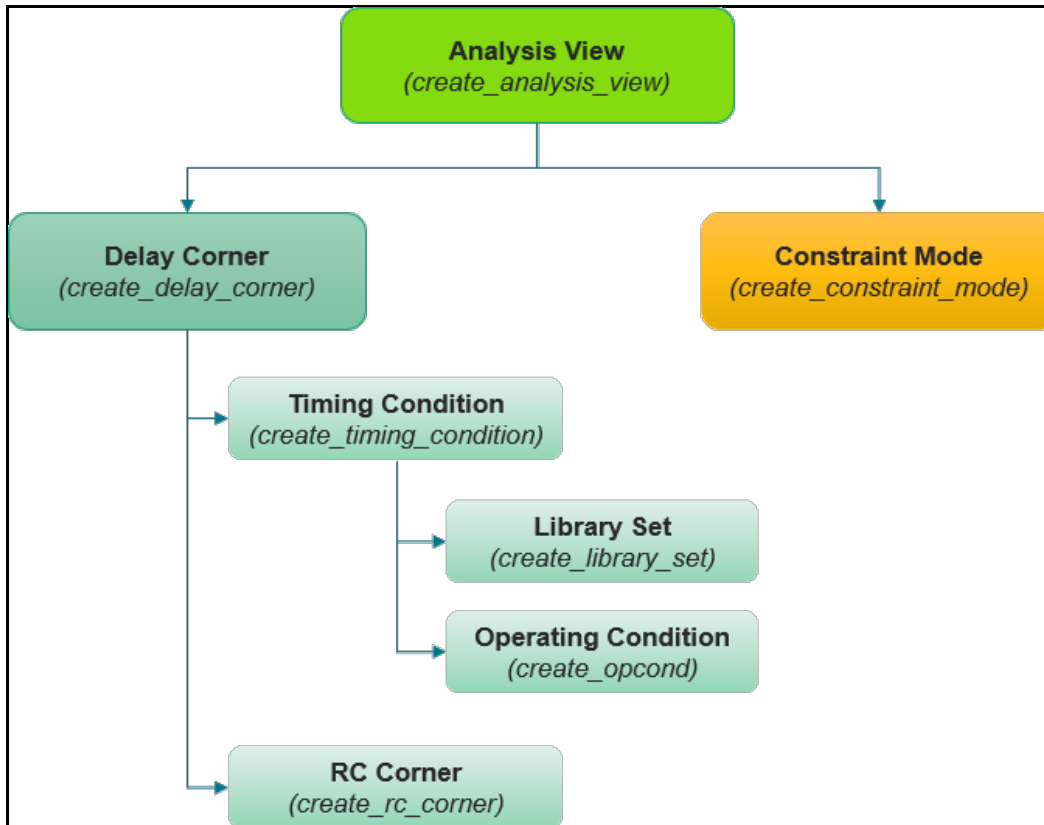


MMMC Setup

MMMC setup file defines analysis views with delay and constraint information.

- Delay information
 - Timing library PVT.
 - RC extractions.
- Constraint information
 - Timing constraints.

The following figure explains the Analysis View:



Example of MMMC Setup

Delay Information

Library sets

```
create_library_set -name default_emulate_libset_max -timing slow.lib
```

```
create_library_set -name default_emulate_libset_min -timing fast.lib
```

Timing conditions

```
create_timing_condition -name default_emulate_tc_max -library_sets  
default_emulate_libset_max
```

```
create_timing_condition -name default_emulate_tc_min -library_sets  
default_emulate_libset_min
```

RC corners

```
create_rc_corner -name default_emulate_rc_corner
```

Delay corners

```
create_delay_corner -name default_emulate_delay_corner -late_timing_condition  
default_emulate_tc_max -early_timing_condition default_emulate_tc_min -rc_corner  
default_emulate_rc_corner
```

Constraint Information

Constraint modes

```
create_constraint_mode -name default_emulate_constraint_mode -sdc_files SoC.sdc
```

Analysis views

```
create_analysis_view -name default_emulate_view -delay_corner  
default_emulate_delay_corner -constraint_mode default_emulate_constraint_mode
```

Active analysis views

```
set_analysis_view -setup default_emulate_view -hold default_emulate_view
```

Constraint Command Summary

Review the failed commands with constraint lint check violations using the following command:

```
report_sdc -status fail -verbose
```

Constraints

Sample report for 'failed commands' generated by using 'report_sdc -status fail':

Constraint commands for fifo (View top_view_max)			
Command	Successful	Failed	Total
all_clocks	1	0	1
all_inputs	5	0	5
all_outputs	4	0	4
create_clock	6	0	6
create_generated_clock	2	0	2
current_design	3	0	3
get_clocks	77	0	77
get_pins	11	0	11
get_ports	31	0	31
remove_from_collection	3	0	3
set_case_analysis	3	0	3
set_clock_groups	3	0	3
set_clock_latency	32	0	32
set_clock_transition	10	0	10
set_clock_uncertainty	16	0	16
set_disable_timing	1	0	1
set_dont_touch_network	1	0	1
set_false_path	4	0	4
set_input_delay	6	0	6
set_input_transition	2	0	2
set_load	2	0	2
set_max_capacitance	3	0	3
set_max_fanout	2	0	2
set_max_transition	3	0	3
set_multicycle_path	8	0	8
set_output_delay	6	0	6
set_sense	1	0	1

Command details (View top_view_max)			
Status	Command	Line Number	File
Pass	create_clock	121	sdcd/fifo_demo.sdc.AFIFO
Pass	TCLCMD-1142	Virtual clock is being created with no source objects.	sdcd/fifo_demo.sdc.AFIFO
Pass	create_clock	139	sdcd/fifo_demo.sdc.AFIFO
Pass	TCLCMD-1142	Virtual clock is being created with no source objects.	sdcd/fifo_demo.sdc.AFIFO
Pass	create_clock	147	sdcd/fifo_demo.sdc.AFIFO
Pass	TCLCMD-1142	Virtual clock is being created with no source objects.	sdcd/fifo_demo.sdc.AFIFO
Pass	create_generated_clock	92	sdcd/fifo_demo.sdc.AFIFO
	TCLCMD-1531	has been applied on hierarchical pin	
	CONST_LINT_REF1	Constraints is applied on hierarchical pin	
	CONST_LINT_CLK7	Clock definition point is not an input port, an output pin of a sequential cell or an output pin of a blackbox	

Constraints Lint

Sample summary report for constraint lint checks. This shows how to get the details of rule check violations:

Any violation by report_rule_check	
CONST_LINT_CLK7: Clock definition point is not an input port, an output pin of a sequential cell or an output pin of a blackbox Severity: Warning Fail Occurrence: 2 CONST_LINT_FLP1: False path does not have both -from and -to Severity: Warning Fail Occurrence: 3 CONST_LINT_MCP1: Multi-cycle path does not have both -from and -to Severity: Warning Fail Occurrence: 6 CONST_LINT_OLD1: Load value outside characterization range Severity: Warning Fail Occurrence: 2 CONST_LINT_REF1: Constraints is applied on hierarchical pin Severity: Warning Fail Occurrence: 10 TCLCMD-1531: has been applied on hierarchical pin Severity: Warning Fail Occurrence: 9 TCLCMD-1005: Using master clock for generated clock in view Severity: Warning Fail Occurrence: 2 TCLCMD-1142: Virtual clock is being created with no source objects. Severity: Warning Fail Occurrence: 3 TA-968: Reading timing constraints file ... Severity: Warning Fail Occurrence: 1	
Details by -verbose	
CONST_LINT_CLK7: Clock definition point is not an input port, an output pin of a sequential cell or an output pin of a blackbox Severity: Warning Fail Occurrence: 2 1: Fail: In line 92, file: sdcd/fifo_demo.sdc.AFIFO (create_generated_clock): clock rclk: Source object: clkrst/mux2_rclk/D0[0]. 2: Fail: In line 105, file: sdcd/fifo_demo.sdc.AFIFO (create_generated_clock): clock wclk: Source object: clkrst/mux2_wclk/D0[0].	

You need to **fix** the violation that creates a failed command.

Constraints - Analysis

Constraints analysis consists of the following:

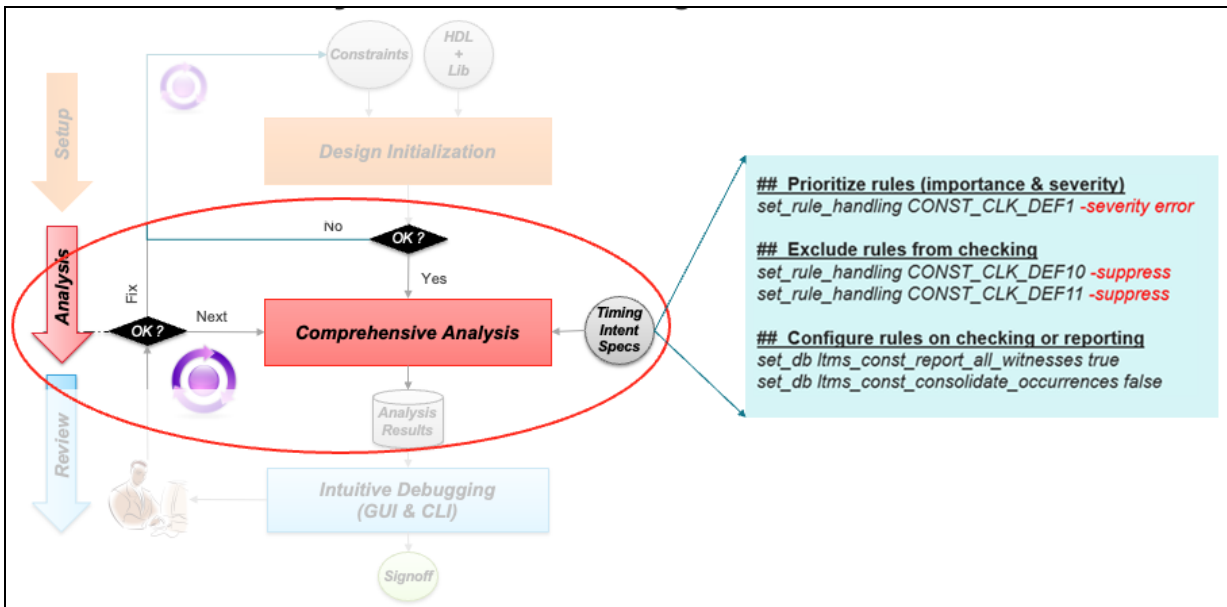
- Define Timing Intent
- Perform Constraints Analysis
- Constraints Checks
- Constraints Analysis Examples

Define Timing Intent

Timing intent is defined to meet design and verification methodology. To define timing intent:

1. Prioritize rules based on importance or severity.
2. Exclude rules from checking.
3. Configure rules on checking or reporting.

The following figure shows methodology to define timing intent:

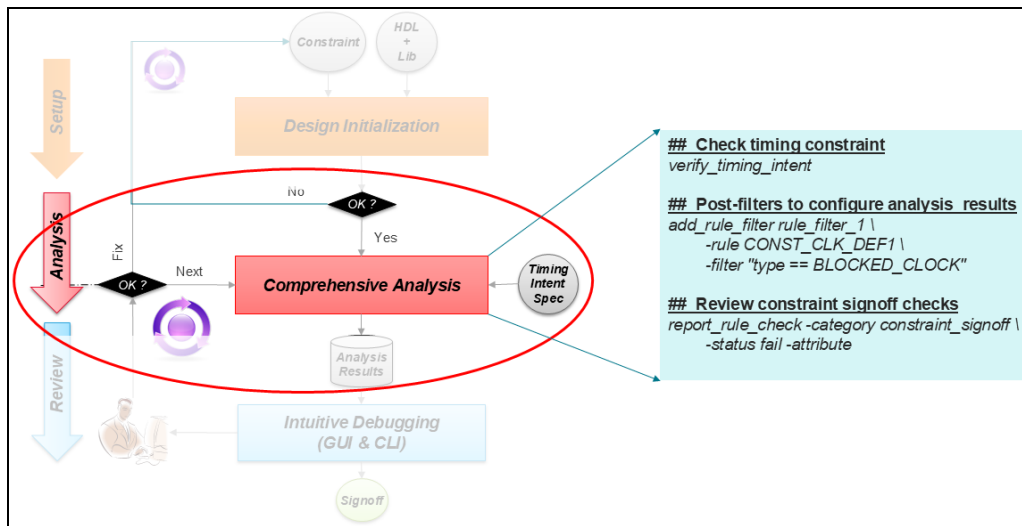


Perform Constraints Analysis

Performing constraints analysis followed by analysis results configuration and report generation. It includes the following:

1. Commands to verify timing intent.
2. Post-filters to configure analysis results.
3. Generate reports for review.
4. Constraints failure modes.

The following figure explains constraints analysis:



Constraints Checks

The following table shows constraints checks based on constraints categories:

Categories	Checks
------------	--------

Clock constraints	CONST_CLK_DEF*: Clock definition CONST_CLK_UNC*: Clock uncertainty CONST_CLK_LAT*: Clock latency CONST_CLK_CTR*: Clock transition CONST_CLK_GRP*: Clock group CONST_CG_CHK*: Clock gating check
IO constraints	CONST_IO_IDL*: Input delay CONST_IO_ODL*: Output delay CONST_IO_ITR*: Input transition CONST_IO_OLD*: Output load
Exceptions	CONST_EXC_FLP*: False path CONST_EXC_MCP*: Multicycle path CONST_EXC_SMD*: Delay path (max/min) CONST_EXC_OLP*: Exception overlapping
Case analysis constraints	CONST_SCA_CHK*: Case analysis check
Design rule constraints	CONST_IO_DRC*: Design rule check
Others	CONST_DGN_CMB*: Combinational path check CONST_DGN_TIM*: Timing path check CONST_DAT_CHK*: Data check CONST_HFN_CHK*: High fanout network check

Constraints Analysis Examples

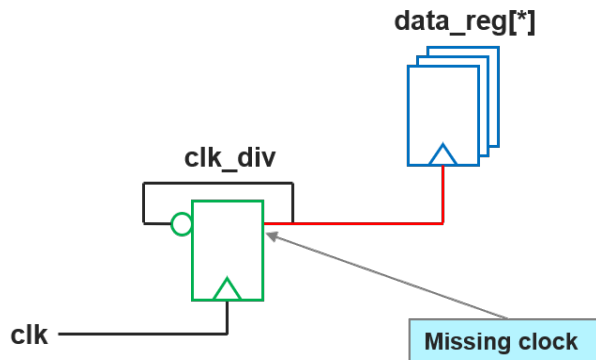
The following are the examples based on constraints categories:

- [Clock Constraints](#)
- [IO Constraints](#)

- [Exception Constraints](#)
- [Case Analysis Constraints](#)

Clock Constraints

The following figure shows an example of clock constraints:



Constraints

```
create_clock -name clk ... [get_ports clk]
```

The generated clock is missing for `data_reg[*]/CK`.

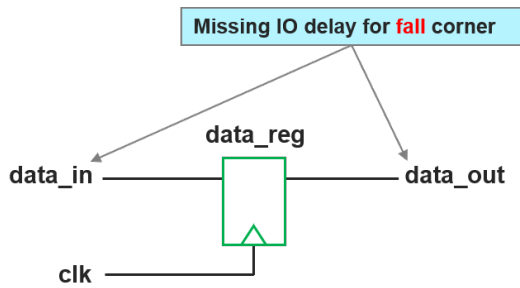
- Failure check: Completeness
 - Missing constraints
- CONST_CLK_DEF1 check flagged
 - The clock is missing at `clk_div/Q` that drives 3 clock pins
 - Add the constraint to fix the violation:

Constraints

```
create_generated_clock -name gclk...[get_pins clk_div/Q]
```

IO Constraints

The following figure shows an example of IO constraints:



```
## Constraints
create_clock -name vclk ...
set_input_delay -clock [get_clocks vclk] \
  -rise ... [get_ports data_in]
set_output_delay -clock [get_clocks vclk] \
  -rise ... [get_ports data_out]
```

IO delay for `-rise` is specified but lack of IO delay for `-fall`. `CONST_IO_IDL4/ODL4` is reported with one violation for the root cause at missing IO delay for `-fall`.

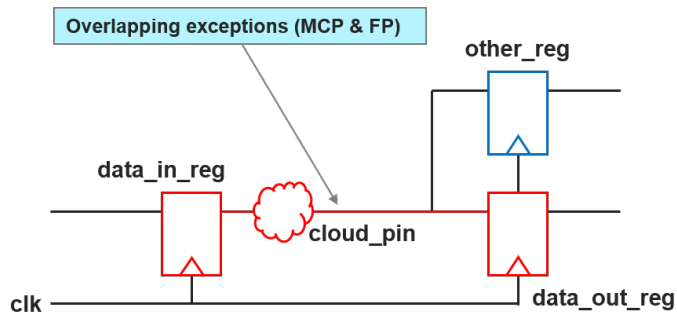
- Failure check: Completeness
 - Missing constraints for corners
- `CONST_IO_IDL4` and `CONST_IO_ODL4` checks flagged
 - IO delay for fall corner is missing for `data_in` and `data_out`
 - Add the constraints to fix the violations:

Constraints

```
set_input_delay -clock [get_clocks vclk] \
  -fall...[get_ports data_in] \
set_output_delay -clock [get_clocks vclk] \
  -fall...[get_ports data_out] \
```

Exception Constraints

The following figure shows an example of exceptions:



Constraints

```
set_multicycle_path -setup 4 \
    -from [get_cells data_in_reg] -to [get_cells data_out_reg]
set_false_path \
    -through [get_pins cloud_pin]
```

CONST_EXC_OLP1a is reported with one violation for the root cause at exception overlapping with the sample path.

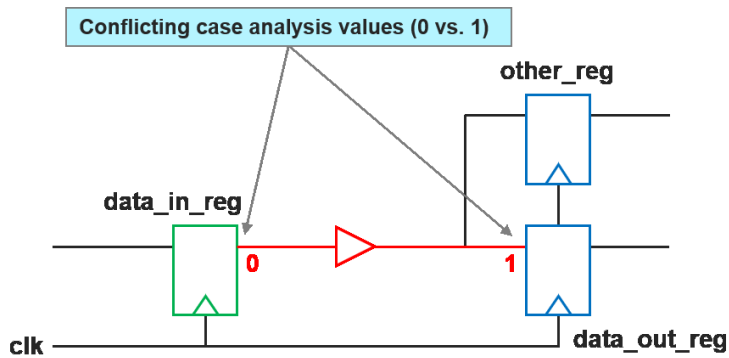
- Failure check: Completeness
 - Inefficient constraints
- CONST_EXC_OLP1a check flagged
 - Multicycle path and false path overlap on the timing path from `data_in_reg` to `data_out_reg`
 - Update the constraints to fix the violation:

Constraints

```
set_multicycle_path -setup 4 \
    -from [get_cells data_in_reg] -to [get_cells data_out_reg] \
set_false_path \
    -from [get_cells data_in_reg] -to [get_cells other_reg]
```

Case Analysis Constraints

The following figure shows an example of case analysis constraints:



Constraints

```
set_case_analysis 0 [get_pins data_in_reg/Q]  
set_case_analysis 1 [get_pins data_out_reg/D]
```

Two `set_case_analysis` values cannot be satisfied at the same time. `CONST_SCA_CHK1` is reported with one violation for the root cause at conflicting case analysis values.

- Failure check: Consistency
 - Conflicting constraints
- `CONST_SCA_CHK1` check flagged
 - Case analysis value 1 at `data_out_reg/D` conflicts with case analysis value 0 at `data_in_reg/Q`
 - Review the constraints to fix the violation:

Constraints #1

```
set_case_analysis 0 [get_pins data_in_reg/Q] \
```

Constraints #2

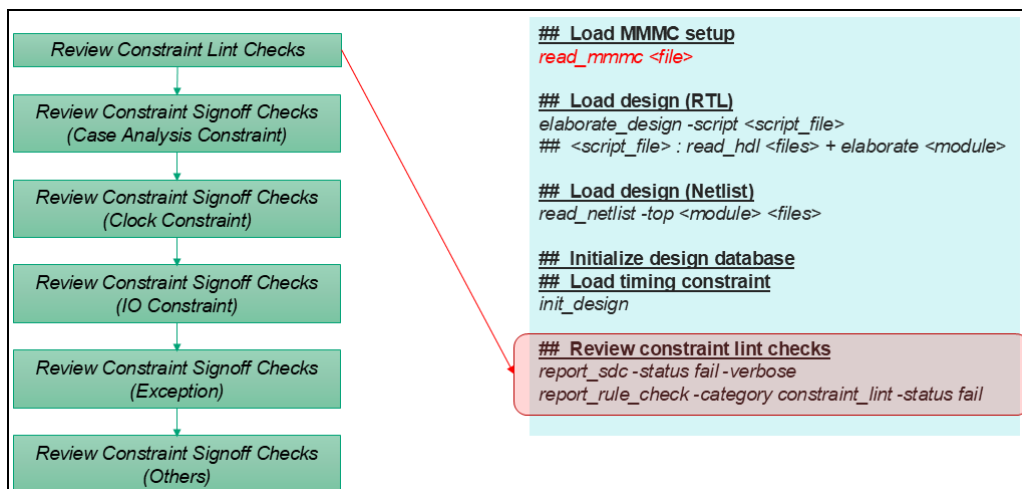
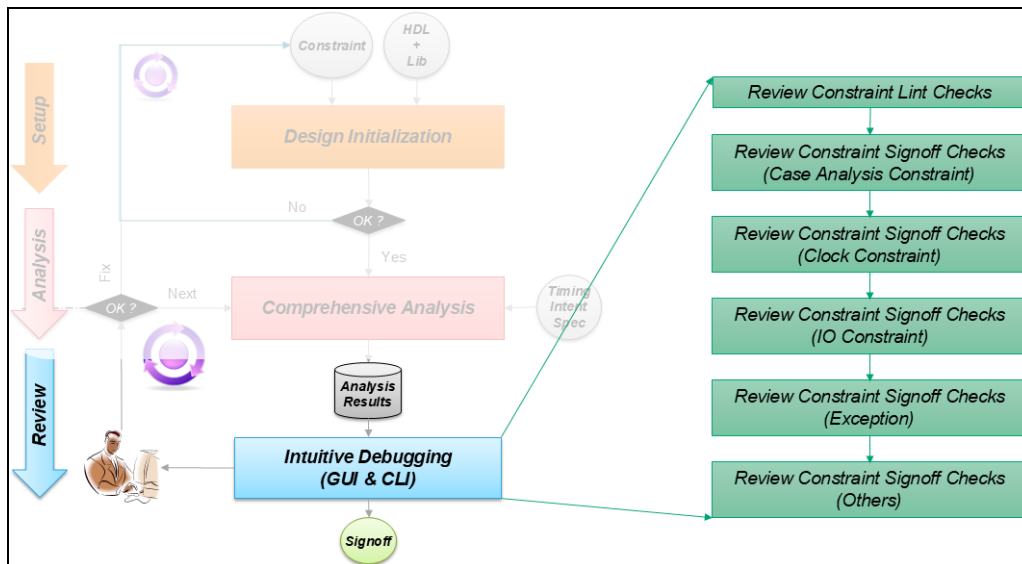
```
set_case_analysis 1 [get_pins data_in_reg/Q] \
```

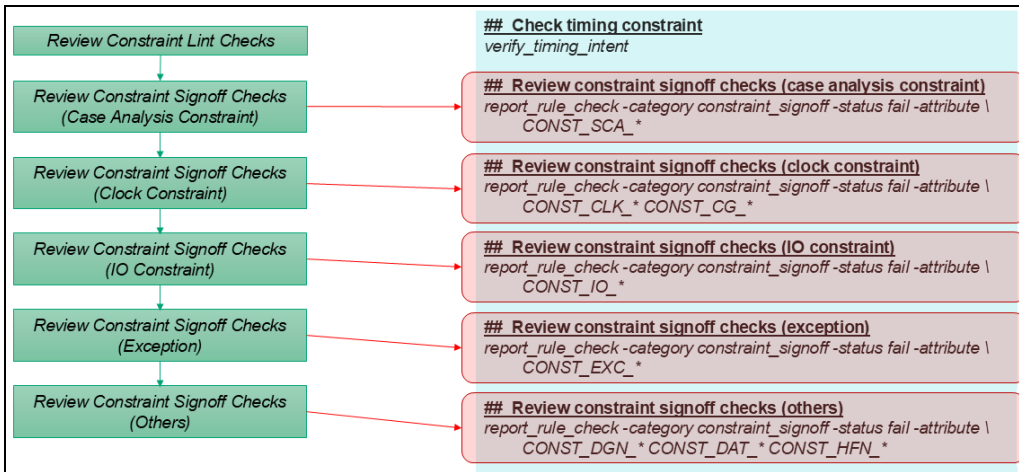
Constraints - Review Methodology

The recommended methodology for constraints review consists of the following:

- Design setup- review libraries and designs.
- Design Initialization- review libraries lint checks.
- Constraints linting- review constraints lint checks.
- Constraints analysis- review constraints signoff checks based on the constraints categories, such as case analysis, clock, IO, exception, and others including environmental and design rule.

The following figures explain the recommended methodology for constraints review:





Constraint Signoff - Sample Run Script

The following figure shows the script for a sample run of constraints signoff flow:

## Netlist	## RTL
Step: Design & Constraint Setup	
## Load MMC setup read_mmc <file> ## Load design (Netlist) read_netlist -top <module> <files>	## Setup timing library (optional) input_libs <files> ## Setup timing constraint input_sdc <files> ## Load MMC setup litmus_setup_mmc ## Load design (RTL) elaborate_design -script <script_file> ## <script_file> : read_hdl <files> + elaborate <module>
Step: Design & Constraint Initialization	
## Initialize design database and load timing constraint init_design ## Review constraint lint checks report_sdc -status fail -verbose report_rule_check -category constraint_lint -status fail	## Initialize design database and load timing constraint init_design ## Review constraint lint checks report_sdc -status fail -verbose report_rule_check -category constraint_lint -status fail
Step: Constraint Analysis (Check)	
## Check timing constraint verify_timing_intent	## Check timing constraint verify_timing_intent
Step: Constraint Review (Report & Diagnosis)	
## Review constraint signoff checks report_rule_check -category constraint_signoff -status fail -attribute	## Review constraint signoff checks report_rule_check -category constraint_signoff -status fail -attribute

Hierarchical Constraints

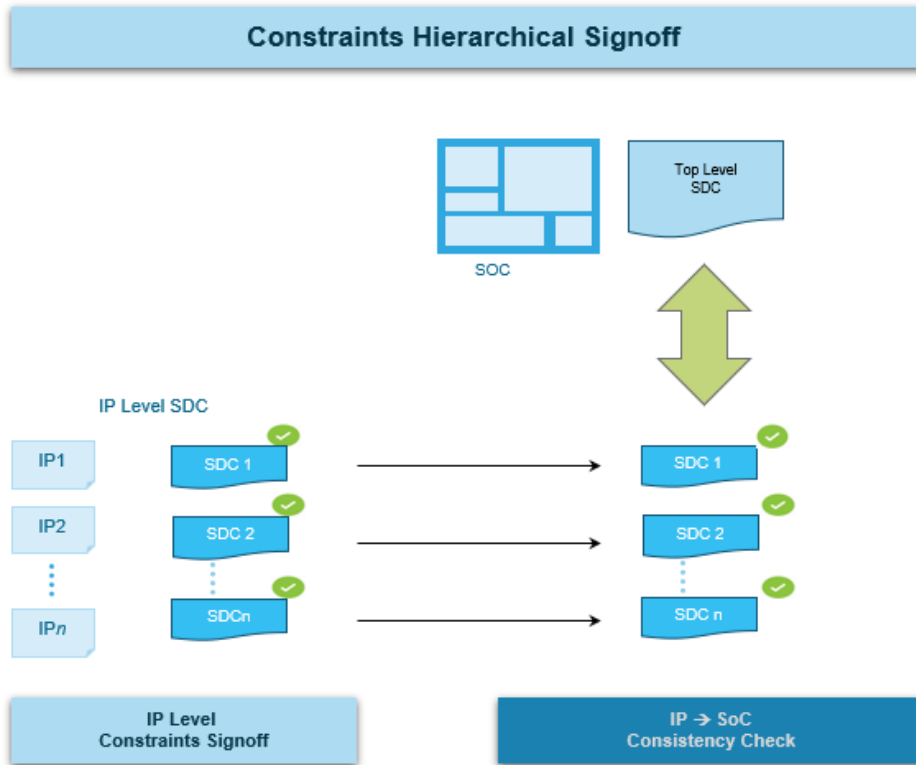
For each class of signoff rule, there is a corresponding set of checks to ensure hierarchical consistency between block-level and top-level constraints.

IP level SoC consistency is at chip integration and design partitioning. It checks that IP level constraints intent is consistent with the SoC level constraints. It enables hierarchical STA methodology.

Litmus supports the following types of consistency:

- Clock consistency
- IO consistency
- Exception consistency
- Case analysis consistency

The following figure shows the Hierarchical implementation and the STA flows.



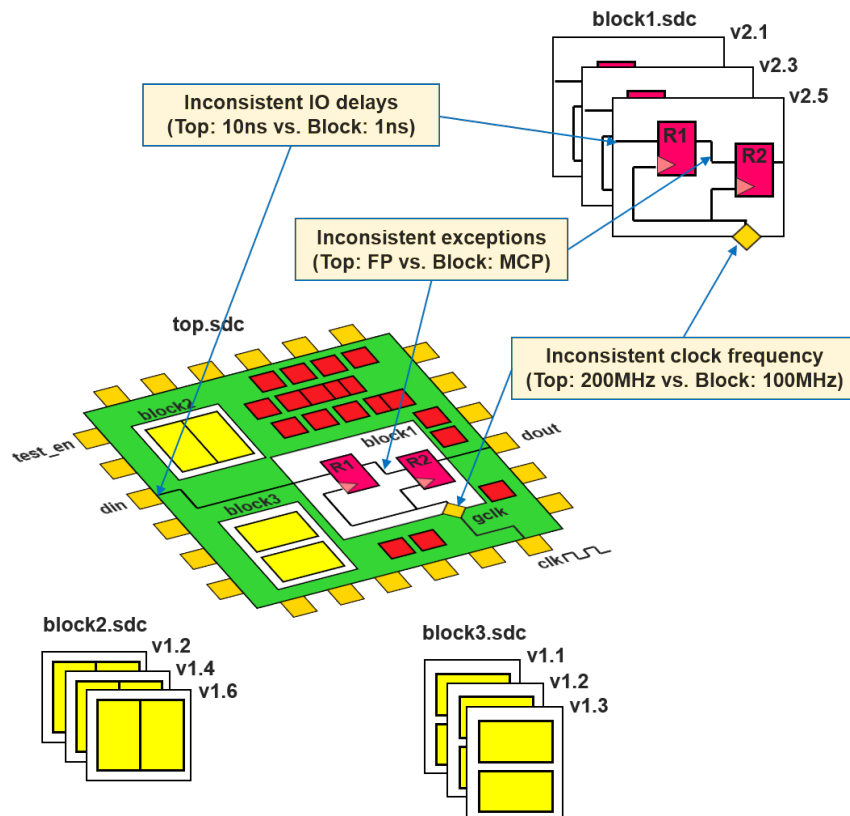
Related Topics

- [Hierarchical Constraints Consistency Checks](#)
- [Hierarchical Constraints Signoff Requirements](#)
- [Hierarchical Constraints Signoff Solution](#)

Hierarchical Constraints Consistency Checks

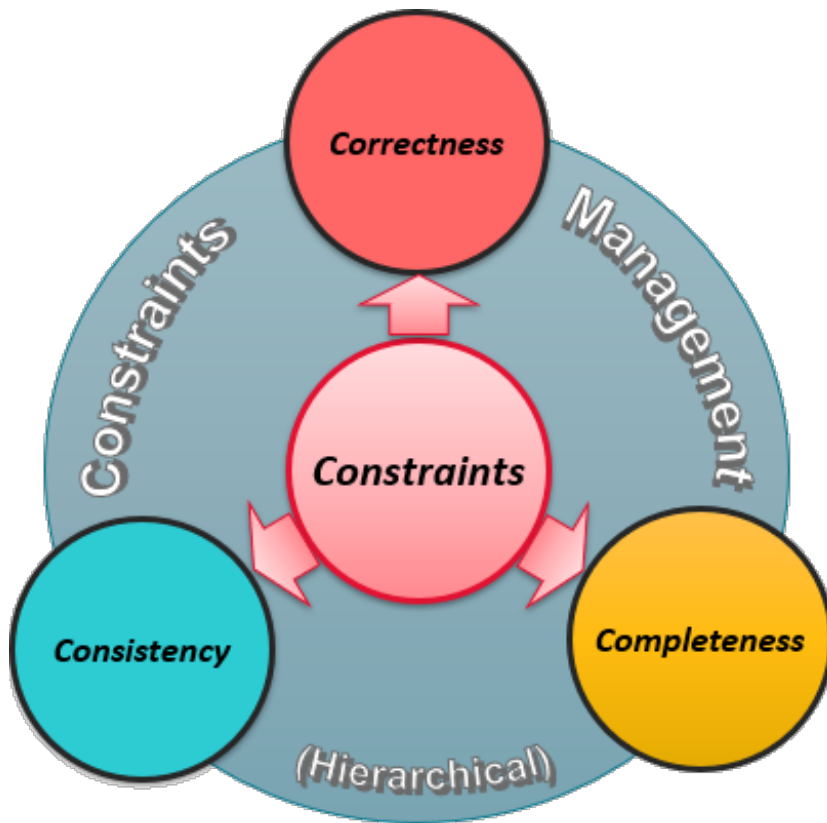
The following are the signoff rules for hierarchical consistency checks:

- Clock constraints
 - `CONST_HIER_CLK*` / `CONST_HIER_UNC*` / `CONST_HIER_LAT*` / `CONST_HIER_CTR*` / `CONST_HIER_GRP*` / `CONST_HIER_CG*` / `CONST_HIER_PLS*`
- IO constraints
 - `CONST_HIER_IO*` / `CONST_HIER_OLD*`
- Exceptions
 - `CONST_HIER_EXC*`
- Case analysis constraints
 - `CONST_HIER_SCA*`
- Others
 - `CONST_HIER_SDT*` / `CONST_HIER_TIM*` / `CONST_HIER_DAT*` / `CONST_HIER_HFN*`



Hierarchical Constraints Signoff Requirements

The following figure shows the requirements for complete coverage of constraints failure modes:



- **Correctness**
 - Language level syntax and semantics checks.
 - Structural checks to ensure correctness.
- **Completeness**
 - Missing constraints.
 - Redundant constraints.
 - Inefficient constraints.
- **Consistency**
 - Conflicting constraints.
 - Inconsistent clock schemes.

- Inconsistent constraints.
- Inconsistent hierarchical constraints.

Hierarchical Constraints Signoff Solution

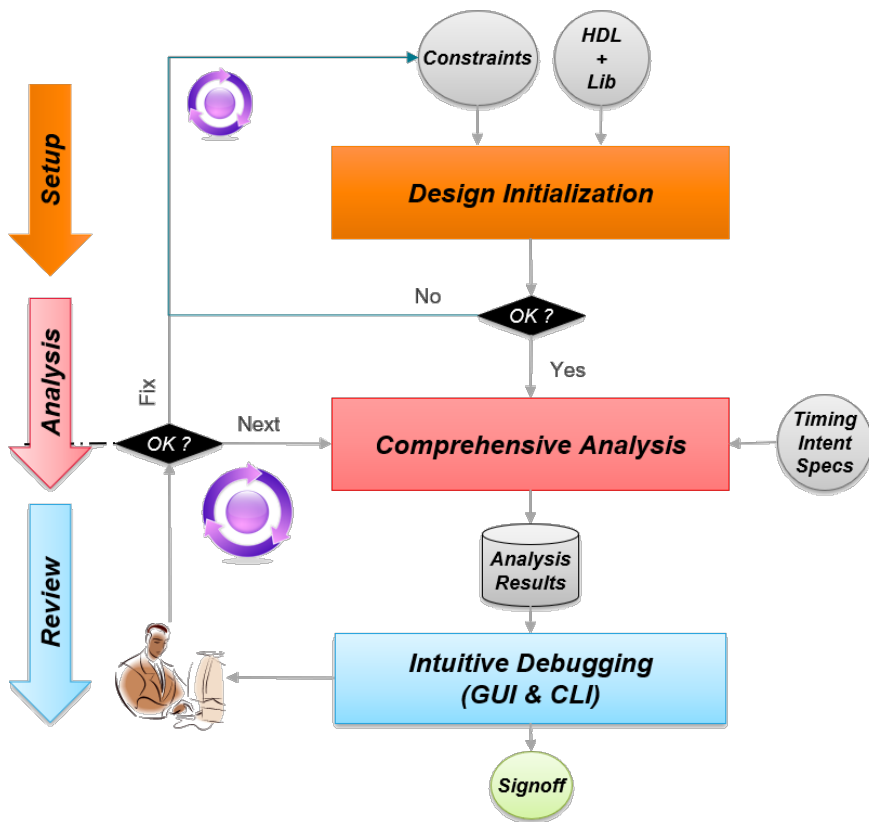
- [Hierarchical Constraints Signoff Use Model](#)
- [Hierarchical Constraints - Design Initialization](#)
- [Hierarchical Constraints - Consistency Analysis](#)
 - [Defining Timing Intent in Hierarchical Constraints Analysis](#)
 - [Performing Hierarchical Constraints Analysis](#)
 - [Hierarchical Constraints Checks](#)
 - [Hierarchical Constraints Analysis Examples](#)
 - [Hierarchical Constraints Analysis Summary](#)
- [Hierarchical Constraints - Consistency Review](#)
 - [Hierarchical Constraints - Consistency Review Methodology](#)
- [Hierarchical Constraint Signoff - Sample Run Script](#)

Hierarchical Constraints Signoff Use Model

Verification is an iterative process that can be minimized by streamlined methodology. Constraints hierarchical signoff flow consists of the following:

- **Setup: Design Initialization**
 - Defines checkpoints in the flow.
 - Specifies constraints that provide full coverage for the design.
 - Validates correctness of the design and constraints setup or coverage.
 - Provides standard CUI setup support, leveraging common design across Cadence implementation/signoff tools.
- **Analysis**
 - Captures timing intent with constraints (hierarchical) signoff checks.
 - Provides additional timing intent specifications that configure rules to meet design methodology.
- **Review: Reporting and Diagnosis**
 - Provides concise and less noisy reports for your review.
 - Annotates failure modes.
 - Provides access to analyze information via CUI DB access commands.

The following figure shows the Constraints Hierarchical Signoff Flow:

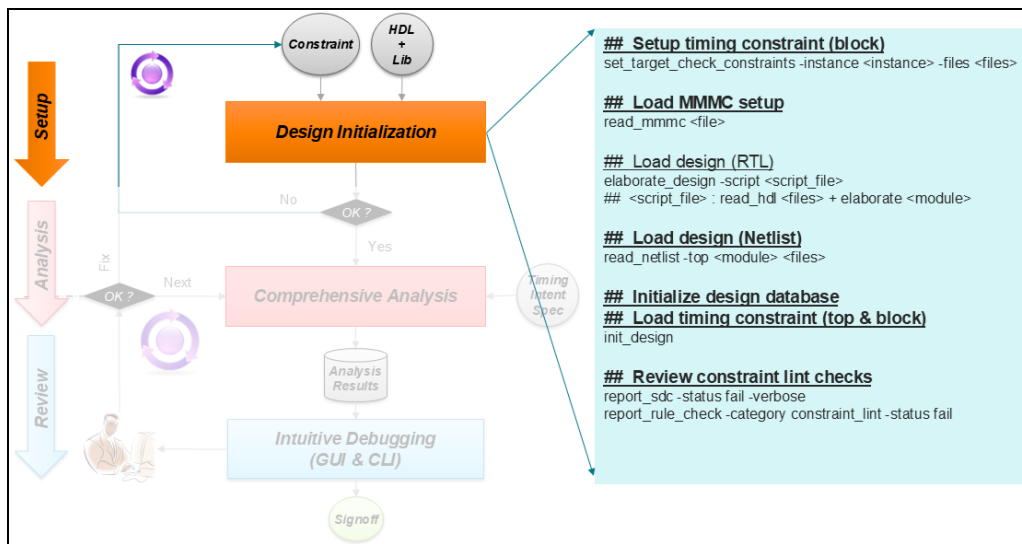


Hierarchical Constraints - Design Initialization

Design initialization consists of the following steps:

- Multi-Mode Multi-Corner (MMMC) initialization flow
- MMMC method for importing design data
- Using only one active view for analysis

The following figure explains the flow:



This step is same as described in the [Constraints - Design Initialization](#) section of Constraints Signoff Solution.

Hierarchical Constraints - Consistency Analysis

Hierarchical Constraints analysis consists of the following:

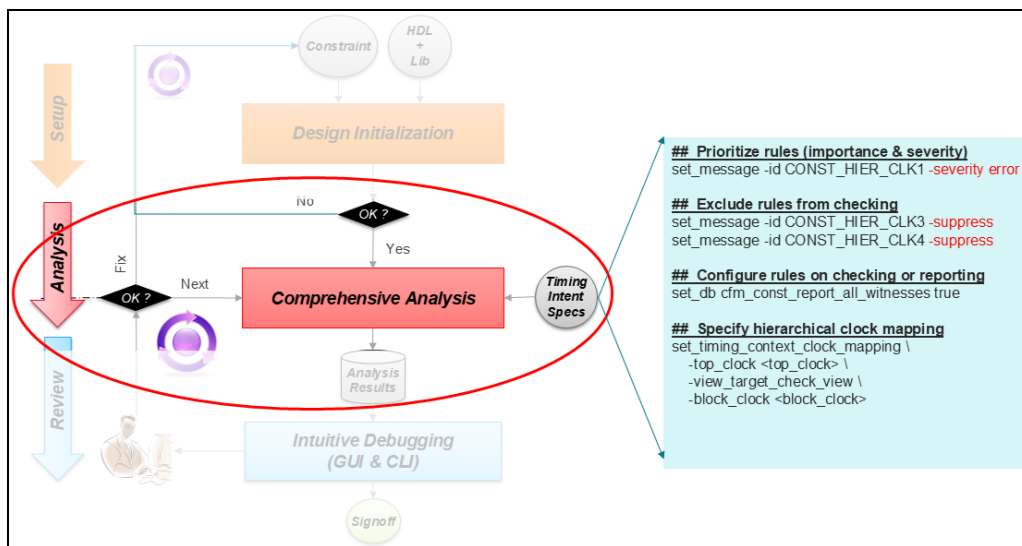
- [Defining Timing Intent in Hierarchical Constraints Analysis](#)
- [Performing Hierarchical Constraints Analysis](#)
- [Hierarchical Constraints Checks](#)
- [Hierarchical Constraints Analysis Examples](#)
- [Hierarchical Constraints Analysis Summary](#)

Defining Timing Intent in Hierarchical Constraints Analysis

Timing intent is defined to meet design and verification methodology. To define timing intent:

1. Prioritize rules based on importance or severity.
2. Exclude rules from checking.
3. Configure rules on checking or reporting.
4. Specify hierarchical clock mapping.

The following figure shows methodology to define timing intent:

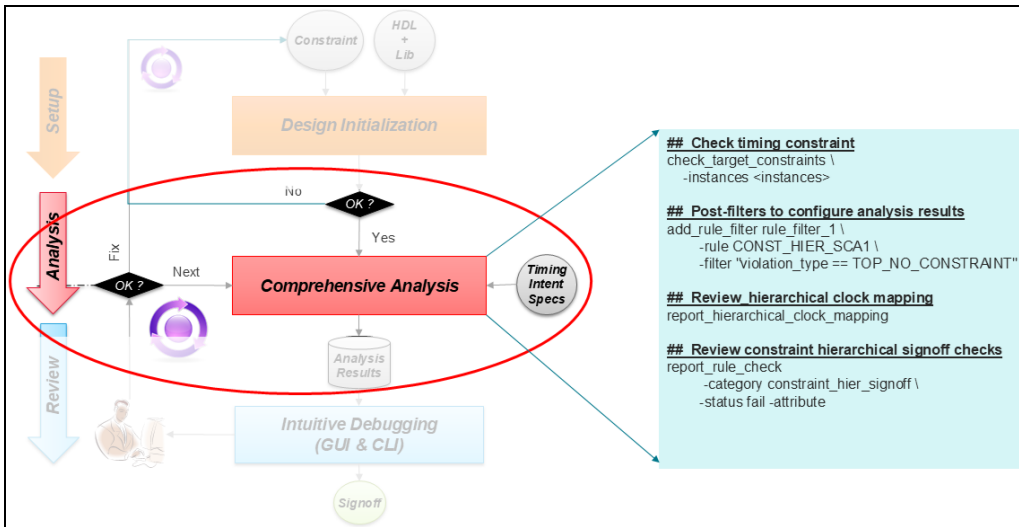


Performing Hierarchical Constraints Analysis

Constraints consistency analysis is run after analysis results configuration and report generation. It includes the following:

1. Commands to verify hierarchical timing intent.
2. Post-filters to configure analysis results.
3. Generate reports for review.
4. Constraints consistency failure modes.

The following figure explains constraints consistency analysis:



Hierarchical Constraints Checks

The following table shows constraints consistency checks based on constraints categories:

Categories	Checks
Clock constraints	CONST_HIER_CLK*: Clock definition CONST_HIER_UNC*: Clock uncertainty CONST_HIER_LAT*: Clock latency CONST_HIER_CTR*: Clock transition CONST_HIER_GRP*: Clock group CONST_HIER_CG*: Clock gating check CONST_HIER_PLS*: Clock minimum pulse width check
IO constraints	CONST_HIER_IO*: Input / Output delay CONST_HIER_OLD*: Output load
Exceptions	CONST_HIER_EXC*: False path / Multicycle path / Delay path (max/min)
Case analysis constraints	CONST_HIER_SCA*: Case analysis check

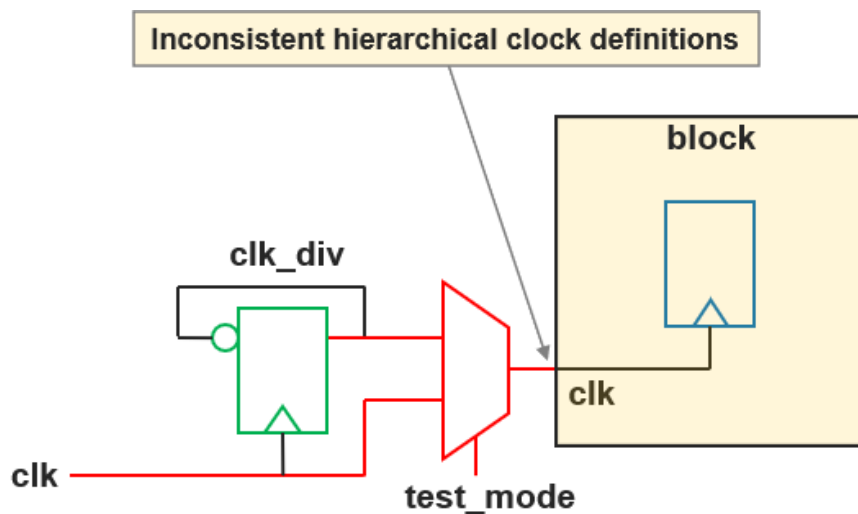
Others	CONST_HIER_SDT*: Disable timing check CONST_HIER_TIM*: Timing path check CONST_HIER_DAT*: Data check CONST_HIER_HFN*: High fanout network check
--------	---

Hierarchical Constraints Analysis Examples

The following are the examples based on constraints categories:

Clock Constraints

The following figure shows an example of clock constraints:



Constraints (top)

```
create_clock -name clk ... [get_ports clk]
create_generated_clock -name clk_div ... [get_pins clk_div/Q]
```

Constraints (block)

```
create_clock -name clk ... [get_ports clk]
```

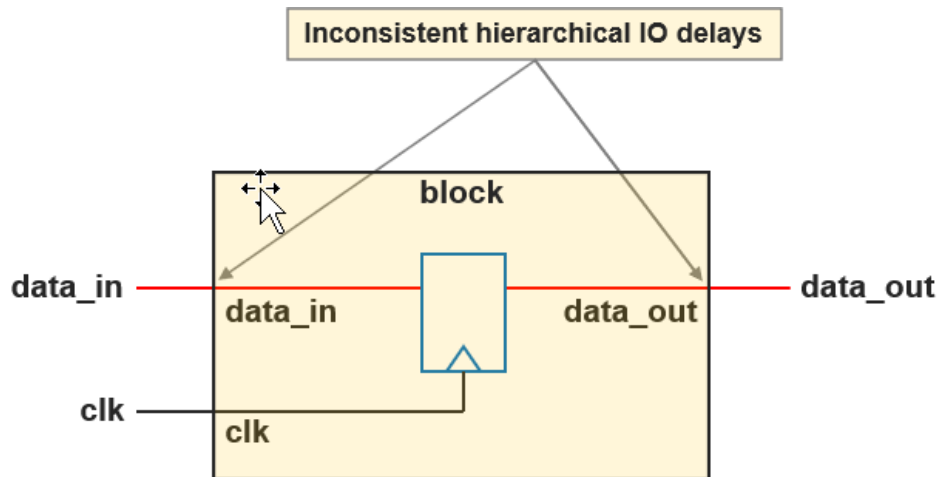
- Failure mode : Consistency
 - Inconsistent hierarchical constraints
- CONST_HIER_CLK1 check flagged

- Hierarchical clock definitions are inconsistent at `block/clock`
 - Top : In the clock network of 2 top clocks `clk` and `clk_div`
 - Block : Only 1 block clock `clk` is defined
- Add the constraint into block to fix the violation :

```
## Constraints (block)
create_clock -name clk_div ... [get_ports clk] -add
```

IO Constraints

The following figure shows an example of IO constraints:



```
## Constraints (top)
create_clock -name vclk ...
set_input_delay -clock [get_clocks vclk] ... [get_ports data_in]
set_output_delay -clock [get_clocks vclk] ... [get_ports data_out]
```

```
## Constraints (block)
create_clock -name vclk ...
```

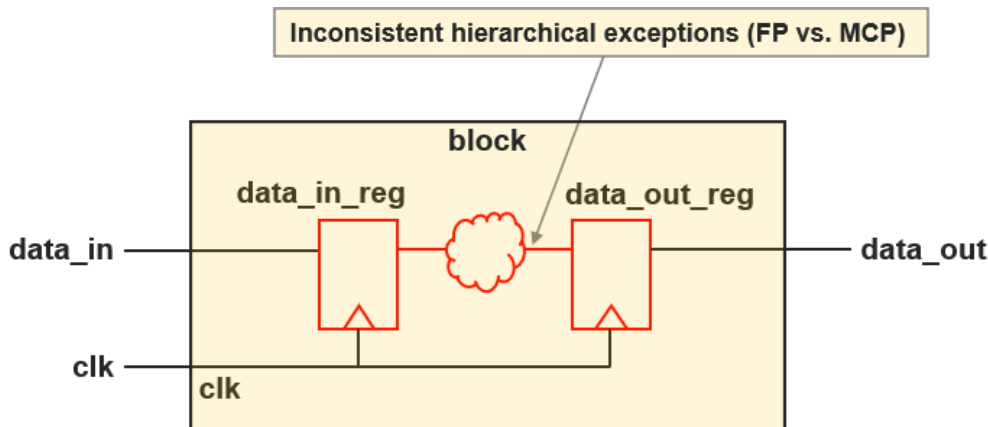
- Failure mode: Consistency
 - Inconsistent hierarchical constraints
- `CONST_HIER_IO2` and `CONST_HIER_IO4` checks flagged
 - Hierarchical IO delays are consistent at `block/ data_in` and `block/ data_out`

- Top: In the timing paths relative to top clock `vclk`
- Block: IO delays relative to block clock `vclk` are missing
- Add the constraints into block to fix the violation:

```
## Constraints (block)
set_input_delay -clock [get_clocks vclk] ... [get_ports data_in]
set_output_delay -clock [get_clocks vclk] ... [get_ports data_out]
```

Exceptions

The following figure shows an example of exceptions:



```
## Constraints (top)
create_clock -name clk ... [get_ports clk]
set_false_path \
    -from [get_cells block/data_in_reg] -to [get_cells block/data_out_reg]
```

```
## Constraints (block)
create_clock -name clk ... [get_ports clk]
set_multicycle_path -setup 4 \
    -from [get_cells data_in_reg] -to [get_cells data_out_reg]
```

- Failure mode: Consistency
 - Inconsistent hierarchical constraints
- `CONST_HIER_EXC4` check flagged
 - Hierarchical exceptions are inconsistent on the timing path from `block/ data_in_reg` to `block/ data_out_reg`

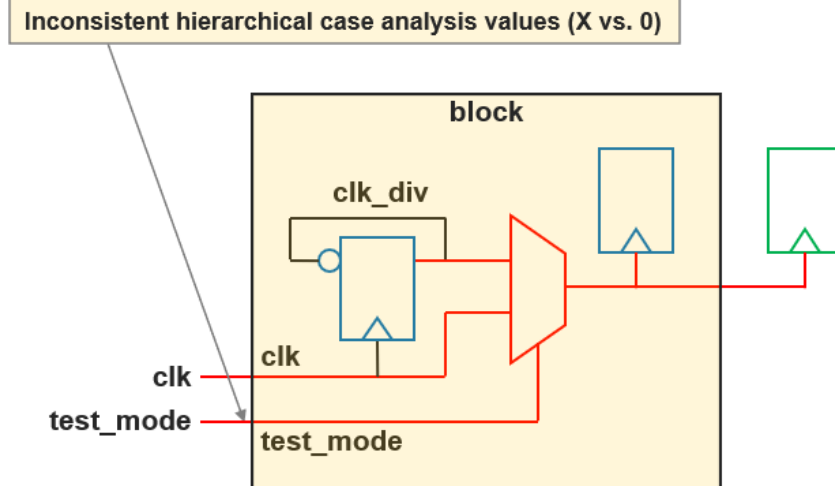
- Top: False path
- Block: Multicycle path
- Review the constraints at block to fix the violation:

```
## Constraints (block) #1
set_multicycle_path -setup 4 \
    -from [get_cells data_in_reg] -to [get_cells data_out_reg]

## Constraints (block) #2
set_false_path \
    -from [get_cells data_in_reg] -to [get_cells data_out_reg]
```

Case Analysis Constraints

The following figure shows an example of case analysis constraints.



```
## Constraints (top)
create_clock -name clk ... [get_ports clk]
create_generated_clock -name clk_div ... [get_pins block/clk_div/Q]

## Constraints (block)
create_clock -name clk ... [get_ports clk]
create_generated_clock -name clk_div ... [get_pins clk_div/Q]
set_case_analysis 0 [get_ports test_mode]
```

- Failure mode: Consistency
 - Inconsistent hierarchical constraints
- CONST_HIER_SCA1 check flagged

- Hierarchical case analysis values are inconsistent at `block/ test_mode`
 - Top: X (No case analysis value specified)
 - Block: 0
- Remove the constraint from block to fix the violation:

```
## Constraints (block)  
set_case_analysis 0 [get_ports test_mode]
```

Hierarchical Constraints Analysis Summary

The following is a sample output report that is generated after completing constraints consistency analysis:

Any violation by report_rule_check

```

CONST_HIER_CLK1: Multiple top-level clocks drive a block level clock
Severity: Warning    Fail Occurrence: 2
CONST_HIER_EXC4: A block's multicycle path is part of a top level false path.
Severity: Warning    Fail Occurrence: 4
CONST_HIER_I0110: Block output is floating
Severity: Warning    Fail Occurrence: 1
CONST_HIER_I02: Block input port is in a timing path within a clock group for which it does not have input delay defined.
Severity: Warning    Fail Occurrence: 2
CONST_HIER_I03: Block output port with delay vs. a clock is not in a timing path within the same clock group at the top level
Severity: Warning    Fail Occurrence: 8
CONST_HIER_I04: Block output port is in a timing path within a clock group for which it does not have output delay defined.
Severity: Warning    Fail Occurrence: 2
CONST_HIER_I06i: Missing set_input_delay in block/glue SDC
Severity: Warning    Fail Occurrence: 4
CONST_HIER_I06o: Missing set_output_delay in block/glue SDC
Severity: Warning    Fail Occurrence: 4
    
```

Details by -verbose -attribute

```

CONST_HIER_CLK1: Multiple top-level clocks drive a block level clock
Severity: Warning    Fail Occurrence: 2
i: Fail: At block 'wctrl', pin 'wctrl/wclk' is in the clock trees of 2 top-level clocks ('clk' , 'wclk'), but has only 1 block clock ('wctrl/wclk') defined.
Attributes:
occurrence_id (unsigned)      : 1
block_view_name (string)      : __target_check_view
top_view_name (string)        : top_view_max
scope (object)                : hinst:fifo/wctrl
block_clock_source (object)    : hpin:fifo/wctrl/wclk
block_clocks (object_list)     : clock:fifo/__target_check_view/wctrl/wclk
top_clocks (object_list)      : clock:fifo/top_view_max/clk clock:fifo/top_view_max/wclk
    
```

Hierarchical Constraints - Consistency Review

Constraints consistency review consists of the following:

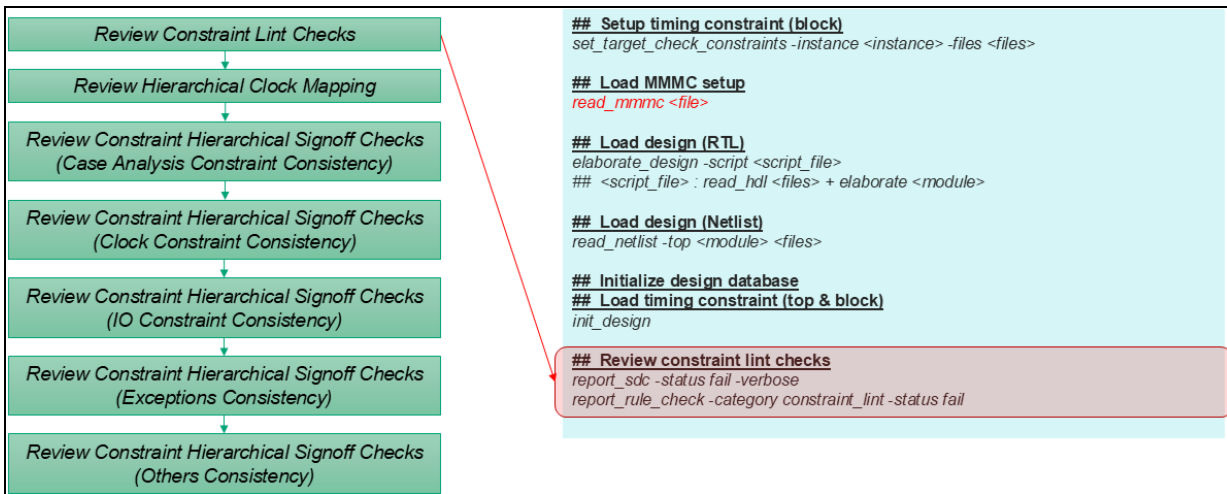
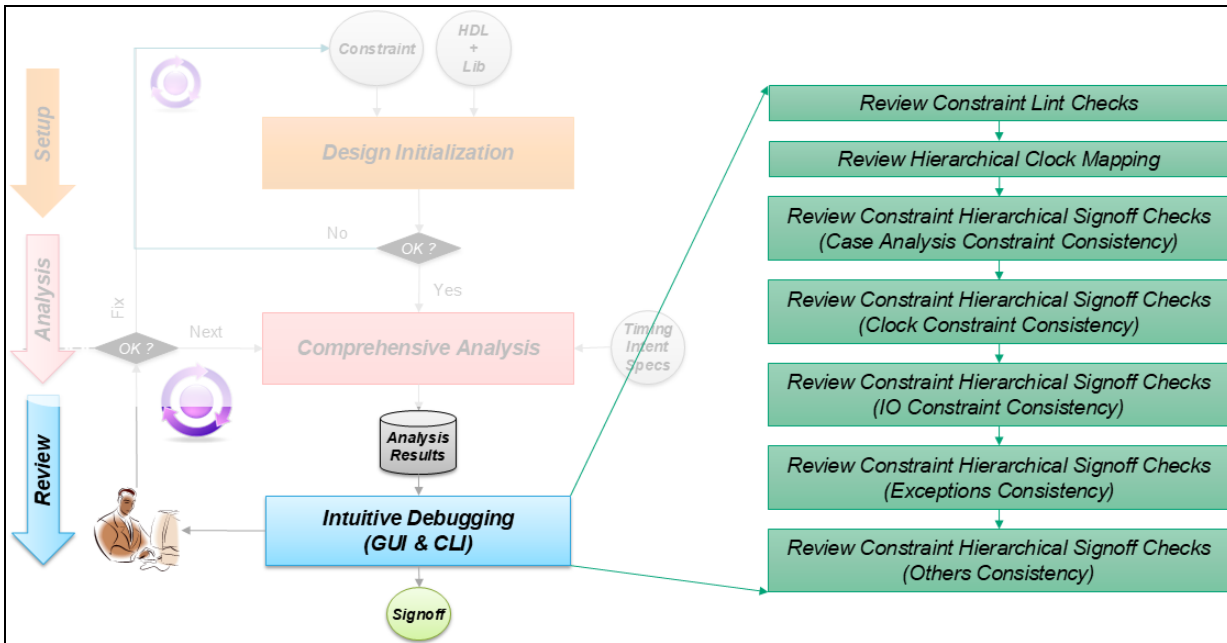
- [Hierarchical Constraints - Consistency Review Methodology](#)

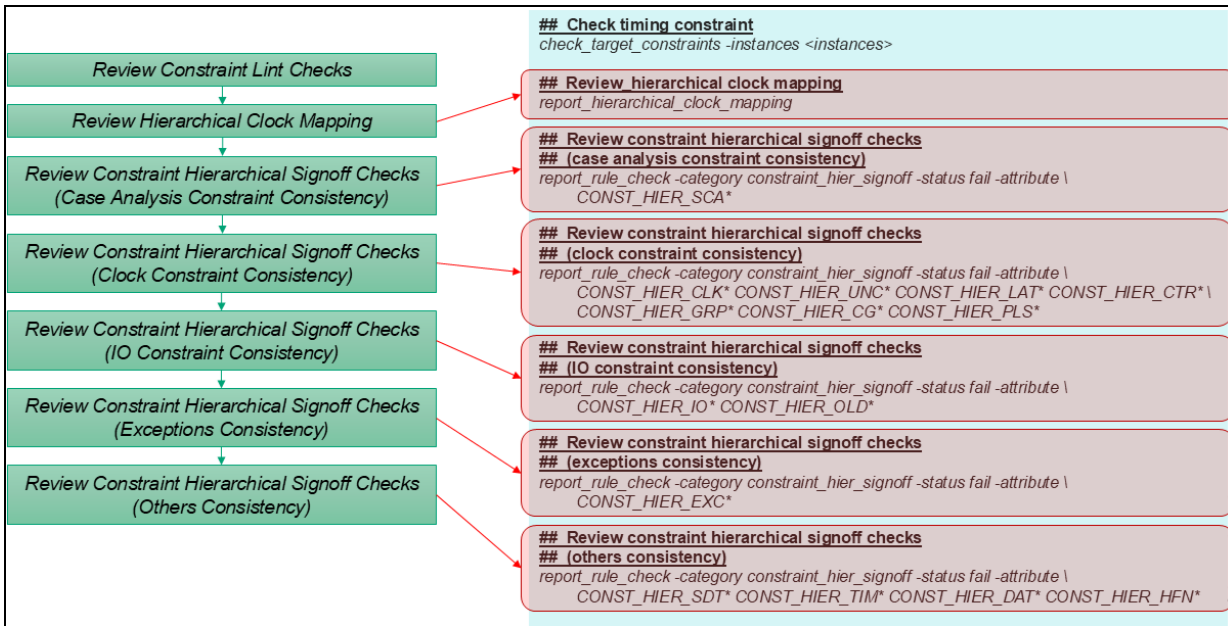
Hierarchical Constraints - Consistency Review Methodology

The recommended methodology for constraints review consists of the following:

- Design setup - review libraries and designs.
- Design Initialization - review libraries lint checks.
- Constraints linting - review constraints lint checks.
- Constraints consistency analysis - review hierarchical clock mapping and constraints hierarchical signoff checks based on the constraints categories.

The following figures explain the recommended methodology for constraints consistency review:





Hierarchical Constraint Signoff - Sample Run Script

The following figure shows the script for a sample run of constraints signoff flow:

## Netlist	## RTL
Step: Design & Constraint Setup	
## Setup timing constraint (block) set_target_check_constraints -instance <instance> -files <files>	## Setup timing constraint (block) set_target_check_constraints -instance <instance> -files <files>
## Load MMC setup read_mmc <file>	## Setup timing library (optional) input_libs <files>
## Load design (Netlist) read_netlist -top <module> <files>	## Setup timing constraint input_sdc <files>
	## Load MMC setup litmus_setup_mmc
	## Load design (RTL) elaborate_design -script <script_file> ## <script_file> : read_hdl <files> + elaborate <module>
Step: Design & Constraint Initialization	
## Initialize design database ## Load timing constraint (top & block) init_design	## Initialize design database ## Load timing constraint (top & block) init_design
## Review constraint lint checks report_sdc -status fail -verbose report_rule_check -category constraint_lint -status fail	## Review constraint lint checks report_sdc -status fail -verbose report_rule_check -category constraint_lint -status fail
Step: Constraint Analysis (Check)	
## Check timing constraint check_target_constraints -instances <instances>	## Check timing constraint check_target_constraints -instances <instances>
Step: Constraint Review (Report & Diagnosis)	

Review_hierarchical clock mapping`report_hierarchical_clock_mapping`**## Review constraint hierarchical signoff checks**`report_rule_check``-category``constraint_hier_signoff``-status fail``-attribute`**## Review\_hierarchical clock mapping**`report_hierarchical_clock_mapping`**## Review constraint hierarchical signoff checks**`report_rule_check``-category constraint_hier_signoff``-status fail``-attribute`

Constraints Comparison

Constraints Comparison provides a structured mechanism to **verify consistency between two constraint views of the same design**. It enables **reference vs target constraint validation**, ensuring that timing intent is preserved across design development, implementation, and constraint evolution flows. Constraints Comparison is conceptually aligned with **Hierarchical Constraints Signoff**, but differs in **scope and comparison dimension**:

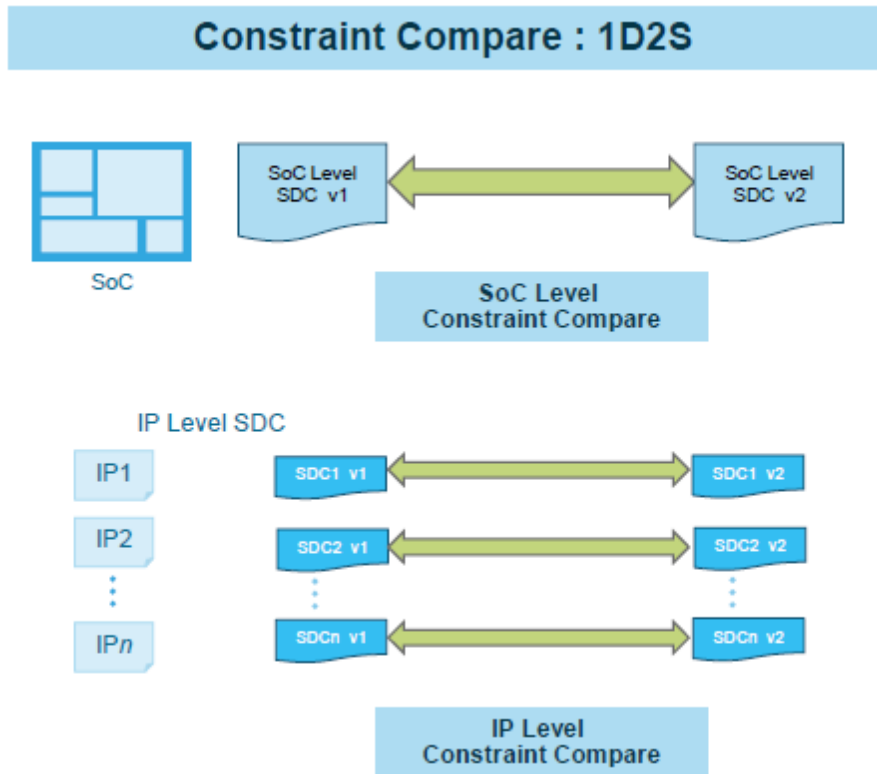
Aspect	Hierarchical Constraints	Constraints Comparison
Comparison scope	Block vs Top	Reference SDC vs Target SDC
Design hierarchy	Multi-level	Same design
Primary intent	Hierarchical consistency	Constraint evolution consistency
Typical usage	IP → SoC integration	Promotion, demotion, refinement
Missing constraint meaning	Hierarchical gap	Violation in target

Reference vs Target Constraint Consistency

Constraints Comparison performs a **direct consistency check between reference and target constraints (1D2S – One Design, Two SDCs)** during:

- Design development
- Design implementation
- Incremental constraint refinement

The comparison framework **detects both missing and mismatched constraints**, enabling early identification of constraint drift.



Violation Rules

Condition	Result
Constraint present in reference but missing in target	Violation
Constraint present in both but values differ	Mismatch
Constraint present only in target	Unique (Target-only)

Constraints Comparison Checks

Constraints Comparison validates consistency across the following constraint classes:

- **Clock Consistency**

- CONST_COMP_CLK* / CONST_COMP_UNC* / CONST_COMP_LAT* / CONST_COMP_CTR* /
CONST_COMP_GRP* / CONST_COMP_CG* / CONST_COMP_PLS*

- **IO Consistency**

- CONST_COMP_IO*

- **Exception Consistency**

- CONST_COMP_EXC*

- **Case Analysis Consistency**

- CONST_COMP_SCA*

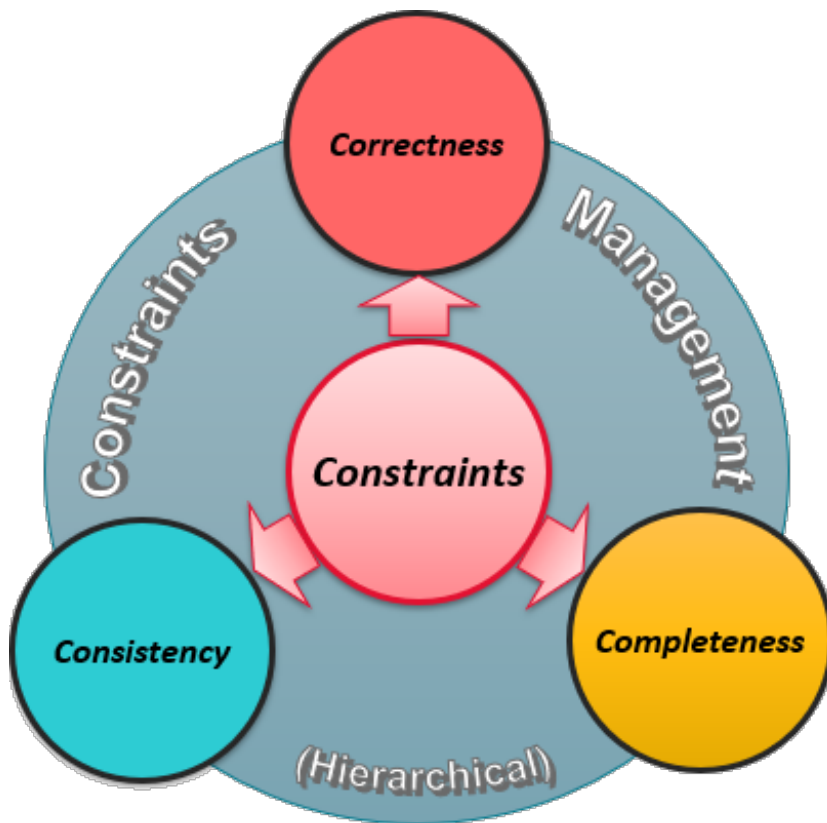
- **Others**

- CONST_COMP_SDT* / CONST_COMP_TIM* / CONST_COMP_DAT* / CONST_COMP_HFN*

These dimensions mirror hierarchical consistency checks, but are applied **across SDC versions instead of hierarchy levels**.

Constraints Comparison Signoff Requirements

Constraints Comparison follows the same **signoff quality principles** as hierarchical constraints:



Correctness

- Valid SDC syntax and semantics
- Proper object resolution

Completeness

- Missing constraints in target
- Partial constraint coverage

Consistency

- Conflicting or modified timing intent
- Clock and exception inconsistencies

Supported Comparison Modes

Timing-Aware Constraint Comparison (1D2S)

Timing-aware comparison analyzes constraints **in the context of design timing topology**, including clock trees and timing paths. It detects:

- Clock hierarchy mismatches
- Period, latency, and uncertainty differences
- Generated clock inconsistencies
- Exception intent conflicts

Results are reported using **rule-based signoff checks**.

Timing-Unaware Constraint Comparison (2SDC)

Timing-unaware comparison performs a **pure SDC-to-SDC comparison**, independent of timing graph analysis. Its key characteristics include:

- Faster execution

- Structural and semantic SDC comparison
- Useful for early validation and auditing

 **Note:** 2SDC Compare provides a timing-unaware, high-performance constraint comparison with structured tabular reporting, automatically identifying mismatches, unique constraints, and anchor points for scalable analysis of large designs.

Running Constraint Comparison

`compare_constraints [-sdc_compare]`

- `compare_constraints`
 - Runs **timing-aware + timing-unaware** comparison
 - **Timing-Aware Reports (Signoff Rules)**
 - `report_rule_check CONST_COMP* -verbose -attribute`
 - Each reported violation includes:
 - Reference and target object mapping
 - Constraint attributes from both views
 - Root cause classification (missing, mismatched, inconsistent)
- `compare_constraints -sdc_compare`
 - Runs **timing-unaware only (2SDC comparison)**
 - **Timing-Unaware Reports**
 - `report_sdc [-sdc_compare] [-verbose]`
 - Reports SDC statistics by default, with `-sdc_compare`, additionally reports:
 - Matched constraints
 - Mismatched constraints
 - Unique constraints

Constraints Comparison Examples

Example 1 - create_clock

Reference SDC:

```
create_clock -period 10 -name CLK -add [get_ports clk]
```

Target SDC:

```
create_clock -period 10 -name CLK -waveform {0 4.5} -add [get_ports clk]
```

1D2S Compare:

```
CONST_COMP_CLK2: A target clock and a related reference have different duty cycle or waveform
Severity: Warning    Fail Occurrence: 1
1: Fail: At target, the source object 'clk' of target clock 'CLK' is in clock tree of reference-level clock 'CLK' but it has different duty-cycle than the latter (0,450000 vs 0,500000) and differnt waveform ({0,000000 4,500000} vs {0,000000 5,000000})

Attributes:
occurrence_id (unsigned)      : 1
target_view_name (string)     : __target_check_view
reference_view_name (string)  : top_view_max
target_clock (object)         : clock:TOP/__target_check_view/CLK
target_clock_source (object)  : port:TOP/clk
reference_clock (object)      : clock:TOP/top_view_max/CLK
target_file_line (unsigned)   : 1
target_file_name (string)     : target.sdc
reference_file_line (unsigned): 1
reference_file_name (string)  : ref.sdc
target_time_period (double)   : 10,0
reference_time_period (double): 10,0
```

2SDC Compare:

Details									
Command	Anchor	Option	Reference	Value Target	Reference	Differences Line Target	Reference	File Target	
create_clock	CLK	waveform(trail)	5.0	4.5	1	1	ref.sdc	target.sdc	

Unique in Reference View				
Command	Anchor	Line	File	

Unique in Target View				
Command	Anchor	Line	File	

Summary					
Command	Unique Reference	Unique Target	Different	Matched	
create_clock	0	0	1	0	

Example 2 - create_generated_clock

Reference SDC:

```
set inst1 blk/create_clock -period 10 -name CLK -waveform {0 5} [get_ports
clk]create_clock -period 20 -name CLK2 -waveform {0 10} [get_ports clk] -
addcreate_generated_clock -name GCLK -source [get_ports clk] -divide_by 2 -master_clock
[get_clocks CLK] -add -invert [get_pins ${inst1}bufclk/Y]
```

Target SDC:

```
set inst blk/create_clock -period 10 -name CLK -waveform {0 5} [get_ports
clk]create_clock -period 20 -name CLK2 -waveform {0 10} [get_ports clk] -
addcreate_generated_clock -name GCLK -source [get_ports clk] -edges {1 3 5} [get_pins
${inst}bufclk/Y]
```

1D2S Compare:

CONST_COMP_CLK2d: A target clock and a related reference clock have different phases

Severity: Warning Fail Occurrence: 1

1: Fail: At target, the source object 'blk/bufclk/Y' of target clock 'GCLK' with Positive polarity is in the clock tree of reference clock 'GCLK' with Negative polarity.

Attributes:

occurrence_id (unsigned)	: 1
target_view_name (string)	: __target_check_view
reference_view_name (string)	: top_view_max
target_clock (object)	: clock:TOP/_target_check_view/GCLK
target_port (object)	: pin:TOP/blk/bufclk/Y
reference_clock (object)	: clock:TOP/top_view_max/GCLK
target_polarity (string)	: Positive
reference_polarity (string)	: Negative
target_file_line (unsigned)	: 6
target_file_name (string)	: target.sdc
reference_file_line (unsigned)	: 6
reference_file_name (string)	: ref.sdc

2SDC Compare:

Details								
Command	Anchor	Option	Reference	Value		Differences		File
				Reference	Target	Reference	Line Target	
create_generated_clock	GCLK							
		invert	1	0	6	6		ref.sdc target.sdc
		master_clock	CLK	-	6	6		ref.sdc target.sdc
		divide_by	2	-	6	6		ref.sdc target.sdc
		edges	-	1 3 5	6	6		ref.sdc target.sdc

Unique in Reference View			
Command	Anchor	Line	File

Unique in Target View			
Command	Anchor	Line	File

Summary				
Command	Unique		Different	Matched
	Reference	Target		
create_generated_clock	0	0	1	0
create_clock	0	0	0	2

Example 3 - set_case_analysis

Reference SDC:

```
set inst1 blk/create_clock -period 10 -name CLK -waveform {0 5} [get_ports
clk]create_clock -period 20 -name CLK2 -waveform {0 10} [get_ports clk] -
addset_input_delay -clock [get_clocks CLK] -rise -max -network_latency_included -
source_latency_included 3.0 [get_pins ${inst1}dinff/CLK]
```

Target SDC:

```
set inst blk/create_clock -period 10 -name CLK -waveform {0 5} [get_ports
clk]create_clock -period 20 -name CLK2 -waveform {0 10} [get_ports clk] -
addset_input_delay -clock [get_clocks CLK] -rise -max -source_latency_included 3.0
[get_pins ${inst}dinf/CLK]
```

1D2S Compare:

```
CONST_COMP_I012i: Inconsistent set_input_delay at the reference-level vs. target
Severity: Warning    Fail Occurrence: 1
1: Fail: Inconsistency in set_input_delay delay value/corner is found on target.
Reference: set_input_delay vs. clock CLK (without -clock_fall) at following pin: blk/dinf/CLK.
Target : set_input_delay vs. clock CLK (without -clock_fall) at following pin: blk/dinf/CLK.
Attributes:
    occurrence_id (unsigned)           : 1
    target_view_name (string)          : __target_check_view
    reference_view_name (string)       : top_view_max
    target_pin (object)                : pin:TOP/blk/dinf/CLK
    reference_pin (object)             : pin:TOP/blk/dinf/CLK
    target_clock (object)              : clock:TOP/__target_check_view/CLK
    reference_clock (object)           : clock:TOP/top_view_max/CLK
    type (string)                     : DIFFERENT_INPUT_DELAYS
    target_file_line (object_list)     : 6
    target_file_name (object_list)     : target.sdc
    reference_file_line (object_list)  : 6
    reference_file_name (object_list)  : ref.sdc
```

2SDC Compare:

Details								
Command	Anchor	Option	Reference	Value Target	Reference	Differences Line Target	Reference	File Target
set_input_delay	pin:TOP/blk/dinf/CLK(CLK)							
		network_latency_included(min, rise)	1	0	6	6	ref.sdc	target.sdc
		network_latency_included(min, fall)	1	0	6	6	ref.sdc	target.sdc
		network_latency_included(max, rise)	1	0	6	6	ref.sdc	target.sdc
		network_latency_included(max, fall)	1	0	6	6	ref.sdc	target.sdc

Unique in Reference View			
Command	Anchor	Line	File

Unique in Target View			
Command	Anchor	Line	File

Summary				
Command	Unique Reference	Unique Target	Different	Matched
set_input_delay	0	0	1	0
create_clock	0	0	0	2

Example 4 - set_input_delay

Reference SDC:

```
set inst1 blk/create_clock -period 10 -name CLK -waveform {0 5} [get_ports
clk]create_clock -period 20 -name CLK2 -waveform {0 10} [get_ports clk] -
addset_input_delay -clock [get_clocks CLK] -rise -max -network_latency_included -
source_latency_included 3.0 [get_pins ${inst1}dinf/CLK]
```

Target SDC:

```
set inst blk/create_clock -period 10 -name CLK -waveform {0 5} [get_ports
clk]create_clock -period 20 -name CLK2 -waveform {0 10} [get_ports clk] -
addset_input_delay -clock [get_clocks CLK] -rise -max -source_latency_included 3.0
[get_pins ${inst}dinf/CLK]
```

1D2S Compare:

```
CONST_COMP_I012i: Inconsistent set_input_delay at the reference-level vs. target
Severity: Warning    Fail Occurrence: 1
1: Fail: Inconsistency in set_input_delay delay value/corner is found on target.
Reference: set_input_delay vs. clock CLK (without -clock_fall) at following pin: blk/dinf/CLK.
Target : set_input_delay vs. clock CLK (without -clock_fall) at following pin: blk/dinf/CLK.
Attributes:
    occurrence_id (unsigned)           : 1
    target_view_name (string)          : __target_check_view
    reference_view_name (string)       : top_view_max
    target_pin (object)                : pin:TOP/blk/dinf/CLK
    reference_pin (object)              : pin:TOP/blk/dinf/CLK
    target_clock (object)               : clock:TOP/__target_check_view/CLK
    reference_clock (object)            : clock:TOP/top_view_max/CLK
    type (string)                      : DIFFERENT_INPUT_DELAYS
    target_file_line (object_list)     : 6
    target_file_name (object_list)     : target.sdc
    reference_file_line (object_list)  : 6
    reference_file_name (object_list)  : ref.sdc
```

2SDC Compare:

Details									
Command	Anchor	Option	Reference	Value Target	Reference	Differences Line Target	Reference	File Target	
set_input_delay	pin:TOP/blk/dinf/CLK(CLK)								
		network_latency_included(min, rise)	1	0	6	6	ref.sdc	target.sdc	
		network_latency_included(min, fall)	1	0	6	6	ref.sdc	target.sdc	
		network_latency_included(max, rise)	1	0	6	6	ref.sdc	target.sdc	
		network_latency_included(max, fall)	1	0	6	6	ref.sdc	target.sdc	
Unique in Reference View									
Command	Anchor	Line	File						
Unique in Target View									
Command	Anchor	Line	File						
Summary									
Command	Unique Reference	Unique Target	Different	Matched					
set_input_delay	0	0	1	0					
create_clock	0	0	0	2					

Example 5 - set_multicycle_path

Reference SDC:

```
set inst1 blk/create_clock -name CLK -period 10 [get_ports clk]create_clock -name CLK2
```

```
-period 20 [get_ports clk] -addset_multicycle_path -setup -start 3 -from [get_clocks
CLK] -through [get_pins ${inst1}dlnff/Q] -to [get_clocks CLK]
```

Target SDC:

```
set inst blk/create_clock -name CLK -period 10 [get_ports clk]create_clock -name CLK2 -
period 20 [get_ports clk] -addset_multicycle_path -setup -start 3 -from [get_clocks
CLK] -through [get_pins ${inst}dlnff/Q] -to [get_clocks CLK2]
```

1D2S Compare:

```
CONST_COMP_EXC5: A target single cycle path is part of a reference multicycle path.
Severity: Warning      Fail Occurrence: 1
1: Fail: Inconsistency in path exception is found
Reference: 'TOP'
Target: 'TOP'
Sample path at reference: from blk/dlnff/CLK (clock: CLK) to blk/dffout/DATA (clock: CLK)
Sample path at target 'TOP': from blk/dlnff/CLK (clock: CLK) to blk/dffout/DATA (clock: CLK)
Attributes:
  occurrence_id (unsigned)      : 1
  target_view_name (string)     : __target_check_view
  reference_view_name (string)  : top_view_max
  reference_file_line (unsigned): 6
  reference_file_name (string)  : ref.sdc
  target_path_startpoint (object): pin:TOP/blk/dlnff/CLK
  reference_path_startpoint (object): pin:TOP/blk/dlnff/CLK
  target_path_endpoint (object): pin:TOP/blk/dffout/DATA
  reference_path_endpoint (object): pin:TOP/blk/dffout/DATA
  reference_exc_cycle (int)     : 3
  reference_exc_cycle_on_start (string): true
  reference_corner (string)     : -late
  reference_exc_through (object_list): pin:TOP/blk/dlnff/Q
  reference_exc_type (string)   : set_multicycle_path
  reference_exc_from_clock (object_list): clock:TOP/top_view_max/CLK
  reference_exc_to_clock (object_list): clock:TOP/top_view_max/CLK
  target_path_startpoint_clock (object): clock:TOP/__target_check_view/CLK
  target_path_endpoint_clock (object): clock:TOP/__target_check_view/CLK
  reference_path_startpoint_clock (object): clock:TOP/top_view_max/CLK
  reference_path_endpoint_clock (object): clock:TOP/top_view_max/CLK
```

2SDC Compare:

Details									
Command	Anchor	Option	Reference	Value	Target	Reference	Line	Target	File
Unique in Reference View									
Command	Anchor				Line		File		
set_multicycle_path	-from CLK -through {blk/dinf/Q} -to CLK (setup, rise)				6	ref.sdc			
	-from CLK -through {blk/dinf/Q} -to CLK (setup, fall)				6	ref.sdc			
Unique in Target View									
Command	Anchor				Line		File		
set_multicycle_path	-from CLK -through {blk/dinf/Q} -to CLK2 (setup, rise)				6	target.sdc			
	-from CLK -through {blk/dinf/Q} -to CLK2 (setup, fall)				6	target.sdc			
Summary									
	Unique								
	Reference	Target	Different	Matched					
multicycle_path	2	2	0	0					
_clock	0	0	0	2					

Example 6 - set_max_delay

Reference SDC:

```
set inst1 blk/create_clock -name CLK -period 10 [get_ports clk]create_clock -name CLK2 -
-period 20 [get_ports clk] -addset_max_delay 3.0001 -from [get_clocks CLK2] -through
[get_pins ${inst1}dinf/Q] -to [get_clocks CLK2]
```

Target SDC:

```
set inst blk/create_clock -name CLK -period 10 [get_ports clk]create_clock -name CLK2 -
period 20 [get_ports clk] -addset_max_delay 3 -from [get_clocks CLK] -through [get_pins
${inst}dffout/DATA] -to [get_clocks CLK2]
```

1D2S Compare:

CONST_COMP_EXC10: A target path without set_max_delay/set_min_delay is part of a reference path with max/min delay settings.

Severity: Warning Fail Occurrence: 1

1: Fail: Inconsistency in path exception is found

Reference: 'TOP'

Target: 'TOP'

Sample path at reference: from blk/dinf/CLK (clock: CLK2) to blk/dffout/DATA (clock: CLK2)

Sample path at target 'TOP': from blk/dinf/CLK (clock: CLK2) to blk/dffout/DATA (clock: CLK2)

Attributes:

occurrence_id (unsigned)	: 1
target_view_name (string)	: __target_check_view
reference_view_name (string)	: top_view_max
reference_file_line (unsigned)	: 6
reference_file_name (string)	: ref.sdc
target_path_startpoint (object)	: pin:TOP/blk/dinf/CLK
reference_path_startpoint (object)	: pin:TOP/blk/dinf/CLK
target_path_endpoint (object)	: pin:TOP/blk/dffout/DATA
reference_path_endpoint (object)	: pin:TOP/blk/dffout/DATA
reference_value (float)	: 3.0001
reference_corner (string)	: -late
reference_exc_through (object_list)	: pin:TOP/blk/dinf/Q
reference_exc_type (string)	: set_max_delay
reference_exc_from_clock (object_list)	: clock:TOP/top_view_max/CLK2
reference_exc_to_clock (object_list)	: clock:TOP/top_view_max/CLK2
target_path_startpoint_clock (object)	: clock:TOP/__target_check_view/CLK2
target_path_endpoint_clock (object)	: clock:TOP/__target_check_view/CLK2
reference_path_startpoint_clock (object)	: clock:TOP/top_view_max/CLK2
reference_path_endpoint_clock (object)	: clock:TOP/top_view_max/CLK2

2SDC Compare:

Details									
Command	Anchor	Option	Reference	Value	Target	Reference	Line	Target	File
=====									
Unique in Reference View									
Command	Anchor						Line		File
set_max_delay	-from CLK2	-through {blk/dinf/Q}	-to CLK2	(late, rise)			6		ref.sdc
	-from CLK2	-through {blk/dinf/Q}	-to CLK2	(late, fall)			6		ref.sdc
=====									
Unique in Target View									
Command	Anchor						Line		File
set_max_delay	-from CLK	-through {blk/dffout/DATA}	-to CLK2	(late, rise)			6		target.sdc
	-from CLK	-through {blk/dffout/DATA}	-to CLK2	(late, fall)			6		target.sdc
=====									
Summary									
Command	Unique	Reference	Target	Different	Matched				
set_max_delay	2	2	0	0					
create_clock	0	0	0	2					

Example 7 - set_false_path

Reference SDC:

```
set inst1 blk/create_clock -name CLK -period 10 [get_ports clk]create_clock -name CLK2
```

```
-period 20 [get_ports clk] -addset_false_path -from [get_clocks CLK] -through [get_pins
${inst1}dffout/DATA] -to [get_clocks CLK2]
```

Target SDC:

```
set inst blk/create_clock -name CLK -period 10 [get_ports clk]create_clock -name CLK2 -
period 20 [get_ports clk] -addset_max_delay 3 -from [get_clocks CLK] -through [get_pins
${inst}dffout/DATA] -to [get_clocks CLK2]
```

1D2S Compare:

```
CONST_COMP_EXC2: A target false path (FP) is part of a reference path without set_false_path.
Severity: Warning    Fail Occurrence: 1
1: Fail: Inconsistency in path exception is found
Reference: 'TOP'
Target: 'TOP'
Sample path at reference: from blk/dinf/CLK (clock: CLK2) to blk/dffout/DATA (clock: CLK2)
Sample path at target 'TOP': from blk/dinf/CLK (clock: CLK2) to blk/dffout/DATA (clock: CLK2)
Attributes:
  occurrence_id (unsigned)      : 1
  target_view_name (string)     : __target_check_view
  reference_view_name (string)  : top_view_max
  target_file_line (unsigned)   : 6
  target_file_name (string)     : target.sdc
  target_path_startpoint (object) : pin:TOP/blk/dinf/CLK
  reference_path_startpoint (object) : pin:TOP/blk/dinf/CLK
  target_path_endpoint (object)  : pin:TOP/blk/dffout/DATA
  reference_path_endpoint (object) : pin:TOP/blk/dffout/DATA
  target_corner (string)        : -early -late
  target_exc_through (object_list) : pin:TOP/blk/dinf/Q
  target_exc_type (string)       : set_false_path
  target_exc_from_clock (object_list) : clock:TOP/__target_check_view/CLK2
  target_exc_to_clock (object_list)  : clock:TOP/__target_check_view/CLK2
  target_path_startpoint_clock (object) : clock:TOP/__target_check_view/CLK2
  target_path_endpoint_clock (object)  : clock:TOP/__target_check_view/CLK2
  reference_path_startpoint_clock (object) : clock:TOP/top_view_max/CLK2
  reference_path_endpoint_clock (object)  : clock:TOP/top_view_max/CLK2
```

2SDC Compare:

Details									
Command	Anchor	Option	Reference	Value	Reference	Line	Reference	File	
				Target		Target		Target	
=====									
Unique in Reference View									
Command	Anchor						Line	File	
set_false_path	-from CLK -through {blk/dffout/DATA} -to CLK2 (hold, rise)						6	ref.sdc	
	-from CLK -through {blk/dffout/DATA} -to CLK2 (hold, fall)						6	ref.sdc	
	-from CLK -through {blk/dffout/DATA} -to CLK2 (setup, rise)						6	ref.sdc	
	-from CLK -through {blk/dffout/DATA} -to CLK2 (setup, fall)						6	ref.sdc	
=====									
Unique in Target View									
Command	Anchor						Line	File	
set_false_path	-from CLK2 -through {blk/dinf/Q} -to CLK2 (hold, fall)						6	target.sdc	
	-from CLK2 -through {blk/dinf/Q} -to CLK2 (setup, rise)						6	target.sdc	
	-from CLK2 -through {blk/dinf/Q} -to CLK2 (hold, rise)						6	target.sdc	
	-from CLK2 -through {blk/dinf/Q} -to CLK2 (setup, fall)						6	target.sdc	
=====									
Summary									
Command	Unique	Unique	Different	Matched					
	Reference	Target							
set_false_path	4	4	0	0					
create_clock	0	0	0	2					

Example 8 - Physically Exclusive Clock Exceptions

In cases where exception constraints specify physically exclusive launch and capture clock pairs using `-from` and `-to`, no timing violation is reported by analysis. **2-SDC Compare** still detects and reports this constraint difference, ensuring such intent mismatches are visible during constraint comparison even when timing behavior is unaffected.

1D2S performs a timing-aware comparison, so when the launch and capture clocks specified in `-from` and `-to` are physically exclusive by design, no timing violation is reported and the constraint difference is effectively filtered out. As a result, 1D2S does not surface this mismatch, even though the constraint definitions differ, whereas 2-SDC Compare reports it explicitly.

Reference SDC:

```
create_clock -name CLK -period 10 [get_ports clk]
create_clock -name CLK2 -period 20 [get_ports clk]
-add
```

Target SDC:

```
set inst blk/create_clock -name CLK -period 10 [get_ports clk]
create_clock -name CLK2 -period 20 [get_ports clk]
-add
set_max_delay 3 -from [get_clocks CLK] -through [get_pins ${inst}dffout/DATA] -to [get_clocks CLK2]
```

1D2S Compare:

```
CONST_COMP_EXC2: A target false path (FP) is part of a reference path without set_false_path.
Severity: Warning    Fail Occurrence: 1
1: Fail: Inconsistency in path exception is found
Reference: 'TOP'
Target: 'TOP'
Sample path at reference: from blk/dinf/CLK (clock: CLK2) to blk/dffout/DATA (clock: CLK2)
Sample path at target 'TOP': from blk/dinf/CLK (clock: CLK2) to blk/dffout/DATA (clock: CLK2)
Attributes:
occurrence_id (unsigned)           : 1
target_view_name (string)          : __target_check_view
reference_view_name (string)        : top_view_max
target_file_line (unsigned)         : 6
target_file_name (string)           : target.sdc
target_path_startpoint (object)     : pin:TOP/blk/dinf/CLK
reference_path_startpoint (object)   : pin:TOP/blk/dinf/CLK
target_path_endpoint (object)       : pin:TOP/blk/dffout/DATA
reference_path_endpoint (object)     : pin:TOP/blk/dffout/DATA
target_corner (string)              : -early -late
target_exc_through (object_list)    : pin:TOP/blk/dinf/Q
target_exc_type (string)            : set_false_path
target_exc_from_clock (object_list) : clock:TOP/__target_check_view/CLK2
target_exc_to_clock (object_list)   : clock:TOP/__target_check_view/CLK2
target_path_startpoint_clock (object): clock:TOP/__target_check_view/CLK2
target_path_endpoint_clock (object) : clock:TOP/__target_check_view/CLK2
reference_path_startpoint_clock (object): clock:TOP/top_view_max/CLK2
reference_path_endpoint_clock (object): clock:TOP/top_view_max/CLK2
```

2SDC Compare:

=====			
Unique in Target View			
Command	Anchor	Line	File

set_max_delay	-from CLK -through {blk/dffout/DATA} -to CLK2 (late, rise)	6	target.sdc
	-from CLK -through {blk/dffout/DATA} -to CLK2 (late, fall)	6	target.sdc
=====			

 When exception constraints specify launch and capture clocks that are physically exclusive by design, no timing violation is reported. In this scenario, the constraint `CONST_COMP_EXC11` is therefore ignored by timing analysis. However, 2-SDC Compare still captures and reports a constraint difference, ensuring that mismatches in constraint intent between Reference and Target SDCs are not missed.

Example 9 - Comprehensive Example - 2SDC Compare

Reference SDC:

```
##-----  
## Clocks  
##-----  
  
create_clock -period 10 -waveform {0 5} -name CK1 [get_ports clk1]  
create_clock -period 8 -waveform {0 4} -name CK2 [get_ports clk3]  
create_generated_clock -divide_by 2 -name GCK3 -source [get_ports clk1] -master_clock  
CK1 [get_pins I1/U1/X] -add  
  
##-----  
## Constants  
##-----  
  
set_case_analysis 0 [get_ports din4]  
  
##-----  
## I/O Delays  
##-----  
  
set_input_delay 2 -clock CK2 [get_ports din1]  
  
##-----  
## Exceptions  
##-----  
  
set_multicycle_path 2 -from [get_ports din1]  
set_max_delay 3 -from [get_ports din3] -to [get_pins B3/out_reg/D]  
set_false_path -through [get_pins I3/U1/X]
```

Target SDC:

```
##-----  
## Clocks  
##-----  
  
create_clock -period 10 -waveform {0 5} -name CK1 [get_ports clk1]  
create_clock -period 8 -waveform {0 4} -name CK2 [get_ports clk3]  
create_generated_clock -divide_by 2 -name GCK3 -source [get_ports clk1] -master_clock  
CK1 [get_pins I1/U1/X] -add  
  
##-----  
## Constants  
##-----  
  
set_case_analysis 0 [get_ports din4]  
  
##-----  
## I/O Delays  
##-----  
  
set_input_delay 2 -clock CK2 [get_ports din1]  
  
##-----  
## Exceptions  
##-----  
  
set_multicycle_path 2 -from [get_ports din1]  
set_max_delay 3 -from [get_ports din3] -to [get_pins B3/out_reg/D]  
set_false_path -through [get_pins I3/U1/X]
```

2SDC Compare:

Details								
Command	Anchor	Option	Reference	Value Target	Reference	Differences Line Target	Reference	File Target
set_multicycle_path	-from din1 (setup, fall)	value	2,0	3,0	22	22	ref.sdc	target.sdc
	-from din1 (setup, rise)	value	2,0	3,0	22	22	ref.sdc	target.sdc
set_max_delay	-from din3 -to B3/out_reg/D (late, fall)	value	3,0	4,0	23	23	ref.sdc	target.sdc
	-from din3 -to B3/out_reg/D (late, rise)	value	3,0	4,0	23	23	ref.sdc	target.sdc
set_input_delay	port:test/din1(CK2)	value(min, rise)	2,0	-	17	17	ref.sdc	target.sdc
		value(min, fall)	2,0	-	17	17	ref.sdc	target.sdc
		value(max, rise)	2,0	3,0	17	17	ref.sdc	target.sdc
		value(max, fall)	2,0	3,0	17	17	ref.sdc	target.sdc
set_case_analysis	port:test/din4	value	0	1	12	12	ref.sdc	target.sdc
create_generated_clock	GCK3	divide_by	2	3	7	7	ref.sdc	target.sdc
create_clock	CK2	waveform(lead)	0,0	3,0	6	6	ref.sdc	target.sdc
		waveform(trail)	4,0	5,0	6	6	ref.sdc	target.sdc
	CK1	period	10,0	9,0	5	5	ref.sdc	target.sdc

Unique in Reference View			
Command	Anchor	Line	File

Unique in Target View			
Command	Anchor	Line	File
set_false_path	-through {din3} (hold, rise)	24	target.sdc
	-through {din3} (hold, fall)	24	target.sdc
	-through {din3} (setup, rise)	24	target.sdc
	-through {din3} (setup, fall)	24	target.sdc
		24	target.sdc

Summary				
Command	Unique Reference	Unique Target	Different	Matched
set_multicycle_path	0	0	2	1
set_max_delay	0	0	2	0
set_input_delay	0	0	1	0
set_case_analysis	0	0	1	0
create_generated_clock	0	0	1	0
set_false_path	0	4	0	1
create_clock	0	0	2	0

Constraints Comparison - Sample Run Script

The following shows the script for a sample run of constraints comparison flow:

```
# Load reference SDC
input_sdc <sdc_reference_file>
## Example
## input_sdc sdc/fifo_demo.sdc.AFIFO.reference

# Setup MMC
litmus_setup_mmc

# Elaborate and initialize design
elaborate_design -script <script_file>
## Example
## elaborate_design -script script/read_hdl.tcl
init_design

# Compare constraints
compare_constraints

# Review failures
report_rule_check CONST_COMP* -status fail
report_rule_check CONST_COMP* -status fail -verbose -attribute
```

 Note:

For 1D2S:

```
compare_constraintsredirect check1d2s.rpt {report_rule_check CONST_COMP* -
verbose}
```

For 2SDC:

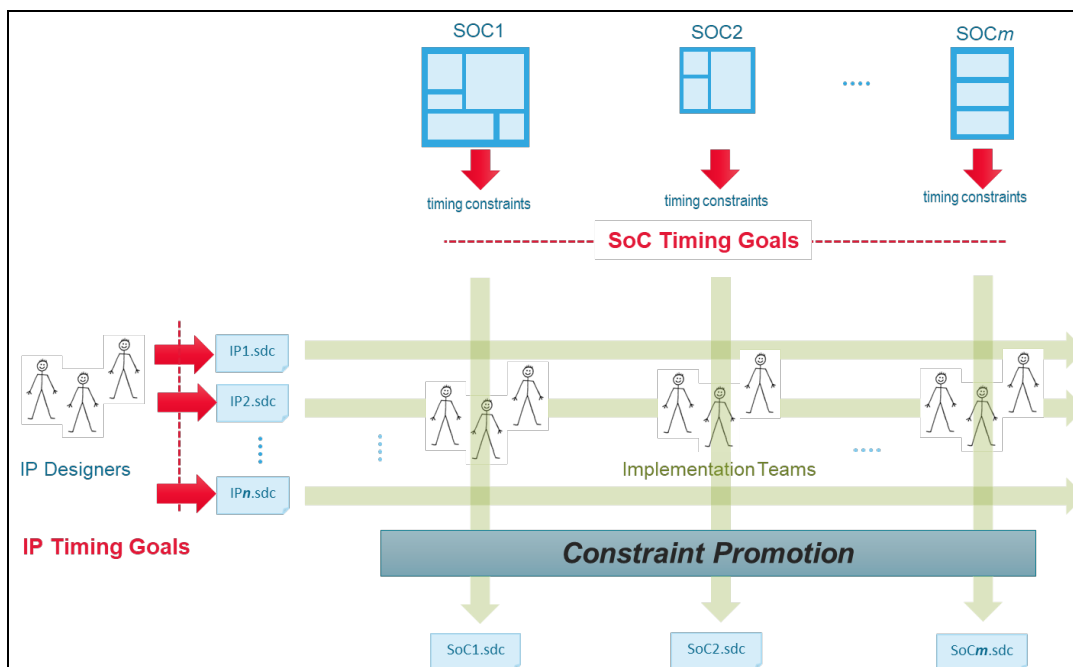
```
compare_constraints -sdc_comparerredirect check2sdc.rpt {report_sdc -sdc_compare -
verbose}
```

For 1D2S and 2SDC in same session:

```
compare_constraintsredirect check1d2s.rpt {report_rule_check CONST_COMP* -
verbose}redirect check2sdc.rpt {report_sdc -sdc_compare -verbose}
```

Constraints Promotion

Modern SoC development brings together a large number of independently designed IP blocks, each with its own timing requirements, clock structures, and exception constraints captured in SDC format. During integration, these block-level SDCs must be correctly interpreted, migrated, verified, and merged into a unified SoC-level constraint set. Manual promotion of block constraints is time-consuming and error-prone. Missing clocks, incorrect IO delays, inconsistent exceptions, or mismatched case analyses result in SoC timing closure failure. **Litmus Constraint Promotion** provides an automated, structured, auditable flow that **promotes block constraints into the SoC design** in a deterministic and rules-driven manner.



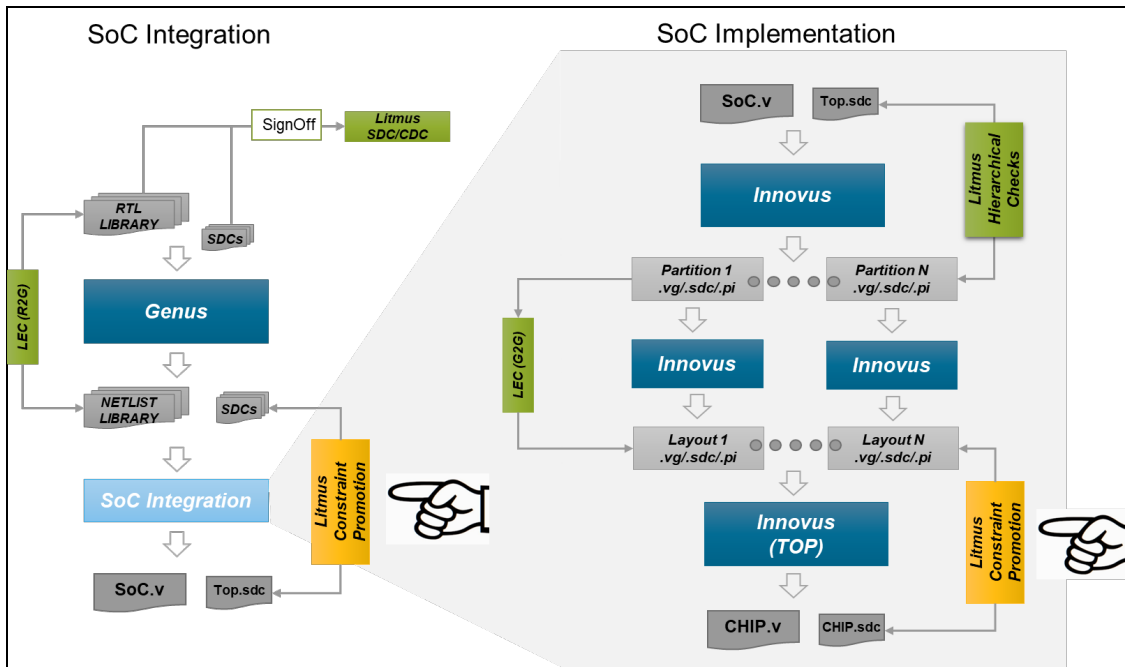
In Litmus, Constraint Promotion is the process of **guided migration** of block-level constraints to the SoC-level environment. It ensures:

- **Consistency** between block and SoC intent
- **Completeness** of SoC-level SDCs
- **Auditability** via Constraints Promotion Tags (CP-tags)
- Deterministic conflict resolution
- Reduced human effort and mistakes

The following figure highlights how constraint promotion acts as a unifying bridge across both SoC integration and implementation phases.

- During integration, Litmus ensures that timing intent, defined at the block level, is accurately propagated into the SoC timing environment before handoff to signoff checks.

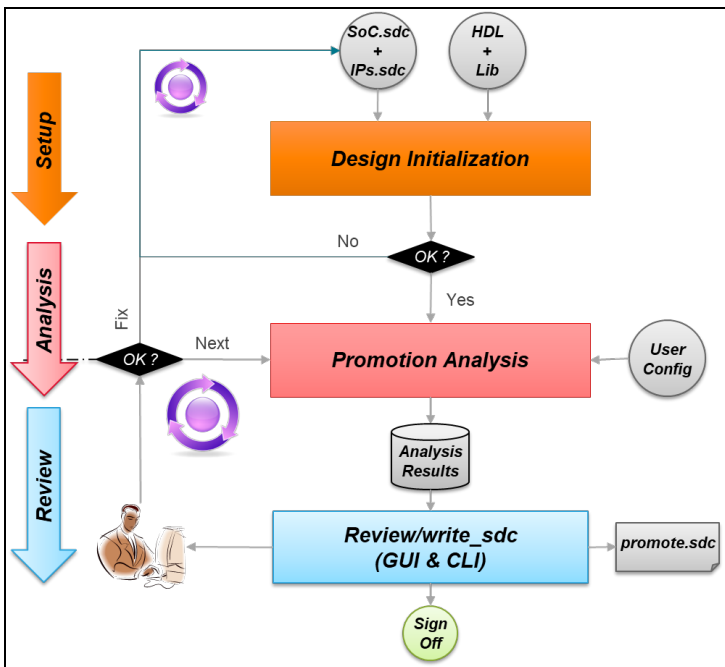
- As the design moves into physical implementation, Innovus partitions consume the promoted SoC-level constraints, and Litmus again validates and promotes hierarchical constraints during partition and top-level assembly.



By appearing consistently at each key transition: RTL to netlist, block to SoC, and partition to top-level, constraint promotion ensures that timing intent is preserved, conflicts are eliminated early, and the final chip-level SDC reflects a coherent, validated view of the entire hierarchy.

Constraint Promotion Flow

Constraint Promotion in Litmus follows a structured three-stage progression: **Setup**, **Analysis**, and **Review** to migrate block-level design constraints to the SoC level. Although promotion is inherently iterative, Litmus minimizes the number of user iterations by enforcing a consistent flow, providing contextual diagnostics, and enabling targeted user inputs only when necessary. The following figure illustrates this flow:



Setup Phase

The *Setup phase* establishes a stable and unified design context before any constraint processing occurs. During this step, Litmus:

- Loads the SoC-level constraints and design data (RTL, netlist, or Innovus database).
- Registers block/IP-level SDCs using a consistent hierarchical mechanism.
- Performs SDC lint checks and structural validation to ensure the design is ready for promotion.
- Simplifies the ingestion of block and SoC constraints so downstream analysis can occur without ambiguity.

A clean Setup phase reduces downstream iteration and ensures consistent interpretation of constraint intent across all blocks.

Analysis Phase

The *Analysis phase* performs the actual constraint promotion. Processing occurs in an orderly, block-by-block sequence, driven by both tool defaults and optional user configuration. During this phase, Litmus:

- Iterates through constraint types in a well-defined order (constants, clocks, IO delays, exceptions).
- Applies incremental propagation of clocks and constants to establish a stable timing foundation.
- Categorizes every examined constraint into a promotion outcome using CP-tags, enabling rapid diagnosis and resolution.
- Accepts targeted user input, such as defining block priority or specifying promotion strategies, only where conflicts or ambiguities arise.

This stage is intentionally structured to minimize unnecessary iteration while ensuring full transparency into how each constraint is interpreted and promoted.

Review Phase

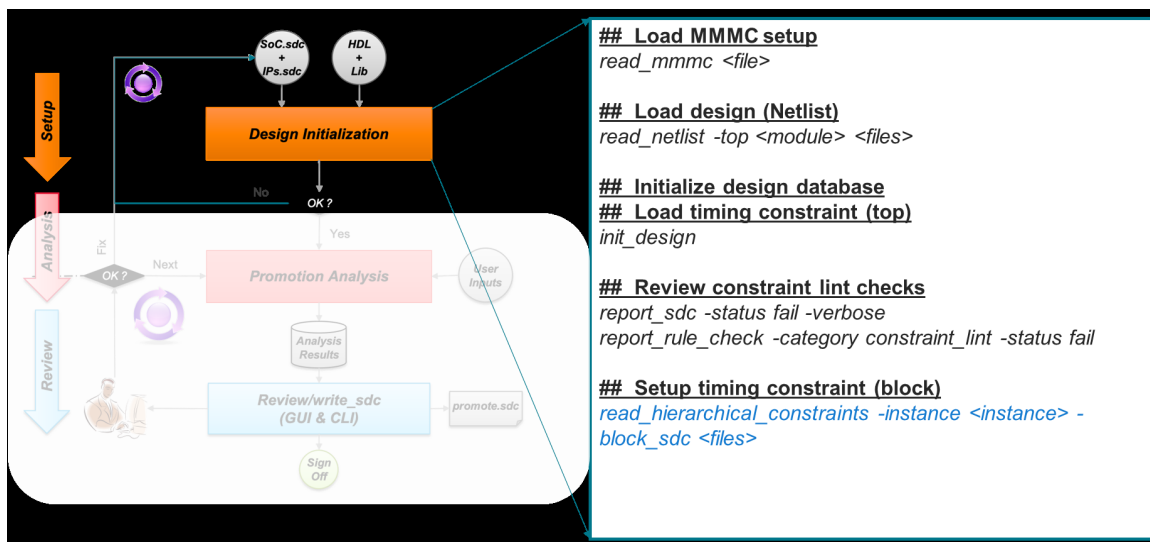
The *Review phase* consolidates the results of the Analysis and enables the engineer to quickly assess and finalize the promoted constraint set. Litmus provides:

- Clean, low-noise reports organized by constraint type, promotion result, and severity.
- Both GUI and CLI environments for reviewing CP-tag annotations, mismatches, and unresolved conflicts.
- A configurable `write_sdc` output mechanism that generates a unified, signoff-ready top-level SDC (`promote.sdc`) based on user preferences.

Once reviewed and validated, the resulting SoC-level SDC becomes the authoritative input for downstream signoff and implementation flows.

Design Initialization and Constraint Setup

This phase establishes the foundational timing context required before constraint promotion can begin. This phase ensures that all SoC-level and block-level inputs are consistently loaded, the design database is fully initialized, and timing constraints are structurally valid. A robust initialization significantly reduces downstream iterations during promotion analysis. The figure below illustrates the Design Initialization step as part of the setup stage:



During this stage, Litmus validates basic design readiness, catches early constraint-lint issues, and registers block-level SDCs for later promotion.

Reading Block-Level Constraints

A central task in design initialization is correctly associating block or instance-specific SDC files with the SoC context. Litmus provides the unified `read_hierarchical_constraints` command for loading top-level and block-level constraint files.

Command Usage

```
read_hierarchical_constraints [-help] -top_sdc <string> \  
    | { -block_sdc <string> { -block <string> | -instances <string> } }
```

Options

-help	Prints usage information.
-top_sdc <string>	Specifies top-level SDCs. These must be loaded separately from block-level SDCs.
-block <string>	Specifies the module name when applying the block-level SDC to all instances of that module.
-block_sdc <string>	Specifies the block-level SDC file(s) to register for promotion.
-instances <string>	Associates the block-level SDC with one or more explicit instances.

Note:

- Top-level and block-level SDCs cannot be loaded in a **single command call**. Use multiple commands and load the SoC constraints first using `-top_sdc`, then load block constraints via `-block_sdc`.
- When a module has **multiple instances**, the `-block` option allows the command to automatically associate the same block SDC with all corresponding instances.
- The `read_hierarchical_constraints` command does not load or apply constraints into the timing database immediately. Instead, it registers and validates the referenced SDC files, and performs checks such as file existence and syntax, so that they can be applied later during the `promote_hierarchical_constraints` step.

Examples

Load block SDC for a specific instance

```
read_hierarchical_constraints -block_sdc module_a.sdc -instance instance_a
```

Load block SDC for all instances of a module

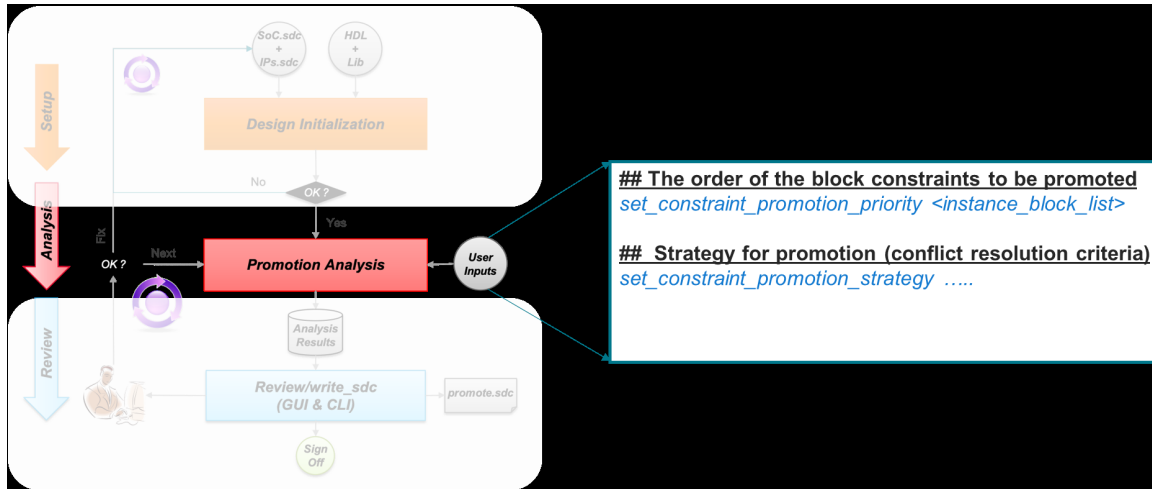
```
read_hierarchical_constraints -block_sdc module_a.sdc -block module_a
```

Load multiple block constraints (mixed module-based and instance-based)

```
read_hierarchical_constraints -block_sdc module_a.sdc -block module_a  
read_hierarchical_constraints -block_sdc module_b.sdc -instance instance_b  
read_hierarchical_constraints -block_sdc module_c.sdc -instance instance_c
```

Promotion Configuration

Constraint Promotion supports a set of configuration controls that specifies how block-level constraints are processed, prioritized, and resolved during the analysis phase. These configurations ensure constraint ordering and conflict-resolution. The following figure highlights where user-defined configuration is applied during promotion analysis:



Configuration consists of two primary settings:

1. **Promotion Priority**
2. **Promotion Strategy**

Both settings can be applied before initiating constraint promotion.

Promotion Priority

The promotion priority list determines the sequence in which block/instance-level constraints are analyzed. By default, Litmus processes the constraints **in alphabetical order of instance names**, but you can override this sequence to align with integration priorities or known design dependencies.

Command Usage

```
set_constraint_promotion_priority [-help] [<obj_list>]
```

Options

-help	Prints command usage.
<obj_list>	Specifies an ordered list of module names and/or instance names.

 Note:

- Constraint promotion for each constraint type is performed in a **block-by-block** sequence.
- If you do not specify a list, the order defaults to **alphabetical instance ordering**.
- When a user-defined list is used, **any missing instances** are automatically **appended** to the end of the list so that the final internal sequence covers *all* instances with associated block constraints.

This ensures that more critical blocks or blocks with complex clocks/exceptions are promoted early.

Promotion Strategy

The **promotion strategy** determines how conflicts and overlaps are resolved between block-level constraints and top-level constraints. You can refine the command based on constraint type or object type.

Command Usage

```
set_constraint_promotion_strategy [-help] -method <enum> \  
[-name <string>] [-types <type_list>] [-object_type <obj_type>]
```

Options

-help	Prints out the command usage.
-method <string>	Selects the promotion strategy. (enum, required) • Available Methods: ◦ <code>block_worst_case</code> (<i>default</i>) — Retains the most conservative constraint. ◦ <code>block_priority</code> — Prioritizes block-level constraints over top-level definitions. ◦ <code>block_ignore</code> — Ignores the specified constraint class during promotion.
-name <string>	Specifies the strategy name (string, optional)

-types <type_list>	Applies the strategy to one or more specific constraint types. (string, optional) • Valid Constraint Types: ◦ case_analysis ◦ clock ((covers create_clock, create_generated_clock) ◦ input_delay ◦ output_delay ◦ clock_latency ◦ clock_transition ◦ clock_uncertainty ◦ clock_gating ◦ disable_clock_gating ◦ load ◦ min_fanout, max_fanout ◦ min_trans, max_trans ◦ ideal_network ◦ false_path ◦ multicycle_path ◦ min_delay ◦ max_delay ◦ clock_group ◦ slew ((covers set_input_transition, set_drive, set_driving_cell)
-object_type <obj_type>	Limits the strategy to specific object types. (enum, optional) • Valid Object Types: ◦ pin ◦ clock ◦ net ◦ lib_pin ◦ promoted_clock ◦ mapped_clock

Examples

Override default behavior and use block priority globally:

```
set_constraint_promotion_strategy -method block_priority
```

Ignore all slew-related constraints from blocks:

```
set_constraint_promotion_strategy -method block_ignore -type slew
```

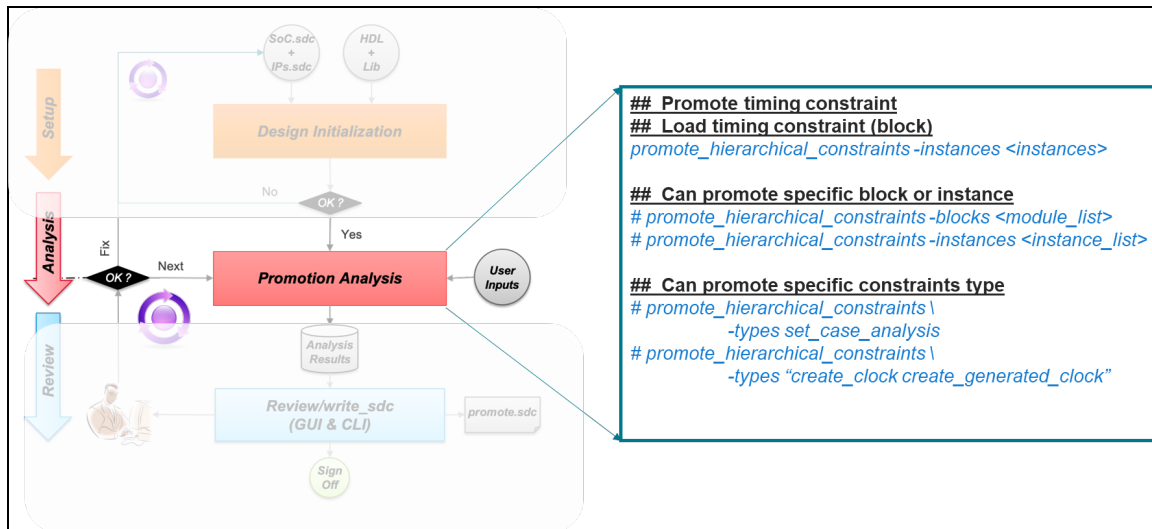
Ignore min/max delay constraints specifically on library pins:

```
set_constraint_promotion_strategy -method block_ignore \  
-type "min_delay max_delay" -object_type lib_pin
```

These examples illustrate how you can customize promotion decisions for fine-grained control over constraint classes.

Running Constraint Promotion

Once the design is initialized and promotion configuration parameters are defined, the actual Constraint Promotion step is triggered using the `promote_hierarchical_constraints` command. This step performs the core Promotion Analysis, applying user-defined priority ordering, conflict resolution strategies, and evaluating registered block-level SDCs. The following illustration shows where this operation takes place in the Constraint Promotion Flow:



Constraint Promotion Command

The `promote_hierarchical_constraints` command supports flexible invocation modes: global promotion, per-block promotion, or per-constraint-type promotion. This makes it easy to use for both full-chip hierarchical signoff and targeted incremental debugging scenarios.

Command Usage

```
promote_hierarchical_constraints [-help] \  
[-blocks <string>] \  
[-instances <string>] \  
[-types <string>] \  
[> <file>] [>> <file>]
```

Options

-help	Prints command usage.
-blocks <string>	Promotes constraints for specific module(s).
-instances <string>	Promotes constraints only for listed instances.
-types <string>	Promotes only selected constraint classes.
> <file>	Writes the promoted SDC to a specific output file.
>> <file>	Appends to an existing file.

Examples

Promote only selected instances:

```
promote_hierarchical_constraints -instances {U1 U2}
```

Promote only clock creation constraints:

```
promote_hierarchical_constraints -types "create_clock create_generated_clock"
```

Promote only case analysis settings:

```
promote_hierarchical_constraints -types set_case_analysis
```

Write output directly to a file:

```
promote_hierarchical_constraints > promote.sdc
```

Root Attributes Controlling Promotion

Root attributes can be used to refine how specific classes of constraints are interpreted during constraint propagation. These attributes are useful when block-level constraints include internal IO delays, loads, or clock structures that may require special handling.

Available Attributes

Attribute	Default	Description
<code>timing_promote_boundary_path_exceptions</code>	true	Enables promotion of exceptions involving block boundary ports.
<code>timing_promote_in_out_delay_over_combinational_cone</code>	true	Allows input/output delays to propagate through combinational logic when promoting constraints.
<code>timing_promote_internal_set_input_delay</code>	false	Controls promotion of <code>set_input_delay</code> inside blocks.
<code>timing_promote_internal_set_output_delay</code>	false	Controls promotion of <code>set_output_delay</code> inside blocks.
<code>timing_promote_set_load_over_combinational_cone</code>	false	Enables promotion of load constraints across combinational logic.
<code>timing_disable_promotion_clock_prefix</code>	false	Removes instance name prefixes from promoted clock names if no conflict exists.
<code>timing_promote_clock_to_top_port</code>	false	Allows block-level clocks to be promoted directly to top-level ports.
<code>timing_disable_promotion_clock_group_prefix</code>	false	Disables the automatic addition of instance name prefixes to clock group names after promotion. Example: Input Block SDC: <code>set_clock_groups -name blk_clk_grp -physically_exclusive -group {CLK} -group {CLK2}</code> Default: <code>set_db timing_disable_promotion_clock_group_prefix false</code> Expected Promoted Constraint: <code>set_clock_groups -name blk/blk_clk_grp -physically_exclusive -group [get_clocks {CLK}] -group [get_clocks {CLK2}]</code> With <code>set_db timing_disable_promotion_clock_group_prefix true</code> Expected Promoted Constraint: <code>set_clock_groups -name blk_clk_grp -physically_exclusive -group [get_clocks {CLK}] -group [get_clocks {CLK2}]</code>

Typical Use Cases

- Enabling `timing_promote_boundary_path_exceptions` ensures correct propagation of false paths or multicycle paths defined on block IO.
- Disabling prefix removal (`timing_disable_promotion_clock_prefix`) preserves hierarchical uniqueness in multi-IP environments.
- Enabling `timing_promote_clock_to_top_port` can simplify SoC-level clock modeling when block clocks directly drive SoC pins.

These options provide low-level control to enable working with complex or non-uniform block constraint topologies.

Constraint Promotion Results

This stage generates a comprehensive set of diagnostics that describe how each block-level constraint was interpreted, matched, promoted, or rejected during Promotion Analysis. This enables you to quickly identify mismatches, conflicts, or structural issues. Litmus assigns every examined constraint a **CP-tag**, which encodes both the category of the result and additional property-specific differences where applicable. These tags appear in the promotion report as structured messages.

Interpretation of CP-Tags

During promotion, Litmus evaluates block constraints against the SoC context. For each constraint, one of the following outcomes is recorded:

CP-Code	Meaning
CP-<base#> + 0	A match exists, but with <i>property differences</i> (e.g., waveform/period mismatch).
CP-<base#> + 1	An equivalent constraint already exists at the top level; block constraint is redundant.
CP-<base#> + 2	Constraint is <i>successfully promoted</i> to the SoC level (missing at top).
CP-<base#> + 3	Multiple matching candidates found at the top level; ambiguous promotion.
CP-<base#> + 4	Promotion blocked due to a <i>conflict</i> with an existing top-level constraint.
CP-<base#> + 5	Promotion rejected because it impacts <i>objects outside the originating block</i> .
CP-<base#> + 6	Constraint is <i>skipped</i> (various reasons depending on constraint type and strategy).

Note:

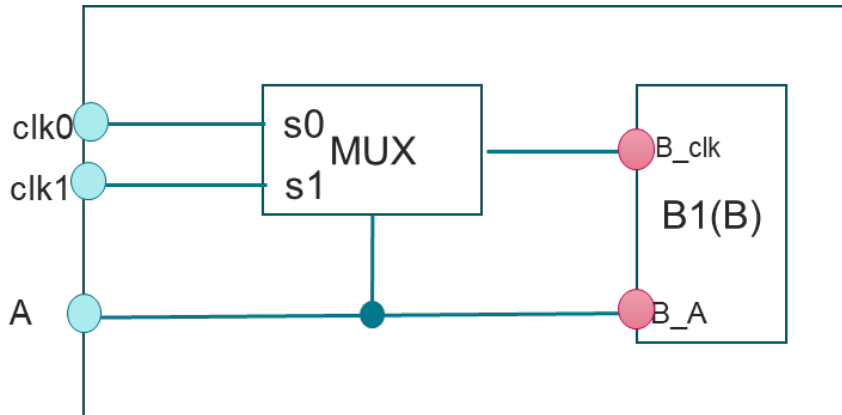
- CP-3, CP-4, and CP-5 constraints are preserved *in their original hierarchical context* and written into reports unless overridden by `set_constraint_promotion_strategy`.
- Each constraint category (clocks, case analyses, IO delays, exceptions) receives its own CP-base code.
- A CP-x0 message may accompany any category when property-level differences (For example, clock waveform or period) are detected.

Detailed CP-Tag Scenarios

The following examples illustrate how Litmus applies CP-tags using simple structural SDC scenarios.

CP +1: Equivalent Constraint Exists at the Top

This scenario occurs when the block defines a constraint that matches an existing top-level constraint.



Top-level SDC:

```
# top.sdc
create_clock -name clk_0 ..[get_ports clk0]
create_clock -name clk_1 ..[get_ports clk1]
set_case_analysis 0 [get_ports A]
```

Block SDC:

```
# B.sdc
create_clock -name foo ..[get_ports B_clk]
set_case_analysis 0 [get_ports B_A]
```

Final Promoted SDC:

```
# top.sdc + promoted sdc (final)
create_clock -name clk_0 ..[get_ports clk0]
create_clock -name clk_1 ..[get_ports clk1]
set_case_analysis 0 [get_ports A]
```

Explanation:

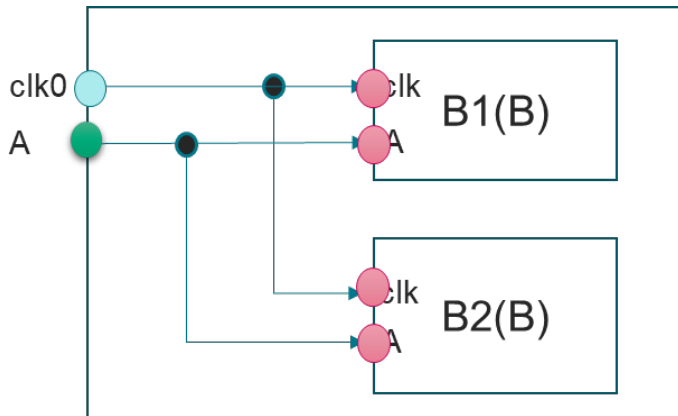
The block's case analysis matches the top-level intent. Promotion is not necessary.

Tag:

CP-xxx +1

CP +2: Constraint Promoted (Missing at Top)

A block constraint is not present at the top and is promoted.



Top-level SDC:

```
# top.sdc
create_clock -name clk_0 ..[get_ports clk0]
```

Block SDC:

```
# B.sdc
create_clock -name foo ..[get_ports clk]
set_case_analysis 0 [get_ports A]
```

Final Promoted SDC:

```
# top.sdc + promoted sdc (final)
create_clock -name clk_0 ..[get_ports clk0]
set_case_analysis 0 [get_ports A]
```

Explanation:

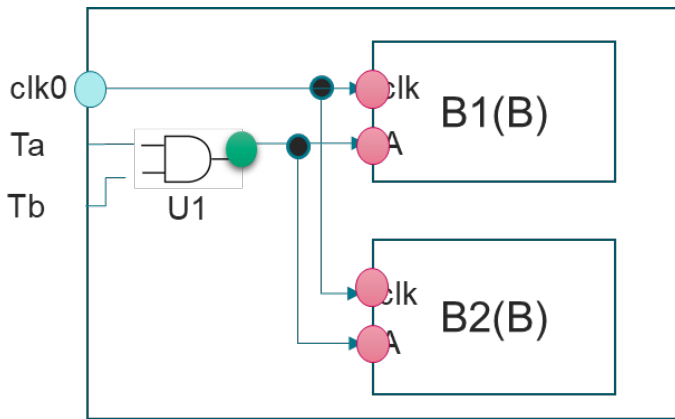
The block's `set_case_analysis` constraint was missing at the top and is promoted.

Tag:

CP-xxx +2

CP +3: Multiple Top-Level Candidates

Promotion is ambiguous because the block's source can map to multiple top-level ports or signals.



Top-level SDC:

```
# top.sdc
create_clock -name clk_0 ..[get_ports clk0]
```

Block SDC:

```
# B.sdc
create_clock -name foo ..[get_ports clk]
set_case_analysis 0 [get_ports A]
```

Final Promoted SDC:

```
# top.sdc + promoted sdc (final)
create_clock -name clk_0 ..[get_ports clk0]
set_case_analysis 0 [get_ports U1/out]
```

Explanation:

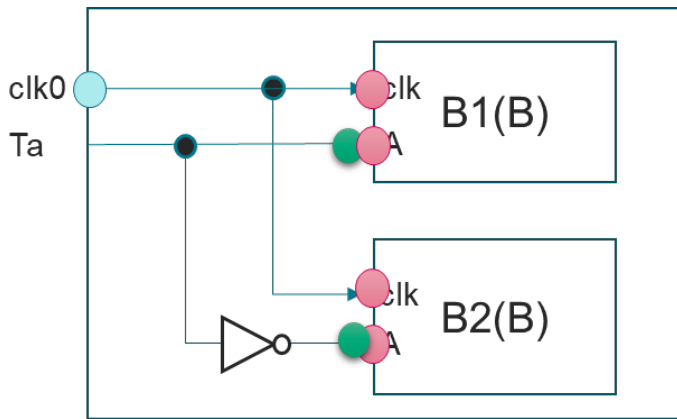
The block port `A` maps to multiple SoC-level nodes (two or more branches), and Litmus flags the ambiguity.

Tag:

CP-xxx +3

CP +4: Conflict With Top-Level Constraint

Promotion is blocked because the block constraint contradicts an existing top-level constraint.



Top-level SDC:

```
# top.sdc
create_clock -name clk_0 ..[get_ports clk0]
```

Block SDC:

```
# B.sdc
create_clock -name foo ..[get_ports clk]
set_case_analysis 0 [get_ports A]
```

Final Promoted SDC:

```
# top.sdc + promoted sdc (final)
create_clock -name clk_0 ..[get_ports clk0]
set_case_analysis 0 [get_ports B1/A]
set_case_analysis 0 [get_ports B2/A]
```

Explanation:

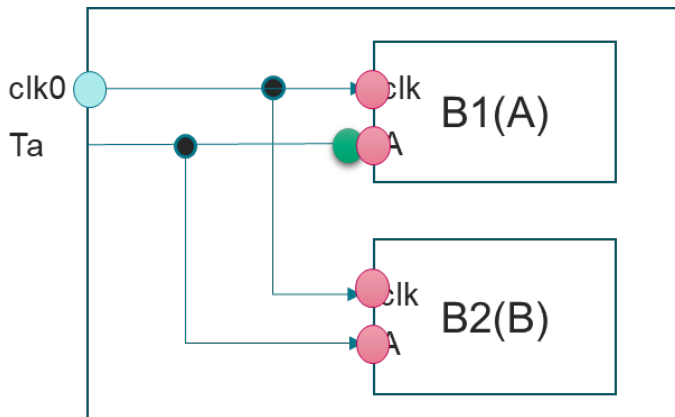
Promoting the block's case analysis creates a conflict across two block instances. The constraint remains written in the context of the respective instances.

Tag:

CP-xxx +4

CP +5: Impact on External Objects

Promotion is rejected because applying the constraint at the SoC level incorrectly affects objects or paths outside the originating block.



Top-level SDC:

```
# top.sdc
create_clock -name clk_0 ..[get_ports clk0]
```

Block SDC (A.sdc or B.sdc):

```
# A.sdc
create_clock -name foo ..[get_ports clk]
set_case_analysis 0 [get_ports A]
# B.sdc
create_clock -name foo ..[get_ports clk]
```

Final Promoted SDC:

```
# top.sdc + promoted sdc (final)
create_clock -name clk_0 ..[get_ports clk0]
set_case_analysis 0 [get_ports B1/A]
```

Explanation:

Promoting the constraint alters timing for paths of other block instances. Thus, it is restricted to the specific instance where context is valid.

Tag:

```
CP-xxx +5
```

Sample Run Scripts

This section provides reference script templates for running Constraint Promotion in both **Netlist-based** and **RTL-based** flows. These scripts reflect the recommended sequence of operations and illustrate how design setup, constraint registration, promotion analysis, and review align in a unified methodology.

1. **Netlist Flow Script:** The Netlist flow is typically used when a synthesized or post-layout gate-level is available.
 - a. `init_design` ensures that all library, connectivity, and SDC contexts are resolved.
 - b. Running `report_sdc` and `report_rule_check` *before* promotion eliminates structural or linting issues early.
 - c. Promotion should start only after all block constraints are registered.
2. **RTL Flow Script:** The RTL flow mirrors the Netlist flow but includes RTL elaboration and optional library loading. This flow is commonly used early in the project lifecycle when block SDCs and top-level constraints are still evolving.
 - a. The `input_libs` step is optional and depends on the level of timing modeling required.
 - b. All RTL must be elaborated before constraint lint checks are executed.
 - c. The rest of the flow is consistent with the Netlist flow, ensuring uniformity across design representations.

The figure below summarizes the flow for both design views:

	## Netlist	## RTL
Design & Constraint Setup	## Load MMMC setup read_mmmc <file>	## Setup timing library (optional) input_libs <files>
	## Load design (Netlist) read_netlist -top <module> <files>	## Setup timing constraint (top) input_sdc <files>
Design & Constraint Initialization	## Initialize design database ## Load timing constraint (top) init_design	## Load MMMC setup litmus_setup_mmmc
	## Review constraint lint checks report_sdc -status fail -verbose report_rule_check -category constraint_lint -status fail	## Load design (RTL) elaborate_design -script <script_file> ## <script_file> : read_hdl <files> + elaborate <module>
Constraint Setup	## Setup timing constraint (block) read_hierarchical_constraints -instance <instance> -block_sdc <files>	## Initialize design database ## Load timing constraint (top) init_design
Constraint Analysis (Promote)	## Promote timing constraint ## Load timing constraint (block) promote_hierarchical_constraints -instances <instances>	## Review constraint lint checks report_sdc -status fail -verbose report_rule_check -category constraint_lint -status fail
Constraint Review (Report & Diagnosis)	## Review constraint promotion results write_sdc -view <top_view> <file>	## Setup timing constraint (block) read_hierarchical_constraints -instance <instance> -block_sdc <files>
		## Promote timing constraint ## Load timing constraint (block) promote_hierarchical_constraints -instances <instances>
		## Review constraint promotion results write_sdc -view <top_view> <file>



To ensure stable and predictable promotion behavior:

- Always validate SDC and lint issues before running promotion.
- Use consistent MMMC environments across flows for alignment with Synthesis and Implementation tools.
- Register block SDCs only after the top-level SDC context is fully loaded.
- Favor instance-based registration when blocks differ across instantiations.
- Always write the final SDC using `write_sdc` to maintain a clean record of promoted constraints.

Working with the GUI

This chapter provides detailed information about the **components, layout, and workflows** of the Litmus GUI. The Litmus GUI is launched using the `gui_show` command from the Litmus shell. Once launched, the GUI displays the **Rules Summary**, which serves as the primary workspace for reviewing and analyzing rule evaluation results. This chapter describes:

- The structure of the Litmus GUI
- The Menu Bar and Rules Summary components
- Cross-probing and filtering workflows

The screenshot shows the Litmus Signoff Solution(TM) 26.1 GUI. The main window is titled "Rules Summary" and contains a table of constraint results. The table has columns for Name, Severity, Count, Status, occurrence_id, scope, witness, clocks, count, clock_pin_witnesses, clock_pin_witnesses_filters, type, shared_driver, and blocks. The table lists various constraints, including CONST_SCA_CHK1, CONST_CLK_DEF1, and CONST_CLK_DEF2, with their respective severity, count, and status. The table also includes a detailed view of the CONST_CLK_DEF1 constraint, showing its occurrence_id, scope, witness, clocks, count, clock_pin_witnesses, clock_pin_witnesses_filters, type, shared_driver, and blocks.

Name	Severity	Count	Status
CONSTRAINT_LIST			
CONSTRAINT_SIGNOFF			
CONST_SCA_CHK1	Fail	1	Fail
CONST_CLK_DEF1	Fail	153	Fail
CONST_CLK_DEF2	Fail	2	Fail
CONST_CLK_DEF13	Fail	1	Fail
CONST_CLK_DEF32	Fail	1	Fail
CONST_CLK_DEF34	Fail	1	Fail
CONST_CLK_DEF36	Fail	2	Fail
CONST_CLK_DEF38	Fail	1	Fail
CONST_CLK_DEF48	Fail	2	Fail
CONST_CLK_DEF40	Fail	1	Fail
CONST_CLK_DEF46	Fail	2	Fail
CONST_CLK_GRP1	Fail	2	Fail
CONST_CLK_GRP2	Fail	1	Fail
CONST_CLK_GRP4	Fail	3	Fail
CONST_CLK_GRP7	Fail	3	Fail
CONST_CLK_LAT4	Fail	4	Fail
CONST_CLK_LAT12	Fail	3	Fail
CONST_CLK_UNC9	Fail	2	Fail
CONST_DGN_CMB1	Fail	3	Fail
CONST_EXC_FLP1	Fail	4	Fail
CONST_EXC_MCP1	Fail	6	Fail
CONST_EXC_OLP2	Fail	2	Fail
CONST_EXC_OLP1a	Fail	2	Fail
CONST_IO_IDL1	Fail	1	Fail
CONST_IO_IDL13	Fail	1	Fail
CONST_IO_IDL14	Fail	4	Fail
CONST_IO_ODL13	Fail	1	Fail
CONST_IO_ODL14	Fail	1	Fail
CONST_IO_ODL15	Fail	10	Fail
CONST_IO_ODL1	Fail	1	Fail
CONST_HFN_CHK1b	Fail	3	Fail
CONST_HFN_CHK2	Fail	1	Fail
CONSTRAINT_HIER_SIGNOFF			
CONSTRAINT_COMP_SIGNOFF			
CONSTRAINT_COVERAGE_SIGNOFF			
CONSTRAINT_ENHANCED_SIGNOFF			

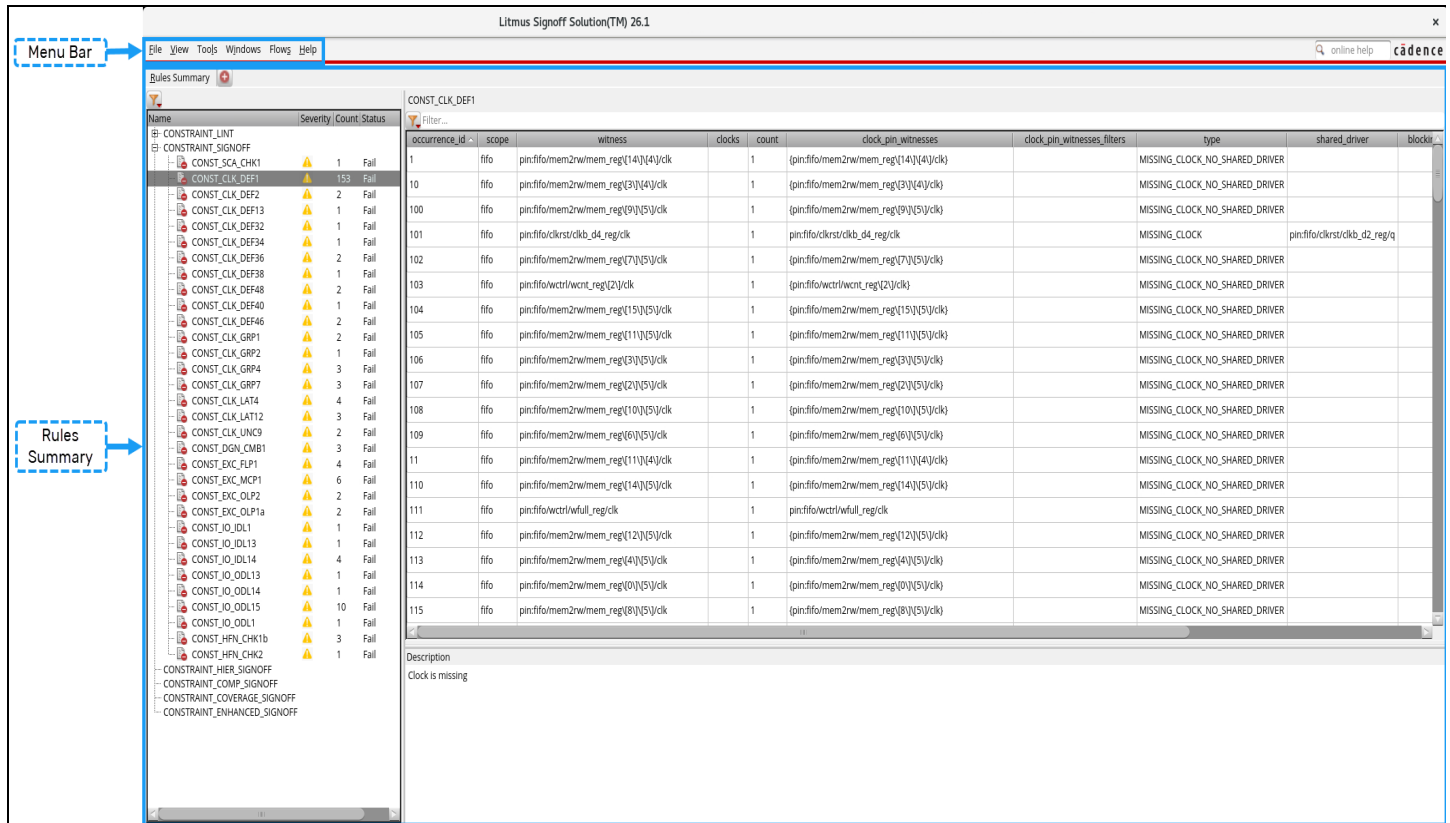
occurrence_id	scope	witness	clocks	count	clock_pin_witnesses	clock_pin_witnesses_filters	type	shared_driver	blocks
1	fifo	pin:ffifo/mem2rw/mem_reg(14)(4)/clk		1	{pin:ffifo/mem2rw/mem_reg(14)(4)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
10	fifo	pin:ffifo/mem2rw/mem_reg(3)(4)/clk		1	{pin:ffifo/mem2rw/mem_reg(3)(4)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
100	fifo	pin:ffifo/mem2rw/mem_reg(9)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(9)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
101	fifo	pin:ffifo/clkrst/clkb_d4_reg/clk		1	pin:ffifo/clkrst/clkb_d4_reg/clk		MISSING_CLOCK	pin:ffifo/clkrst/clkb_d2_reg/q	
102	fifo	pin:ffifo/mem2rw/mem_reg(7)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(7)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
103	fifo	pin:ffifo/wctrl/wcnt_reg(2)(5)/clk		1	{pin:ffifo/wctrl/wcnt_reg(2)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
104	fifo	pin:ffifo/mem2rw/mem_reg(15)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(15)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
105	fifo	pin:ffifo/mem2rw/mem_reg(11)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(11)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
106	fifo	pin:ffifo/mem2rw/mem_reg(3)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(3)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
107	fifo	pin:ffifo/mem2rw/mem_reg(2)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(2)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
108	fifo	pin:ffifo/mem2rw/mem_reg(10)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(10)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
109	fifo	pin:ffifo/mem2rw/mem_reg(6)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(6)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
11	fifo	pin:ffifo/mem2rw/mem_reg(11)(4)/clk		1	{pin:ffifo/mem2rw/mem_reg(11)(4)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
110	fifo	pin:ffifo/mem2rw/mem_reg(14)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(14)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
111	fifo	pin:ffifo/wctrl/wfull_reg/clk		1	pin:ffifo/wctrl/wfull_reg/clk		MISSING_CLOCK_NO_SHARED_DRIVER		
112	fifo	pin:ffifo/mem2rw/mem_reg(12)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(12)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
113	fifo	pin:ffifo/mem2rw/mem_reg(4)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(4)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
114	fifo	pin:ffifo/mem2rw/mem_reg(0)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(0)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		
115	fifo	pin:ffifo/mem2rw/mem_reg(8)(5)/clk		1	{pin:ffifo/mem2rw/mem_reg(8)(5)/clk}		MISSING_CLOCK_NO_SHARED_DRIVER		

Description
Clock is missing

For information on launching the GUI and managing its lifecycle, refer to [Launching the Litmus GUI](#).

GUI Components

This section describes the structure of the Litmus GUI and the function of each area. At a high level, the Litmus GUI consists of a Menu Bar for global application actions and a Rules Summary workspace for reviewing and analyzing rule evaluation results.

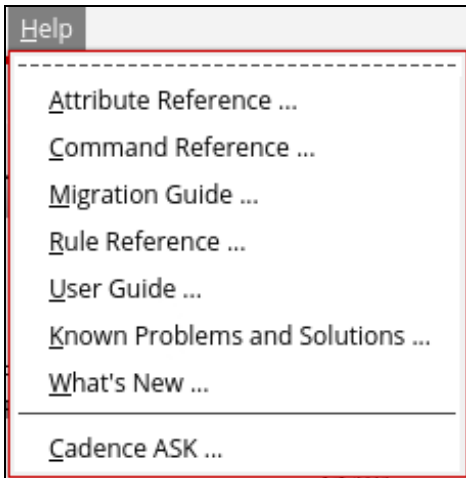


Menu Bar

The **Menu Bar** provides access to global, application-level functionality in the Litmus GUI. It allows you to perform common operations related to design loading, visualization, navigation, debugging, workspace configuration, and access to documentation. Actions available through the Menu Bar apply across the GUI session and are not limited to a specific rule or result selection.

File View Tools Windows Flows Help

Use the *Help* menu to access documentation and reference material, including user guides, rule references, command reference information, attribute references, migration guides, known issues, and What's New updates.



Rules Summary

The **Rules Summary** is the primary workspace for reviewing rule evaluation results. It consolidates rule navigation, detailed violation data, and contextual rule information to support efficient analysis and debug. It displays information about rule check results in the design.

The screenshot shows the 'Rules Summary' window. On the left is the 'Navigation Pane' containing a tree view of rules. The main area displays a table of rule occurrences. On the right, callouts point to the 'Occurrences Table' and the 'Description Tab' at the bottom.

occurrence_id	scope	witness	clocks	count	clock_pin_witnesses	clock_pin_witnesses_filters	type	shared_driver	blocker
1	fifo	pinfifo/mem2w/mem_reg(14)(4)clk		1	(pinfifo/mem2w/mem_reg(14)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
2	fifo	pinfifo/wctrl/sync/sdata_q1_reg(4)(4)clk		1	(pinfifo/wctrl/sync/sdata_q1_reg(4)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
3	fifo	pinfifo/mem2w/mem_reg(6)(4)clk		1	(pinfifo/mem2w/mem_reg(6)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
4	fifo	pinfifo/mem2w/mem_reg(2)(4)clk		1	(pinfifo/mem2w/mem_reg(2)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
5	fifo	pinfifo/mem2w/mem_reg(10)(4)clk		1	(pinfifo/mem2w/mem_reg(10)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
6	fifo	pinfifo/mem2w/mem_reg(8)(4)clk		1	(pinfifo/mem2w/mem_reg(8)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
7	fifo	pinfifo/mem2w/mem_reg(0)(4)clk		1	(pinfifo/mem2w/mem_reg(0)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
8	fifo	pinfifo/mem2w/mem_reg(12)(4)clk		1	(pinfifo/mem2w/mem_reg(12)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
9	fifo	pinfifo/mem2w/mem_reg(4)(4)clk		1	(pinfifo/mem2w/mem_reg(4)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
10	fifo	pinfifo/mem2w/mem_reg(3)(4)clk		1	(pinfifo/mem2w/mem_reg(3)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
11	fifo	pinfifo/mem2w/mem_reg(11)(4)clk		1	(pinfifo/mem2w/mem_reg(11)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
12	fifo	pinfifo/mem2w/mem_reg(7)(4)clk		1	(pinfifo/mem2w/mem_reg(7)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
13	fifo	pinfifo/mem2w/mem_reg(15)(4)clk		1	(pinfifo/mem2w/mem_reg(15)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
14	fifo	pinfifo/wctrl/wctrl_reg(1)(4)clk		1	(pinfifo/wctrl/wctrl_reg(1)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
15	fifo	pinfifo/wctrl/sync/sdata_q0_reg(1)(4)clk		1	(pinfifo/wctrl/sync/sdata_q0_reg(1)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
16	fifo	pinfifo/mem2w/mem_reg(13)(4)clk		1	(pinfifo/mem2w/mem_reg(13)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
17	fifo	pinfifo/mem2w/mem_reg(5)(4)clk		1	(pinfifo/mem2w/mem_reg(5)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
18	fifo	pinfifo/mem2w/mem_reg(1)(4)clk		1	(pinfifo/mem2w/mem_reg(1)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		
19	fifo	pinfifo/mem2w/mem_reg(9)(4)clk		1	(pinfifo/mem2w/mem_reg(9)(4)clk)		MISSING_CLOCK_NO_SHARED_DRIVER		

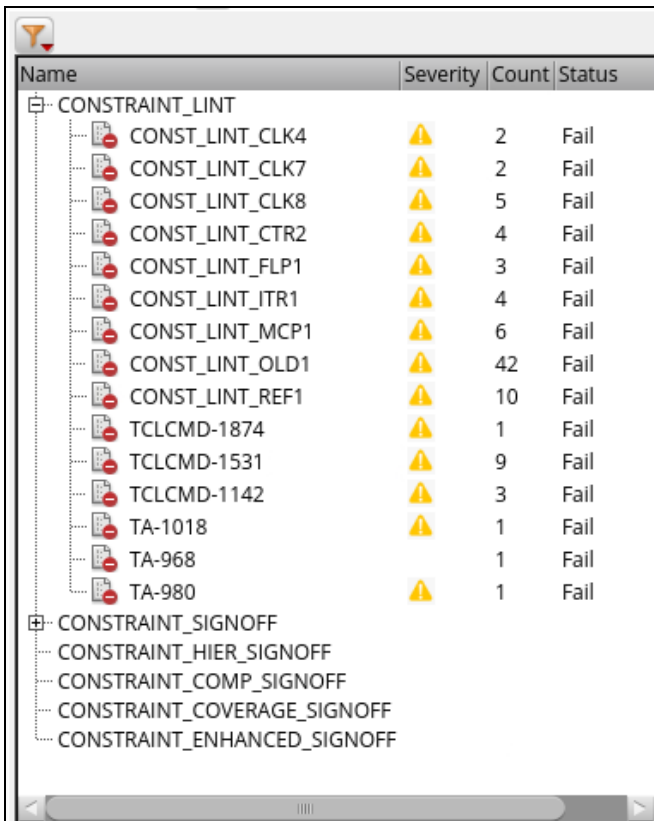
Description
Clock is missing

The Rules Summary is organized into the following sections:

- **Navigation Pane:** Provides a hierarchical view of rule categories and individual rules.
- **Occurrences Table:** Displays the attribute information of each occurrence of the rule selected in the navigation pane.
- **Description Tab:** Displays contextual details for the rule selected in the navigation pane.

Navigation Pane

The **Navigation Pane** provides a hierarchical view of rule categories and individual rules evaluated during the Litmus run. Rules are grouped under their respective categories (for example, *CONSTRAINT_LINT*, *CONSTRAINT_SIGNOFF*, and related signoff groups), allowing you to expand or collapse categories to navigate efficiently across large rule sets.



The screenshot shows the Navigation Pane with a tree view. The tree is expanded to show the **CONSTRAINT_LINT** category, which contains several sub-rules. Each rule entry in the table includes a red icon with a yellow warning triangle, a name, a severity indicator (yellow triangle), a count, and a status (Fail). The **CONSTRAINT_SIGNOFF** category is also visible but collapsed.

Name	Severity	Count	Status
CONSTRAINT_LINT			
CONST_LINT_CLK4	⚠	2	Fail
CONST_LINT_CLK7	⚠	2	Fail
CONST_LINT_CLK8	⚠	5	Fail
CONST_LINT_CTR2	⚠	4	Fail
CONST_LINT_FLP1	⚠	3	Fail
CONST_LINT_ITR1	⚠	4	Fail
CONST_LINT_MCP1	⚠	6	Fail
CONST_LINT_OLD1	⚠	42	Fail
CONST_LINT_REF1	⚠	10	Fail
TCLCMD-1874	⚠	1	Fail
TCLCMD-1531	⚠	9	Fail
TCLCMD-1142	⚠	3	Fail
TA-1018	⚠	1	Fail
TA-968	⚠	1	Fail
TA-980	⚠	1	Fail
CONSTRAINT_SIGNOFF			
CONSTRAINT_HIER_SIGNOFF			
CONSTRAINT_COMP_SIGNOFF			
CONSTRAINT_COVERAGE_SIGNOFF			
CONSTRAINT_ENHANCED_SIGNOFF			

Each rule entry displays key information, including:

- **Rule name**
- **Severity indicator**
- **Occurrence count**
- **Rule status**

Severity is indicated using icons:

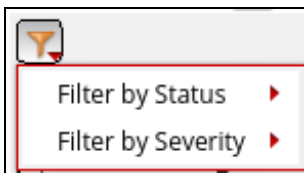
- A **yellow icon** indicates a *Warning*.
- A **red icon** indicates an *Error*.
- Rules with *Info* severity are displayed without an icon.

Selecting a rule in the Navigation Pane drives the content displayed in the **Occurrences Table** and the **Description Tab**, enabling detailed inspection of rule occurrences and rule intent.

✓ Tip:
Selecting a rule in the Navigation Pane and pressing **F1** opens the Cadence Doc Assistant (CDA) with the corresponding **Rule Reference** for the selected rule.

Filtering Rules in the Navigation Pane

The Navigation Pane includes a **Filter icon** at the top, which allows you to refine the displayed rule list based on **status** and **severity**. This is especially useful when working with large designs that contain many evaluated rules. The filter provides the following options:



- *Filter by Status*
 - *Show Fail* – Displays only rules with failed status
 - *Show Pass* – Displays only rules that passed
 - *Show Not-Run* – Displays rules that were not executed
 - *Show All* – Displays all rules regardless of their status
- *Filter by Severity*
 - *Show Warning* – Displays rules with warning severity
 - *Show Error* – Displays rules with error severity
 - *Show Info* – Displays rules with information severity
 - *Show All* – Displays rules of all severities

These filters can be applied independently to quickly focus on rules of interest during analysis and debug.

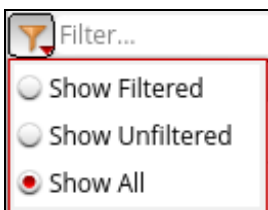
Occurrences Table

The **Occurrences Table** displays occurrence-level information for the rule selected in the Navigation Pane. Each row in the table represents a distinct occurrence of the selected rule. The set of columns displayed in the Occurrences Table is **rule-dependent**. When a rule is selected, only the attributes relevant to that rule are shown as table columns. For example, a rule such as `CONST_CLK_LAT4` may display columns as shown in the figure below. The *Occurrences Table* allows you to review, filter, and analyze multiple occurrences of the same rule independently.

CONST_CLK_LAT4						
Filter...						
occurrence_id ^	scope	first_clock	second_clock	max_min_option	early_late_option	rise_fall_option
1	fifo	clock:fifo/top_view_max/clka	clock:fifo/top_view_max/test_clk	max	early	rise
2	fifo	clock:fifo/top_view_max/clkb	clock:fifo/top_view_max/test_clk	max	early	rise
3	fifo	clock:fifo/top_view_max/test_clk	clock:fifo/top_view_max/rclk	max	early	rise
4	fifo	clock:fifo/top_view_max/test_clk	clock:fifo/top_view_max/wclk	max	early	rise

Filtering in the Occurrences Table

There are filtering controls at the top of the *Occurrences Table* to help you refine the display, which is especially useful when a rule reports a large number of occurrences.



- The **Occurrence-based** filters control how occurrences affected by rule filters are displayed. The following options are available:
 - *Show Filtered* – Displays only occurrences that have been filtered using rule filters.
 - *Show Unfiltered* – Displays only occurrences that are not affected by rule filters.
 - *Show All* – Displays both filtered and unfiltered occurrences together.

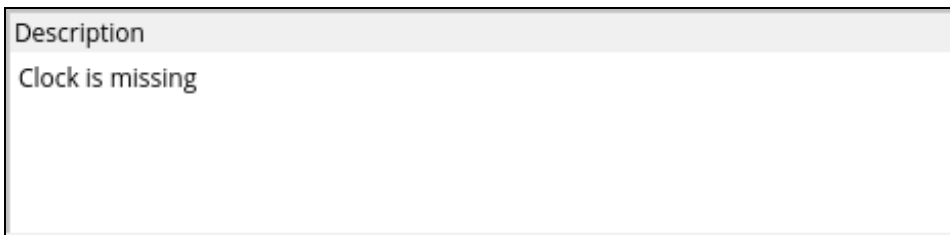
 **Note:**
The occurrence-based filtering options reflect the filters defined using the [Create Rule Filters](#) feature.

- The **Name-based** filtering is enabled through the **text input field** next to the Filter icon. You can enter a search string in this field to dynamically filter the table rows based on matching attribute values. Only occurrences whose attribute values contain the specified search string are displayed.

These filtering mechanisms allow you to quickly narrow down the list of occurrences and focus on specific objects, attributes, or filtered results during analysis.

Description Tab

The **Description Tab** displays information and contextual details for the selected rule. This includes the rule header and explanatory text describing what the rule checks and the condition being reported. This helps you understand the intent of the rule and interpret the occurrence data shown in the Rules Summary workspace. For example, the following is displayed for the rule `CONST_CLK_DEF1`:



GUI Features

This section describes key GUI features that support detailed analysis and debug of rule violations. These features enable you to navigate from reported rule occurrences to corresponding design representations and to manage large sets of rule occurrences efficiently.

- [Cross-Probing](#)
- [Creating Rule Filters](#)

Cross-Probing

Cross-probing enables you to navigate directly from a reported rule occurrence in the **Occurrences Table** to the corresponding design representation. This capability helps correlate rule violations with RTL, schematic, or constraint definitions and supports efficient root-cause analysis.

Cross-probing actions are available through the **right-click context menu** on a selected rule occurrence row. Depending on the type of information associated with the selected occurrence, different cross-probing options are presented.

- [Cross-Probing to HDL / RTL](#)
- [Cross-Probing to Schematic](#)
 - [Open Schematic View \(Cone\)](#)
 - [Open Schematic View \(Module\)](#)
- [Cross-Probing to SDC](#)

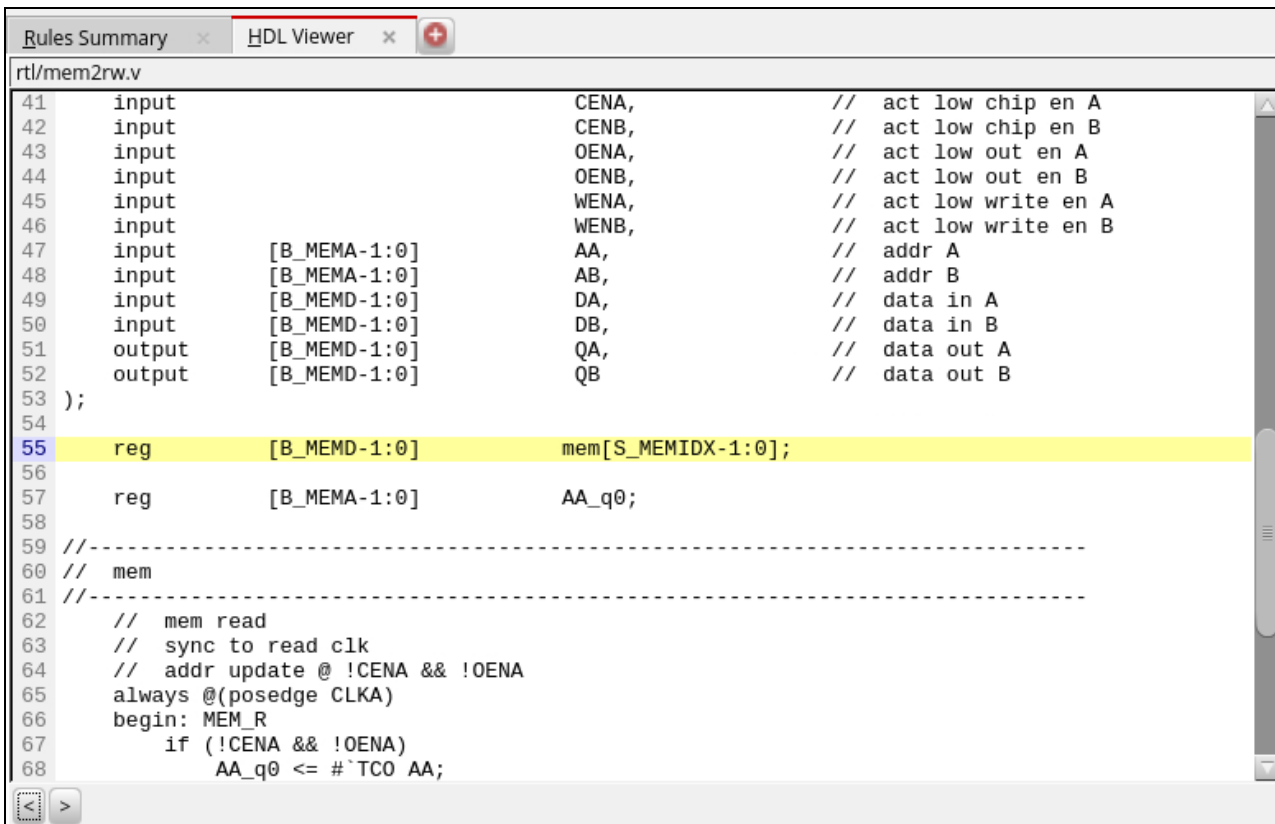
Cross-Probing to HDL / RTL

For rule occurrences associated with RTL objects (for example, pins), you can cross-probe to the HDL/RTL source code using the *Open HDL Viewer* option from the right-click context menu.

When *Open HDL Viewer* is selected, the *HDL viewer* opens and navigates to the exact source-code location corresponding to the selected object. The relevant object is highlighted in the HDL/RTL code, enabling you to quickly identify the implementation context related to the reported rule violation.

This form of cross-probing allows direct correlation between a rule occurrence and the corresponding RTL construct without manual source navigation.

pin:fifo/mem2rw/mem_reg[6]\[4]	Create Rule Filter	{pin:fifo/mem2rw/mem_reg[6]\[4]
pin:fifo/mem2rw/mem_reg[6]\[4]/clk	Open HDL Viewer ▶	pin:fifo/mem2rw/mem_reg[6]\[4]/clk
pin:fifo/mem2rw/mem_reg[10]\[4]	Open Schematic View (Cone) ▶	{pin:fifo/mem2rw/mem_reg[10]\[4]
pin:fifo/mem2rw/mem_reg[8]\[4]	Open Schematic View (Module) ▶	{pin:fifo/mem2rw/mem_reg[8]\[4]



```
rtl/mem2rw.v
41  input          CENA,          // act low chip en A
42  input          CENB,          // act low chip en B
43  input          OENA,          // act low out en A
44  input          OENB,          // act low out en B
45  input          WENA,          // act low write en A
46  input          WENB,          // act low write en B
47  input          [B_MEMA-1:0]  AA,          // addr A
48  input          [B_MEMA-1:0]  AB,          // addr B
49  input          [B_MEMD-1:0]  DA,          // data in A
50  input          [B_MEMD-1:0]  DB,          // data in B
51  output         [B_MEMD-1:0]  QA,          // data out A
52  output         [B_MEMD-1:0]  QB,          // data out B
53 );
54
55  reg             [B_MEMD-1:0]  mem[S_MEMIDX-1:0];
56
57  reg             [B_MEMA-1:0]  AA_q0;
58
59  //-----
60  // mem
61  //-----
62  // mem read
63  // sync to read clk
64  // addr update @ !CENA && !OENA
65  always @(posedge CLKA)
66  begin: MEM_R
67      if (!CENA && !OENA)
68          AA_q0 <= #`TCO AA;
```

 **Note:**
Cross-probing to HDL/RTL is not supported for gate-level netlists.

Cross-Probing to Schematic

The Litmus GUI supports schematic cross-probing through two distinct options available in the right-click context menu of a rule occurrence:

- [Open Schematic View \(Cone\)](#)
- [Open Schematic View \(Module\)](#)

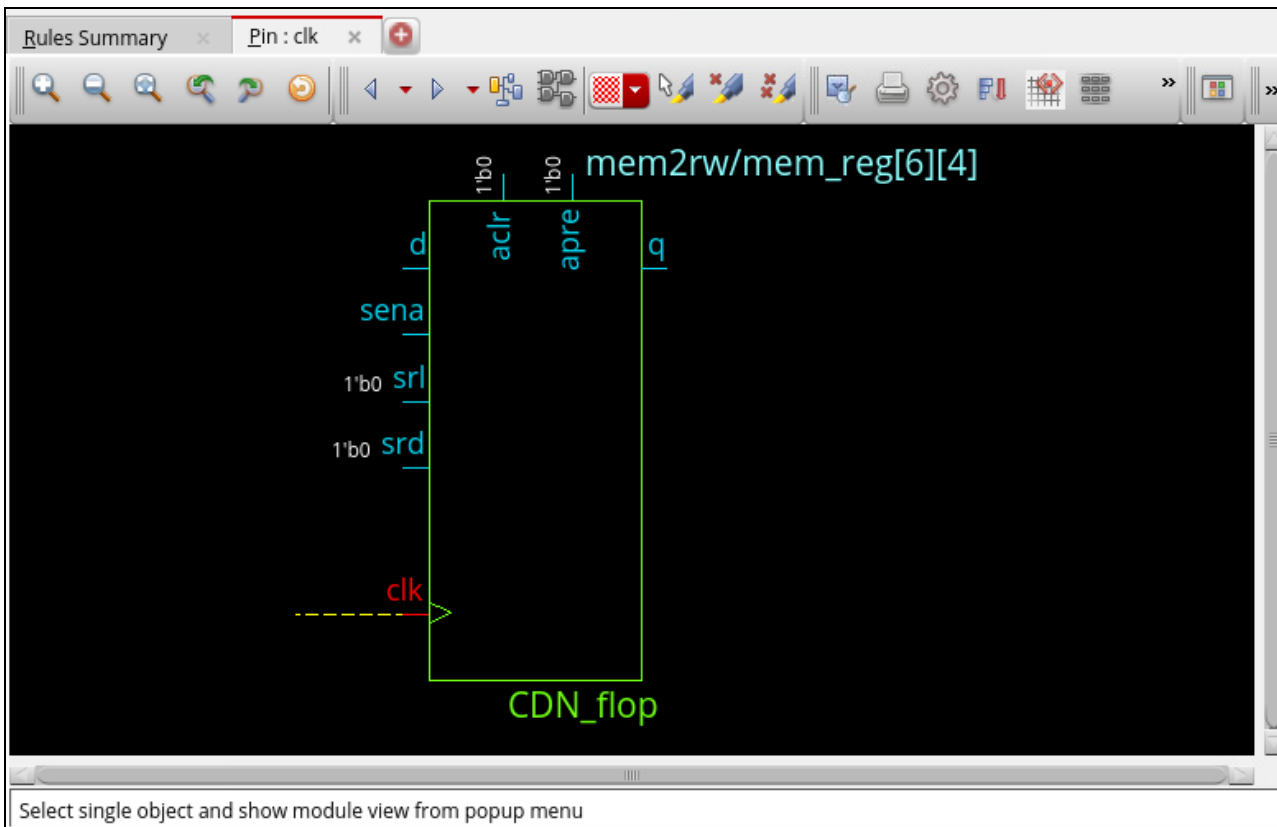
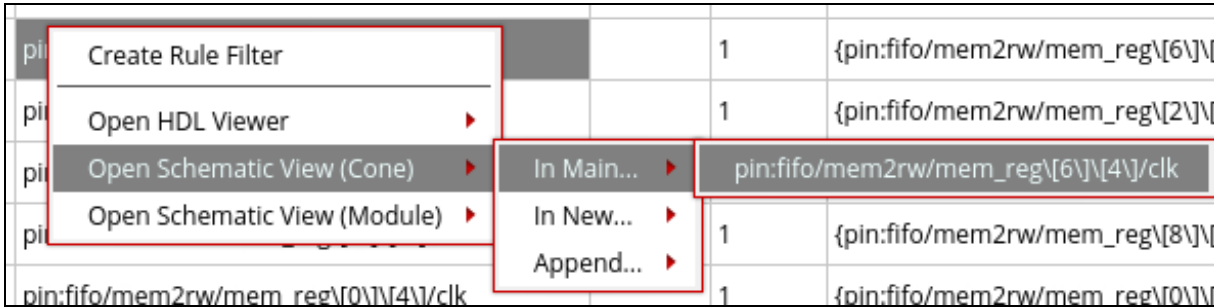
For both Cone and Module views, standard zoom and navigation controls can be used to inspect the schematic in detail.

Open Schematic View (Cone)

The **Cone** view focuses on the local logic cone around the selected object. Selecting *Open Schematic View (Cone)* opens a schematic that shows the immediate connectivity and surrounding logic relevant to the rule occurrence.

The Cone schematic view provides the following display options:

- *In Main* – Opens the schematic view in the main GUI window
- *In New* – Opens the schematic view in a new window
- *Append* – Appends the schematic content to an already open schematic viewer



Open Schematic View (Module)

The **Module** view displays the complete module containing the selected object. Selecting *Open Schematic View (Module)* opens the full module-level schematic, providing a broader structural context for the rule occurrence within the design hierarchy.

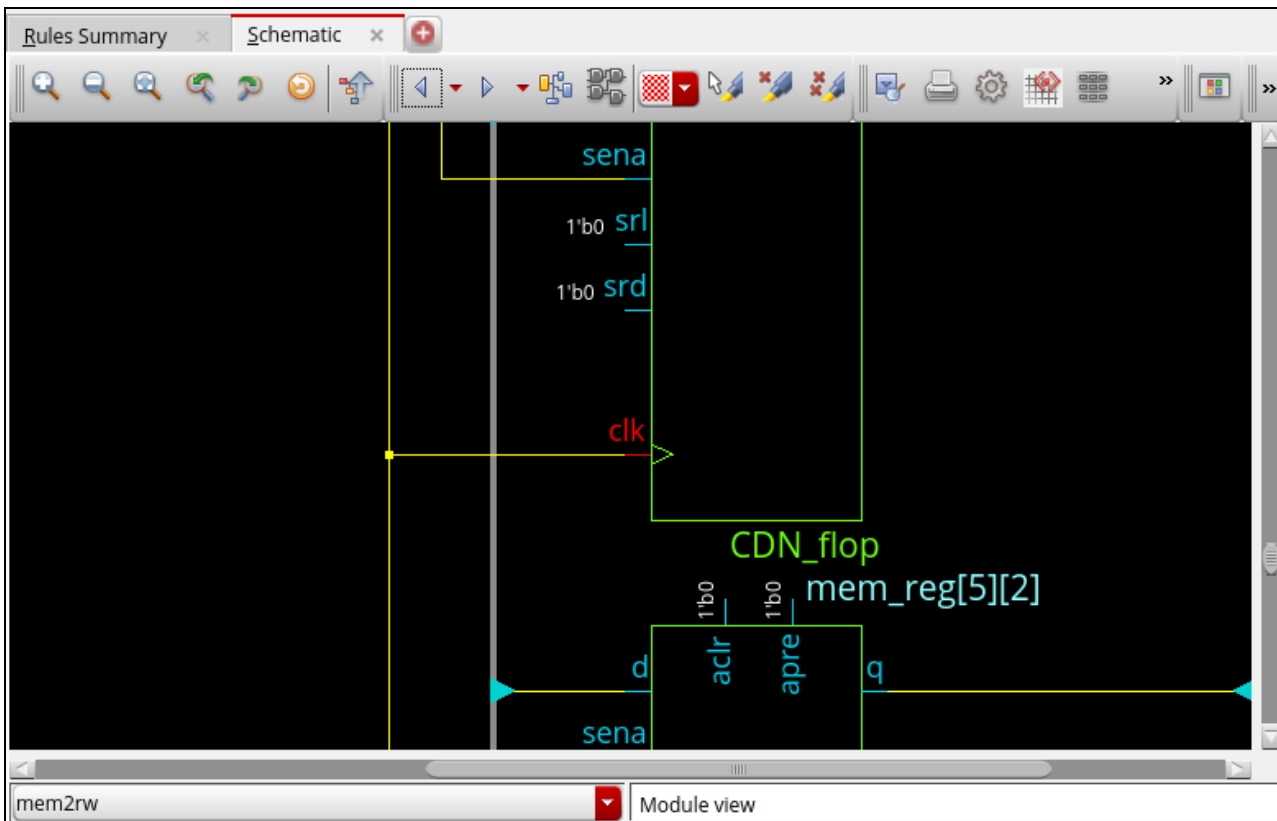
The Module schematic view provides the following display options:

- *In Main* – Opens the module schematic in the main GUI window
- *In New* – Opens the module schematic in a new window



Note:

An **Append** option is not provided for the Module view, as the entire module is already displayed as a single schematic entity.



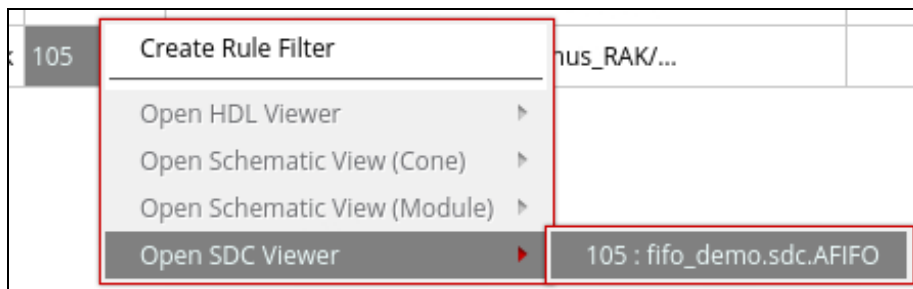
Cross-Probing to SDC

For rule occurrences that include SDC-related information (for example, an SDC file and line reference), the Litmus GUI supports cross-probing directly into the corresponding SDC constraint location. This capability helps to quickly inspect the constraint definition associated with a reported rule occurrence.

Cross-probing to SDC is invoked from the right-click context menu on a selected rule occurrence row in the Occurrences Table using the *Open SDC Viewer* option. In the context menu, the SDC entry is shown with the associated **line number and SDC file name** (for example, 105 :

`fifo_demo.sdc.AFIFO`).

When *Open SDC Viewer* is selected, the *SDC Viewer* opens and loads the referenced SDC file. The viewer automatically navigates to the referenced line number and highlights the corresponding constraint line for easy identification. This allows you to validate the constraint content at the exact source location associated with the selected rule occurrence.



```

99 set_clock_transition -max 0.500 [get_clocks rclk]
100 set_clock_transition -min 0.250 [get_clocks rclk]
101
102 ## [demo] CONST_CLK_DEF1 : Clock is missing at clkrst/clkb_d2_reg/q (/Q) which drives clkrs
103 ## [demo] CONST_CLK_DEF40 : Generated clock source path is not found : clkb -> clkrst/mux2_
104
105 create_generated_clock -name wclk -divide_by 4 -source [get_ports clkb] [get_pins clkrst/mux
106 set_clock_uncertainty -setup 0.250 [get_clocks wclk]
107 set_clock_uncertainty -hold 0.250 [get_clocks wclk]
108 set_clock_latency -source -max 0.500 [get_clocks wclk]
109 set_clock_latency -source -min 0.250 [get_clocks wclk]
110 set_clock_latency -max 1.000 [get_clocks wclk]
111 set_clock_latency -min 0.500 [get_clocks wclk]
112 set_clock_transition -max 0.500 [get_clocks wclk]
113 set_clock_transition -min 0.250 [get_clocks wclk]
114
115 ## [demo] CONST_CLK_DEF1 : Clock is blocked by SCA at scan_mode and SS at clkrst/mux2_rclk/
116 ## [demo] CONST_CLK_DEF1 : Clock is blocked by SCA at scan_mode and SDT at clkrst/mux2_wclk
117
118 set_sense -stop_propagation -clocks rclk [get_pins clkrst/mux2_rclk/Q]
119 set_disable_timing [get_pins clkrst/mux2_wclk/Q]
120
121 create_clock -name vrcclk -period 10.000 -waveform [list 0.000 5.000]
122 set_clock_uncertainty -setup 0.250 [get_clocks vrcclk]
123 set_clock_uncertainty -hold 0.250 [get_clocks vrcclk]
124 set_clock_latency -source -max 0.000 [get_clocks vrcclk]
125 set_clock_latency -source -min 0.000 [get_clocks vrcclk]

```

Creating Rule Filters

When a rule reports a large number of occurrences, **rule filters** allow you to selectively filter specific occurrences without modifying the underlying rule results. Rule filters are created using the *Create Rule Filter* option available from the GUI context menu. It helps manage large violation sets by isolating occurrences of interest. From an analysis perspective, rule filters act as **occurrence-level waivers**, temporarily suppressing selected occurrences from active consideration while preserving the original rule evaluation data.

Creating Rule Filters from the Navigation Pane

A rule filter can be created at the rule level by right-clicking on a rule in the *Navigation Pane* and selecting *Create Rule Filter*.

Invoking the rule filter from the Navigation Pane opens the *Add Rule Filter* dialog for the selected rule. In this case, no specific occurrence is preselected by default, allowing you to define filter criteria from scratch. This is useful when you want to create a rule filter based on:

- A specific *Occurrence ID* entered manually, or
- A *Custom range* of occurrence IDs associated with the selected rule.

Name	Severity	Count	Status
CONSTRAINT_LINT			
CONSTRAINT_SIGNOFF			
CONST_SCA_CHK1		1	Fail
CONST_CLK_DEF1		153	Fail
CONST_CLK_DEF2		2	Fail
CONST_CLK_DEF13			Fail
CONST_CLK_DEF32			Fail
CONST_CLK_DEF34		1	Fail
CONST_CLK_DEF36		2	Fail
CONST_CLK_DEF38		1	Fail

Add Rule Filter

Filter Name

Criteria

Prefix (Optional)

☒ Occurrence Id

☐ Custom range. e.g. 1,4,5-12

☐ Write to File

☒ Execute Rule Filter

New

Open

Close

Apply

Creating Rule Filters from the Occurrences Table

Rule filters can also be created for a **specific occurrence** directly from the Occurrences Table. You can right-click a row corresponding to a particular occurrence and select *Create Rule Filter* from the context menu.

When a rule filter is created from a specific occurrence:

- The *Occurrence ID* corresponding to the selected row is preselected in the *Add Rule Filter* dialog.
- The preselected *Occurrence ID* can be edited if required, allowing you to refine or expand the filtering scope with *Custom range*.

This is particularly useful when you identify a specific occurrence in the Occurrences Table that needs to be filtered.

CONST_CLK_DEF1

Filter...

occurrence_id	scope	witness
1	fifo	pin:fifo/mem2rw/mem_reg\[14\]\[4\]/clk
2	fifo	pin:fifo/wctrl/sync/sdata_q1_reg\[4\]/clk
3		n_reg\[6\]\[4\]/clk
4		n_reg\[2\]\[4\]/clk
5		n_reg\[10\]\[4\]/clk
6		n_reg\[8\]\[4\]/clk
7	fifo	pin:fifo/mem2rw/mem_reg\[0\]\[4\]/clk
8	fifo	pin:fifo/mem2rw/mem_reg\[12\]\[4\]/clk

Add Rule Filter ×

Filter Name

Criteria

Prefix (Optional)

☒ Occurrence Id
 ☐ Custom range. e.g. 1,4,5-12

☐ Write to File

☒ Execute Rule Filter

Add Rule Filter Dialog

The **Add Rule Filter** dialog provides options to define the filter criteria and specify how the rule filter should be handled.

The dialog includes the following fields and options:

- *Filter Name* – Displays the default filter name generated based on the selected rule and occurrence.
- *Criteria*
 - *Prefix (Optional)* – Allows you to specify an optional prefix for the rule filter name.
 - *Occurrence ID* – Allows filtering of a specific occurrence by its ID. When the rule filter is created from the Occurrences Table, this field is auto-populated with the selected

occurrence ID.

- *Custom range* – Allows you to specify a range of occurrence IDs (for example, 1, 4, 5-12) to be filtered.
- *Write to File* – Saves the generated rule filter expression to a file for reuse in future sessions.
- *Execute Rule Filter* – Executes the rule filter immediately in the current GUI session.
 - When *Execute Rule Filter* is enabled and the dialog is applied, the rule filter is executed in the current session.

When the *Apply* action is performed, a confirmation message is displayed indicating that the rule filter has been applied. Once executed, the effect of the rule filter can be observed using the filtering controls available in the Occurrences Table, as described earlier in the section [Filtering in the Occurrences Table](#).

SVA Generation for Functional Checks

Litmus generates System Verilog Assertions (SVA) corresponding to timing constraints. These SVAs help to ensure the constraint's functional correctness.

The following constraints are supported for generating SVAs:

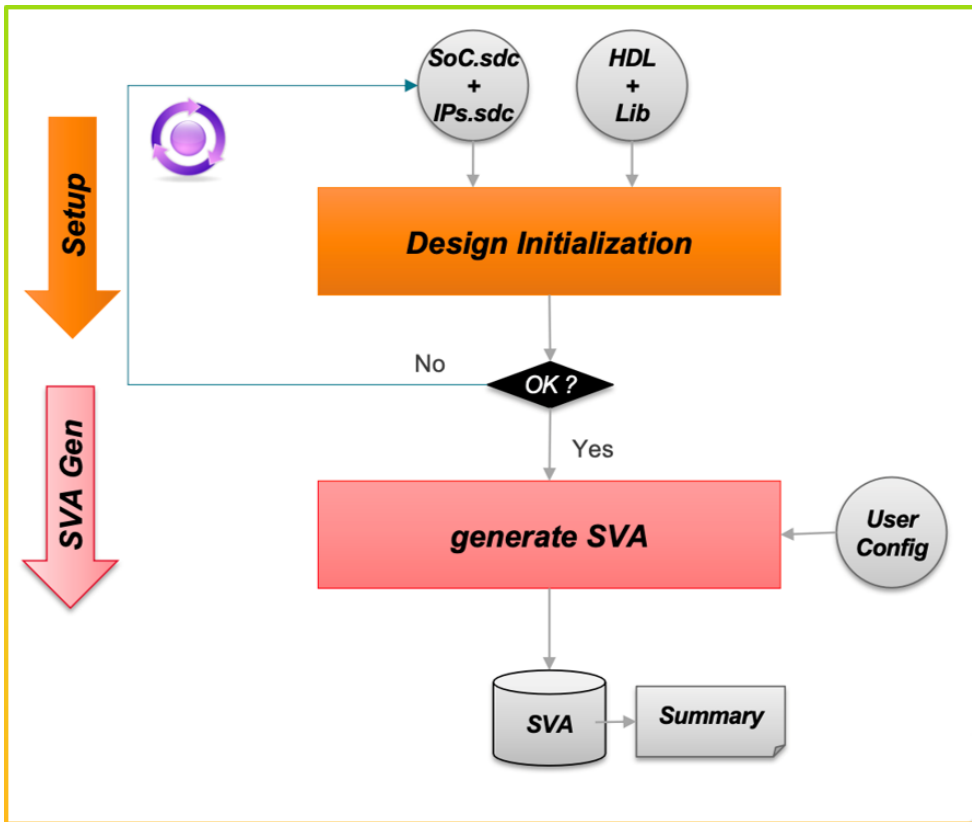
- [Set False Path \(set_false_path\)](#)
- [Set Multicycle Path \(set_multicycle_path\)](#)
- [Generated Clock \(create_generated_clock\)](#)
- [Set Case Analysis \(set_case_analysis\)](#)
- [Set Disable Timing \(set_disable_timing\)](#)

SVA Generation Flow

The process of SVA generation is covered in two phases:

1. Design Initialization
2. Generate SVA

The following figure shows the SVA Generation Flow:



⚠ SVA generation flow does not validate constraints.

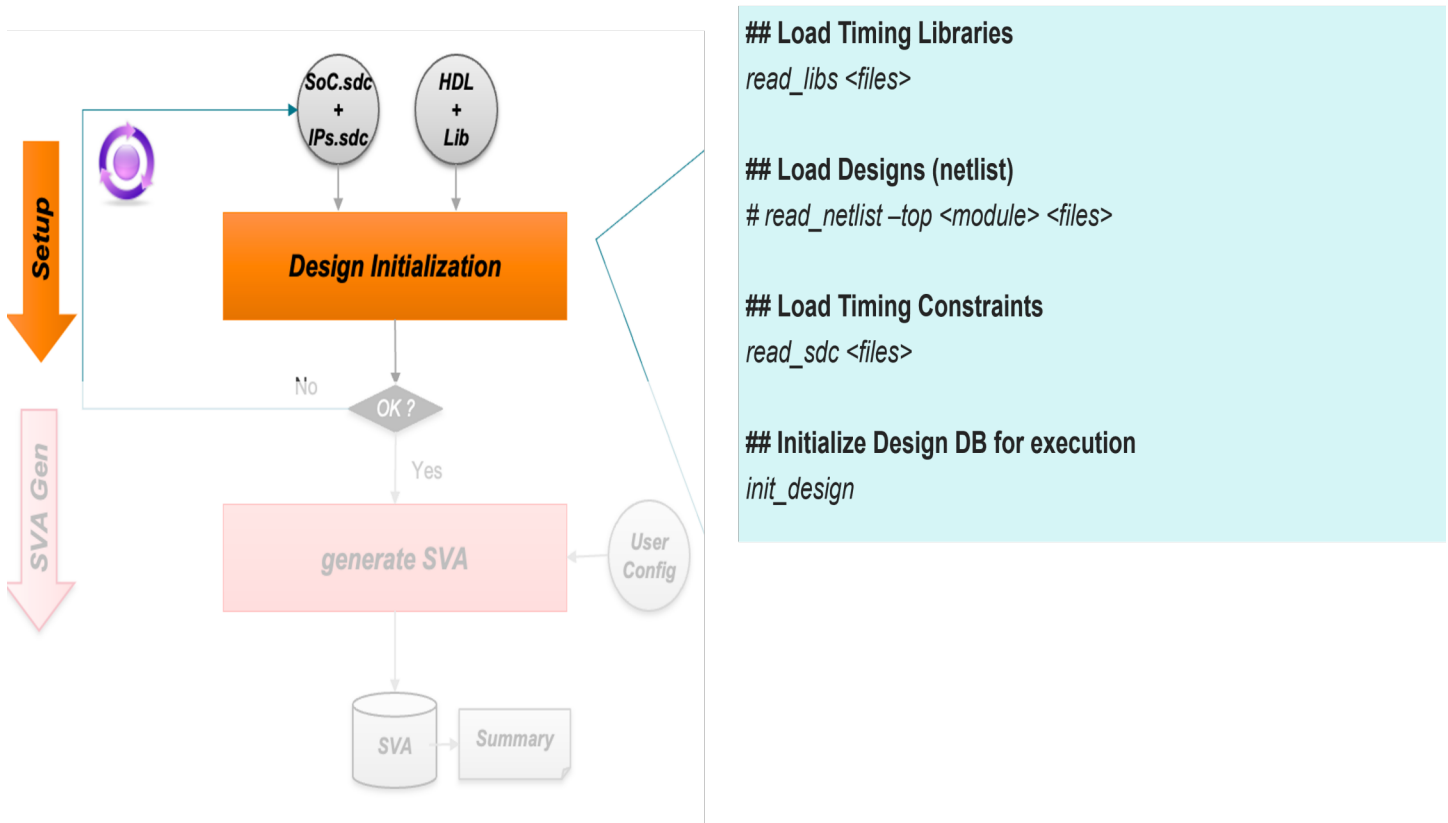
It is recommended to clean up constraints using Litmus rule checking before generating SVA.

Related Topics

- [Design Initialization for SVA](#)
- [Generate SVA](#)

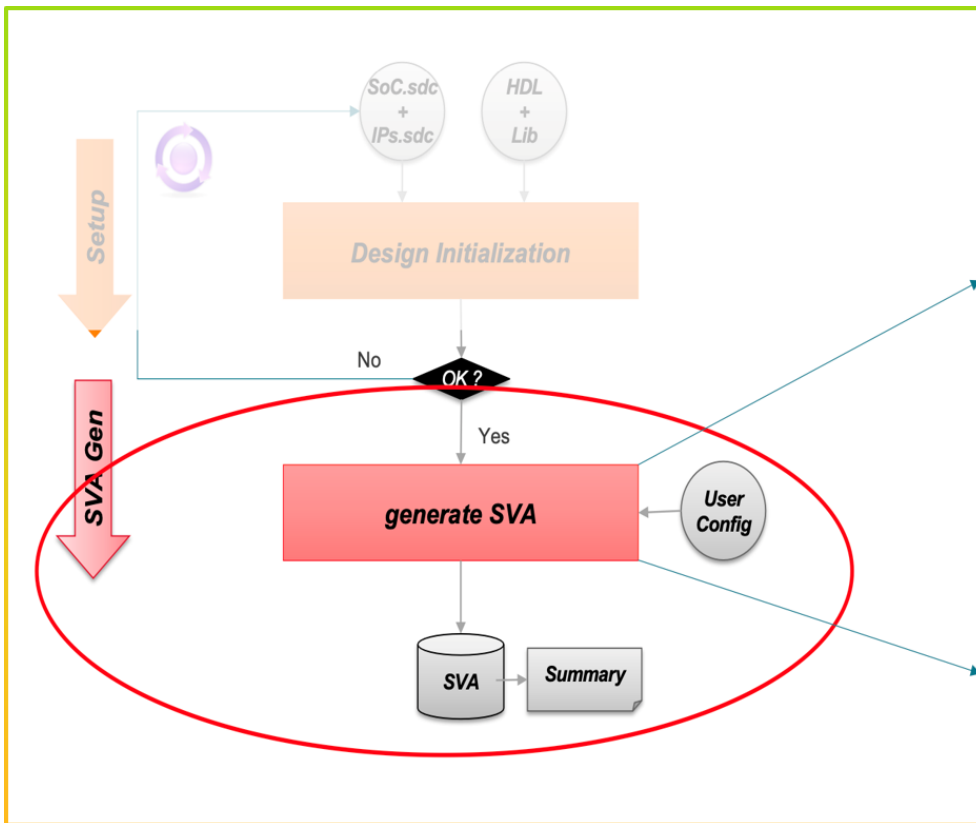
Design Initialization for SVA

The design initialization steps for SVA are same as followed for Litmus SDC checks and work for CUI mode only.



Generate SVA

SVA generation can be done after design initialization is complete.



Generation of SVAs can be configured use these [root attributes](#).

Constraints for SVA

The following constraints are supported for SVA generation:

- [Generated Clock \(create_generated_clock\)](#)
- [SFP and MCP Exceptions](#)
- [Set Case Analysis \(set_case_analysis\)](#)
- [Set Disable Timing \(set_disable_timing\)](#)

Generated Clock (create_generated_clock)

Generated (or derived) clock definitions in timing constraints are assumed behavior of the clock.

Logic that drives the generated clock definition object is assumed to create the frequency (clock edges) as defined by `create_generated_clock`.

The following command option is used to write assertions for generated clocks: `generate_sva -cgc`

CGC check is triggered only when generating clock toggles.

Master Clock	Generated Clock	SVA assertions
toggles	toggles	Triggered to check waveform correlation
toggles	constant	Not triggered
constant	toggles	Triggered and should report failure
constant	constant	Not Triggered

Assertions are composed of two parts

- Constant check - Ensures that a `gclk` toggle is caused by `mclk` toggle
- Waveform check - Checks the correlation between `mclk` and `gclk`

Examples

- Constant Check

```
// master clock: CLK0 at pin clk0
// generated clock: GCLK41 at pin mx1/Y
timeunit 1ns;
bit clk1ns;
initial forever #0.5 clk1ns=!clk1ns;

GCLK41___CGC_DIVIDED_BY_CONST_rise_fall: assert property( disable
iff(`CFM_RESET)
@(clk1ns) ##2 $rose(`mx1/Y) && (`clk0) |-> $past (`clk0) == 1`b0);

GCLK41___CGC_DIVIDED_BY_CONST_rise_rise: assert property (disable
iff(`CFM_RESET)
@(clk1ns) ##2 $rose(`mx1/Y) && (!`clk0) |-> $past (`clk0) == 1`b1);

GCLK41___CGC_DIVIDED_BY_CONST_fall_fall: assert property (disable
iff(`CFM_RESET)
@(clk1ns) ##2 $fell(`mx1/Y) && (`clk0) |-> $past (`clk0) == 1`b0);

GCLK41___CGC_DIVIDED_BY_CONST_fall_rise: assert property (disable
iff(`CFM_RESET)
@(clk1ns) ##2 $fell(`mx1/Y) && (!`clk0) |-> $past (`clk0) == 1`b1);
```

At every clk1ns toggle, if a rise of mx1/Y is observed and clk0 is high, it implies that the previous clk0 should be low.

- **Waveform Check**

```
// master clock: CLK0 at pin clk0
// generated clock: GCLK41 at pin mx1/Y

GCLK2__edge_1_4_5_6_9_rise : assert property (disable iff(`CFM_RESET)
@(`\clk0 ) ##2 $rose(`mx1/Y) |->
(##1 @(`\clk0) `mx1/Y == 1 ##1 (`\clk0) `mx1/Y = 1 ##1 (`\clk0) `mx1/Y = 0
##1 (`\clk0) `mx1/Y = 1 ##1 (`\clk0) `mx1/Y = 0 ##1 (`\clk0) `mx1/Y = 0 ##1
(`\clk0) `mx1/Y = 0 ##1 (`\clk0) `mx1/Y = 1 )
or
(##1 @(`\clk0) `mx1/Y == 0 ##1 (`\clk0) `mx1/Y = 0 ##1 (`\clk0) `mx1/Y = 0
##1 (`\clk0) `mx1/Y = 1 ##1 (`\clk0) `mx1/Y = 1 ##1 (`\clk0) `mx1/Y = 1 ##1
(`\clk0) `mx1/Y = 0 ##1 (`\clk0) `mx1/Y = 1 );

GCLK2__edge_1_4_5_6_9_fall : assert property (disable iff(`CFM_RESET)
@(`\clk0 ) ##2 $fell(`mx1/Y) |->
(##1 @(`\clk0) `mx1/Y == 1 ##1 (`\clk0) `mx1/Y = 0 ##1 (`\clk0) `mx1/Y = 0
##1 (`\clk0) `mx1/Y = 0 ##1 (`\clk0) `mx1/Y = 1 ##1 (`\clk0) `mx1/Y = 1 ##1
(`\clk0) `mx1/Y = 1 ##1 (`\clk0) `mx1/Y = 0)
or
(##1 @(`\clk0) `mx1/Y == 0 ##1 (`\clk0) `mx1/Y = 0 ##1 (`\clk0) `mx1/Y = 1
##1 (`\clk0) `mx1/Y = 1 ##1 (`\clk0) `mx1/Y = 1 ##1 (`\clk0) `mx1/Y = 0 ##1
(`\clk0) `mx1/Y = 1 ##1 (`\clk0) `mx1/Y = 0 );
```

SFP and MCP Exceptions

Litmus can generate SVAs for `set_false_path` and `set_multicycle_path` exceptions in user constraints. These SVAs can then be included in the testbenches for functional verification, which can be verified using simulators like Cadence Xcelium.

Litmus samples each timing path where the exception is applied and generates assertions on the selected timing points.

There are three levels of optimization available to control the number of sampling points in assertions for a specific exception:

- Level 1 : Assertions are applied on the following timing points:
 - Output pins of multi-input cells
 - Star/endpoint of timing path
 - Points (pins) specified in the constraint command
- Level 2 : Optimizes the number of sample points by removing points, paths through which converge downstream. Subsequently, keep only the convergence point.

- Level 3 : Asserts only the points (pins) specified in the constraint command

Notes:

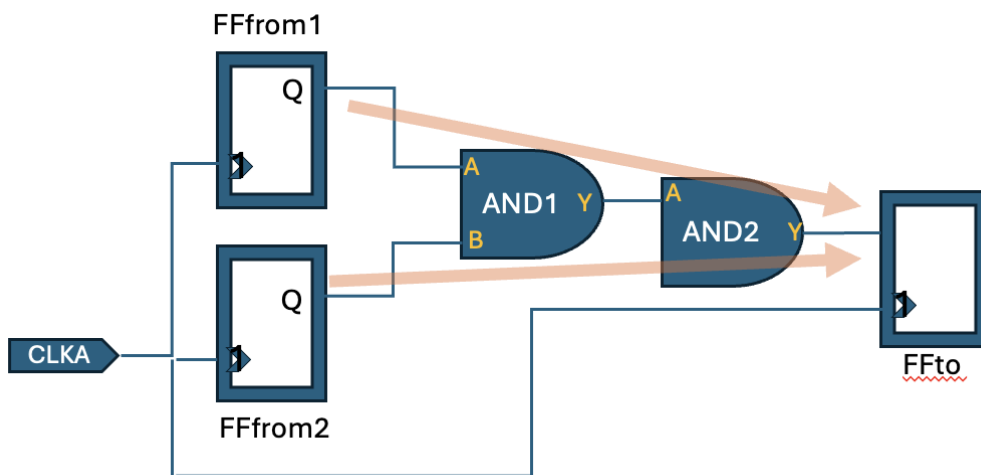
1. Buffer pins are excluded from assertions
2. If assertion points to multiple timing paths are the same set, only one assertion is generated for all those timing path.

Set Multicycle Path (set_multicycle_path)

This topic details the path-based sampling methods for multicycle path.

- [Level 1](#)
- [Level 2](#)
- [Level 3](#)

Level 1



Constraint: `set_multicycle_path -setup 2 -to [get_clocks CLKA] -through [FFto/D]]`

Litmus command used:

```
# This is default
set_db ltms_sva_path_based_sampling_opt_level 1

# Generate SVAs
generate_sva -mcp
```

Output SVA

```
mcp.sv
assert property (mcp_stable(`CLKA, `RESETA, $stable(FFto.D) || $stable(AND2.Y) ||
$stable(AND1.Y) || $stable(FFfrom1.Q),2))
assert property (mcp_stable(`CLKA, `RESETA, $stable(FFto.D) || $stable(AND2.Y) ||
$stable(AND1.Y) || $stable(FFfrom2.Q),2))
property mcp_stable(CLK, RESET, STABLE_PATH, CYCLES);
    @(posedge CLK) disable iff (RESET)
        ##1 !STABLE_PATH ==> STABLE_PATH[*CYCLES-1];
endproperty
```

Level 2

If there are transition points with the same value (one-to-one connection), Litmus merges them and retains output pins or pins specified in constraint.

Exception: `set_multicycle_path -setup 2 -to [get_clocks CLKA] -through [FFto/D]`

Litmus command used:

```
# Enable Mode 2
set_db ltms_sva_path_based_sampling_opt_level 2

#Generate SVAs
generate_sva -mcp
```

Output SVA

```
mcp.sv
assert property (mcp_stable_cycle(`CLKA, `RESETA, $stable(FFto.D) || $stable(AND1.Y)
|| $stable(FFfrom1.Q),2))
assert property (mcp_stable_cycle(`CLKA, `RESETA, $stable(FFto.D) || $stable(AND1.Y)
|| $stable(FFfrom2.Q),2))
property mcp_stable_cycle(CLK, RESET, STABLE_PATH, CYCLES);
    @(posedge CLK) disable iff (RESET)
        ##1 !STABLE_PATH ==> STABLE_PATH[*CYCLES-1];
endproperty
```

Level 3

Litmus only samples the points specified in the SDC constraint command.

Exception: `set_multicycle_path -setup 2 -to [get_clocks CLKA] -through [FFto/D]]`

Litmus command used:

```
# Enable SDC Mode
set_db ltms_sva_path_based_sampling_opt_level 3

#Generate SVAs
generate_sva -mcp
```

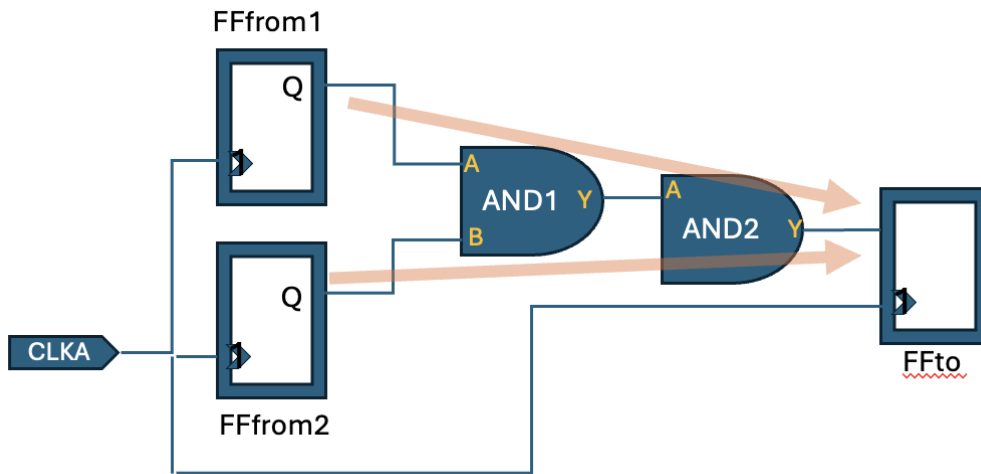
Output SVA

```
mcp.sv
assert property (mcp_stable_cycle(`CLKA, `RESETA, $stable(FFto.D))
property mcp_stable_cycle(CLK, RESET, STABLE_PATH, CYCLES);
  @(posedge CLK) disable iff (RESET)
    ##1 !STABLE_PATH |=> STABLE_PATH[*CYCLES-1];
endproperty
```

Set False Path (set_false_path)

This topic details the path-based sampling methods for set false path.

-
- [Level 1](#)
- [Level 2](#)
- [Level 3](#)



Level 1

Constraint: `set_false_path -to [get_clocks CLKA] -through [Ffto/D]`

Litmus commands used:

```
# This is default
set_db ltms_sva_path_based_sampling_opt_level 1

# Generate SVAs
generate_sva -sfp
```

Output SVA

```
sfp.sv
assert property (sfp_stable_cycle(`CLKA, `RESETA, $stable(Ffto.D) ||
$stable(AND2.Y) || $stable(AND1.Y) || $stable(FFfrom1.Q),2))
assert property (sfp_stable_cycle(`CLKA, `RESETA, $stable(Ffto.D) ||
$stable(AND2.Y) || $stable(AND1.Y) || $stable(FFfrom2.Q),2))
property sfp_stable_cycle(CLK, RESET, STABLE_PATH, CYCLES);
  @(posedge CLK) disable iff (RESET)
  ##1 !STABLE_PATH ==> STABLE_PATH[*CYCLES-1];
endproperty
```

Level 2

If there are transition points with the same value (one-to-one connection), Litmus merges them and retains output pins or pins specified in constraint.

Exception: `set_false_path -from [get_clocks CLKA] -to [get_clocks CLKB] -through [FFto/D]]`

Litmus commands used:

```
# Enable Mode 2
set_db ltms_sva_path_based_sampling_opt_level 2

#Generate SVAs
generate_sva -sfp
```

Output SVA

```
sfp.sv
assert property (sfp_stable_cycle(`CLKB, `RESETA, $stable(FFto.D) || $stable(AND1.Y)
|| $stable(FFfrom1.Q),2))
assert property (sfp_stable_cycle(`CLKB, `RESETA, $stable(FFto.D) || $stable(AND1.Y)
|| $stable(FFfrom2.Q),2))
property sfp_stable_cycle(CLK, RESET, STABLE_PATH, CYCLES);
    @(posedge CLK) disable iff (RESET)
        ##1 !STABLE_PATH | => STABLE_PATH[*CYCLES-1];
endproperty
```

Level 3

Litmus only samples the points specified in the SDC constraint command.

Exception: `set_false_path -from [get_clocks CLKA] -through [FFto/D]]`

Litmus commands used:

```
# Enable SDC Mode
set_db ltms_sva_path_based_sampling_opt_level 3

#Generate SVAs
generate_sva -sfp
```

Output SVA

```
sfp.sv
assert property (sfp_stable_cycle(`CLKA, `RESETA, $stable(FFto.D))
property sfp_stable_cycle(CLK, RESET, STABLE_PATH, CYCLES);
  @(posedge CLK) disable iff (RESET)
  ##1 !STABLE_PATH | => STABLE_PATH[*CYCLES-1];
endproperty
```

Set Case Analysis (set_case_analysis)

Signal value of the definition point should always be equal to the set value.

An example below:

```
set_case_analysis 0 [get_pins select]
```

#	Condition	Pass / Fail
1.	signal at 'select' pin is fixed to 0	Pass
2.	signal at 'select' pin is fixed to 1	Fail
3.	signal at 'select' pin toggles from 0 to 1	Fail
4.	signal at 'select' pin toggles from 1 to 0	Fail
5.	signal at 'select' pin toggles from 0 to 1 to 0 outside of the verification period and remains fixed at 0 thereafter	Pass

Verilog testbench generation for `set_case_analysis` does not support the following options: rise, rising, fall, falling, non-switching

Set Disable Timing (set_disable_timing)

For each instance impacted with SDT, check that the disabled arcs never toggle.

The following command option is used for checking assertions: `generate_sva -sdt`

Examples

- `set_disable_timing [get_pins INST/A]`
SVA for arc from "INST/A" to "INST/Y" will be generated.
- `set_disable_timing [get_pins INST/Y]`
SVA for arc from "INST/A" to "INST/Y" and from "INST/B" to "INST/Y" will be generated.
- `set_disable_timing [get_cells INST] -from A -to Y`
SVA for arc from "INST/A" to "INST/Y" will be generated.
- `set_disable_timing [get_cells INST]`
SVA for arc from "INST/A" to "INST/Y" and from "INST/B" to "INST/Y" will be generated.

CUI Scripting Capability

Litmus Stylus CUI provides shared setup and reporting with Genus, Innovus, and Tempus.

Additional information for supported objects and attributes is available in [Litmus Attribute Reference](#).

Templates

- [RTL Signoff Template](#)
- [Netlist Signoff Template](#)
- [Hierarchical Check Template](#)

RTL Signoff Template

```
input_sdc {top.sdc block.sdc ...}  
litmus_setup_mmmc  
elaborate_design -script load_rtl.tcl  
init_design
```

The file `load_rtl.tcl` should contain the loading commands for RTL files, followed by an elaborate command:

```
read_hdl top.v  
read_hdl ...  
elaborate top
```

Netlist Signoff Template

```
set libdir lib
set net <design>.v
set sdc <design_mode>.sdc

set top test

read_lib ${libdir}/stdells.lib.gz

read_netlist $net -top $top

read_sdc $sdc

init_design

report_sdc

report_rule_check -category constraint_lint
verify_timing_intent

report_rule_check -verbose -category constraint_signoff
```

Hierarchical Check Template

```
read_libs tutorial.lib
set_target_check_constraints -instance i_blk_0 -files blk_0.sdc
read_netlist -top top top.v
read_sdc top.sdc
init_design
check_target_constraints -instances "i_blk_0"
report_rule_check CONST_HIER_* -verbose -attribute
```

